Threshold Batch Identity-based Encryption without Epochs

Dan Boneh, Evan Laufer, and Ertem Nusret Tas

Stanford University

Abstract. Suppose Alice holds a master secret key sk in an identity-based encryption scheme. For a given set of identities $\mathcal{I}$, Alice wants to create a short pre-decryption key that lets anyone decrypt the ciphertexts encrypted under any identity in $\mathcal{I}$ and nothing else. This problem is called *batch identity-based encryption (batch IBE)*. When the secret key sk is shared among a number of decryption parties, the problem is called *threshold batch identity-based encryption*. This question comes up in the context of an encrypted mempool where the goal is to publish a short pre-decryption key that can be used to decrypt all ciphertexts in a block. Prior work constructed batch IBE schemes with some limitations.
In this work, we construct new batch IBE and threshold batch IBE schemes. We first observe that a key-policy ABE (KP-ABE) scheme directly gives a batch IBE scheme. However, the best KP-ABE schemes happen to be lattice-based, and do not thresholdize efficiently. We then use very different techniques to construct a new lattice-based batch IBE scheme. Our construction employs a recent preimage sampler due to Waters, Wee, and Wu. Along the way, we develop a two-round protocol for preimage lattice sampling when the lattice trapdoor is secret shared among the participants.

Table of Contents

1 Introduction

A batch identity-based encryption (batch IBE, or BIBE for short) scheme is a tuple of four PPT algorithms (Setup, Enc, PreDec, Dec) where

- $\text{Setup}(1^\lambda, 1^\ell) \to (\mathsf{pk}, \mathsf{sk})$: takes as input a maximum batch size ℓ and outputs a key pair pk, sk;
- $\text{Enc}(\mathsf{pk}, r, m) \to \mathsf{ct}$: encrypts m with respect to a unique identity tag r;
- $\text{PreDec}(\mathsf{sk}, (r_i)_{i\in B}) \to \mathsf{sbk}$: takes as input a tuple of up to ℓ pairwise-distinct identity tags and outputs a corresponding short pre-decryption key sbk; and
- $\text{Dec}(\mathsf{pk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in B}) \to (m_i)_{i\in B}$: uses sbk to decrypt a batch of ciphertexts $(\mathsf{ct}_i)_{i\in B}$. Decryption should succeed if ct_i was properly encrypted using the identity tag r_i, for all $i \in B$.

In its simplest form, the encryptor can sample the identity tag r at random from a sufficiently large space to ensure that r is unique with high probability, assuming all encryptors sample r at random. The main property of batch IBE is that the pre-decryption key sbk is short — its length should be at most polylog in the maximum batch size ℓ, and preferably independent of the batch size. This short pre-decryption key enables the decryption of every ciphertext that was encrypted with respect to an identity tag in $\{r_i\}_{i\in B}$, and nothing else. We give a precise definition in Section 3.1. Batch IBE is useful whenever many ciphertexts need to be decrypted concurrently by a third party. This most notably comes up in the context of an encrypted mempool, as explained in Section 1.1 below.

Batch IBE was introduced by Choudhuri, Garg, Piet, and Policharla [29] under the name *batch encryption*. In their scheme, however, each message was encrypted with respect to both an identity tag r and an epoch id e, and only a *single* pre-decryption key could be published for any epoch e. Publishing more than one key per epoch, say for two distinct sets B_1 and B_2, would compromise security. This is called epoch-based batch encryption, and as we explain below, it makes the scheme difficult to use in the context of an encrypted mempool. The original scheme required a new setup at the beginning of every epoch, but this was later improved [59,4,30] to a single one-time setup at the beginning of time (but still only one pre-decryption key is allowed per epoch).

The first batch encryption scheme without epochs, called BEAT-MEV, was proposed by Bormet, Faust, Othman, and Qu [26]. BEAT-MEV uses an alternative syntax, where PreDec takes as input the set of full ciphertexts to decrypt, namely, $\text{PreDec}(\mathsf{sk}, \{\mathsf{ct}_i\}_{i\in B}) \to \mathsf{sbk}$, and sbk decrypts only the ciphertexts in $\{\mathsf{ct}_i\}_{i\in B}$ and nothing else. Thus, BEAT-MEV is a batch encryption primitive, which can also be built using batch IBE combined with a one-time signature [17]. Notably, BEAT-MEV does not need epoch ids, allowing pre-decryption keys to be published for an unlimited number of batches. However, each transaction must be encrypted with respect to a certain *index* value in $\mathbb{N}$, and pre-decryption keys can be generated only for batches containing ciphertexts with distinct indices. The indices are different from the identity tags employed by the batch IBE constructions: in batch IBE, once a pre-decryption key is released for a set containing an identity tag, ciphertexts that reuse the same tag no longer enjoy confidentiality guarantees. In contrast, in BEAT-MEV, ciphertexts in different batches can share the same index values, and publishing pre-decryption keys for a batch does not compromise the security of the ciphertexts in another batch. As an independent contribution, we suggest a plausibly post-quantum secure lattice-based version of BEAT-MEV in Section A.4 using the BLMR key homomorphic PRF [21].

The difficulty with BEAT-MEV is that it has a large public key that scales with the size of the index space. One can shrink the public key by restricting the index space to the maximum batch size ℓ; however, this would require the parties submitting ciphertexts to a batch to coordinate in advance to assign a unique index for each ciphertext. BEAT-MEV removes the coordination

requirement by assigning transactions to multiple *sub-batches* such that within any sub-batch, all indices are distinct. When the encryptors select the indices uniformly at random from $\{1, \ldots, \ell\}$, with high probability, there are at most $O(\log \ell)$ ciphertexts with the same index, and BEAT-MEV has to publish $O(\log \ell)$ pre-decryption keys, one per sub-batch. However, this introduces a censorship vulnerability: an attacker can choose its index values adversarially and submit $\Omega(\log \ell)$ ciphertexts to a batch, all with the same index. Then, either there must be $\Omega(\log \ell)$ sub-batches and keys to accommodate all the ciphertexts sharing the same index, or most transactions sharing the common index must be excluded from this batch. This censorship vulnerability is shared by other *index-dependent* batch encryption schemes that build on BEAT-MEV [2,25,40].

The first epochless batch (threshold) IBE scheme was given by TrX [42], who built on the earlier batch IBE primitives in [4,30] (thus, like these earlier schemes, TrX is also index-independent). Although its public keys grow linearly with the number of batches, in practice, TrX enables reusing a shorter public key in the context of PoS blockchains that periodically run a fresh DKG. The first index-independent batch encryption schemes were presented in [23,53,3]. However, their constructions specifically satisfy a batch encryption primitive rather than batch IBE.

Our contributions. Our main contribution is an *epochless* batch IBE scheme with a succinct public key, ciphertexts, and pre-decryption keys. Our main construction is lattice-based, which makes it plausibly post-quantum secure.

In Section 5, we present our batch IBE scheme based on the Learning with Errors (LWE) assumption. Our scheme can be viewed as a composition of the lattice IBE scheme in [27] (in the random oracle model) with the shifted multi-preimage trapdoor sampler of Waters, Wee, and Wu [62, §6]. The lattices in our construction have two parts: a trapdoor for the left lattice is used to generate the pre-decryption keys, while trapdoors for the right lattice, obtained by programming the random oracle, enable simulation of pre-decryption keys for all batches excluding the adversary's challenge identity.

In Section 6, we present an efficient, two-round protocol for thresholdizing the pre-decryption algorithm of our LWE-based batch IBE scheme. The protocol enables parties to sample short pre-decryption key shares using only a Shamir secret share of a lattice trapdoor. The same protocol also yields efficient, two-round, threshold GPV signatures [43] (Section 7).

In Section 8, we observe that a batch IBE scheme can be built generically using key-policy attribute-based encryption (KP-ABE), constructable from LWE and its variants [19,31,32,63]. Recent KP-ABE schemes improve the asymptotics of this generic approach (Table 1): Cini and Wee [32] removes the attribute-length dependence from the public key under plain LWE, while Wee [63] removes the attribute-length dependence from the public key, ciphertext, and secret key under succinct LWE. However, it is unclear how to thresholdize the KP-ABE based constructions.

For completeness, in Section 9, we present a very efficient batch IBE system constructed using a trilinear map. We build on the aggregate IBE scheme introduced by Goyal et al. [45, §7]. The trilinear map is used to shrink the linear (ℓ) sized ciphertext of the pairing-based scheme in [45, §7]. The result achieves optimal parameter sizes: a single group element for the pre-decryption key, and the ciphertext is the same length as the plaintext plus a single group element.

1.1 An Application to Encrypted Mempools

When a transaction is posted to a blockchain, it first goes into a public mempool and waits there until it is eventually included in a block that is committed on chain. While the transaction is in the

	$\lvert\mathsf{pk}\rvert$	$\lvert\mathsf{sbk}\rvert$	$\lvert\mathsf{ct}\rvert$
KP-ABE by BGG+14 [19]	$O_\lambda(k \log^2 \ell)$	$O_\lambda(\log^2 \ell)$	$O_\lambda(k \log^2 \ell)$
KP-ABE by CW23 [31]	$O_\lambda(k \log^2 \ell)$	$\mathsf{poly}(\lambda)$	$O_\lambda(k \log^2 \ell)$
KP-ABE by Cini–Wee24 [32]	$\mathsf{poly}(\lambda, \log \ell)$	$\mathsf{poly}(\lambda, \log \ell)$	$O_\lambda(k \log^2 \ell)$
KP-ABE by Wee25 [63]	$\mathsf{poly}(\lambda, \log \ell)$	$\mathsf{poly}(\lambda, \log \ell)$	$\mathsf{poly}(\lambda, \log \ell)$
Our BIBE Scheme from LWE	$O_\lambda\left(k \log^2 \ell\right)$	$O_\lambda\left(\log \ell\right)$	$O_\lambda\left(k \log^2 \ell\right)$
Our BIBE Scheme from Trilinear Map	$\lvert\mathbb{G}_1'\rvert + (\ell - 1)\lvert\mathbb{G}_2'\rvert$	$\lvert\mathbb{G}_3'\rvert$	$\lvert\mathbb{G}_1'\rvert + O_\lambda(1)$

Table 1: Comparison of the parameter sizes of epoch-less batch IBE (BIBE) systems. $\lvert\mathsf{pk}\rvert$ denotes the total size of the public parameters of the scheme (not the size of a single identity tag or index value), $\lvert\mathsf{sbk}\rvert$ denotes the size of the pre-decryption key published per batch, and $\lvert\mathsf{ct}\rvert$ denotes the ciphertext size. We denote the security parameter by λ and the maximum batch size by ℓ. Identity tags are chosen from a set of size 2^k. To simplify the expressions, we assume that $\ell \geq \lambda$.

mempool, it is visible to the world and this can lead to front-running attacks that cause financial loss for the transaction issuer. This issue is called MEV [35]. An encrypted mempool [55,9] is a way to protect transactions while they are in the mempool. Instead of submitting transactions in the clear, the transaction issuer encrypts the transaction with a public key pk and submits it to the mempool. The corresponding secret key sk is shared among the nodes that operate the blockchain. Once a block is committed on chain, the nodes publish decryption shares that enable anyone to decrypt and execute all the transactions in that block. This way, a transaction becomes public only after it is included in a block that is committed on chain. Recent work [16] shows how to publish decryption shares concurrently with finalizing a block, eliminating an extra round for decryption. Encrypted mempools are already in use in some blockchains such as Osmosis, Shutter [41], and Aptos.

Batch encryption/IBE is a way to improve the efficiency of an encrypted mempool. With a standard public key encryption scheme, for every block, a blockchain node would need to publish ℓ decryption shares to enable the decryption of all ℓ transactions in a block. In other words, for every block, the node must publish $O(\lambda\ell)$ bits to enable decryption of that block. With threshold batch IBE, the node would publish a short pre-decryption key whose length is only $O(\lambda\ \mathsf{polylog}(\ell))$ or just $O(\lambda)$. Prior batch encryption/IBE schemes [30,29,4,59,26] were proposed for exactly this purpose. In epoch-based batch encryption/IBE, the block number (or height) would be used as the epoch. To use such a scheme, the transaction issuer would need to guess the block number where its transaction would land, and encrypt using that epoch (and the random identity tag). If the issuer guesses wrong, then the transaction would never be included in a block, and must be resubmitted. Epoch-free schemes eliminate this difficulty.

1.2 The Risk of Replicated Identities

A batch IBE scheme identifies a ciphertext by its identity tag r. This makes the encrypted mempool vulnerable to an attacker who replicates the identity tag of an honest user's transaction. Specifically, suppose an adversary observes in the mempool a ciphertext ct generated by an honest user under identity tag r. The adversary can create a *dummy* ciphertext encrypted under the *same* identity r. If this dummy ciphertext is included in a block but the honest user's ciphertext ct is not included, then the pre-decryption key generated for that block can be used to decrypt ct before it is committed on chain. This defeats the purpose of an encrypted mempool. Solutions to this problem were discussed in the earlier works [30,29,4,59,26] and also apply to our construction. For example, the transaction

issuer can generate a key pair for a signature scheme, use the public key as the identity tag, and post the encrypted transaction along with a signature on the encrypted transaction. The nodes will only include a transaction in a block if it has a valid signature with respect to its identity tag. This way, the adversary cannot post a dummy transaction with the same identity tag.

Of course, to prevent malleability attacks of a similar nature, it is crucial that batch IBE and decryption schemes provide CCA2 security. To this end, standard techniques (e.g., [17,15,16]) can be employed to elevate threshold CPA security of batch IBE to full threshold CCA2 security.

1.3 Additional Related Work

Early work on batch encryption [51,39] proposed to use Identity Based Encryption (IBE), where a transaction issuer uses an *epoch id* as the identity tag. Blockchain nodes then release the corresponding IBE secret key once the block at that epoch id is committed on chain. These mechanisms, however, lack the ability to selectively exclude ciphertexts from the batch, as well as any security guarantees for such excluded ciphertexts. Consequently, they are not suitable for encrypted mempools.

Practical IBE schemes have been realized under various cryptographic assumptions, including the bilinear Diffie-Hellman problem [18], quadratic residuosity modulo composites [33,20], and LWE [43,5]. The main difference between IBE and batch IBE is that in the former each identity is associated with a distinct secret key, whereas the latter seeks to obtain a single secret key, called the pre-decryption key, capable of decrypting ciphertexts associated with multiple identity tags (thus motivating the term 'batch' IBE [4]).

Gong, Waters, Wee, and Wu [44] obtain the first selectively-secure batch IBE scheme (with epochs) based on elliptic curves in the plain model under a q-type assumption. After an initial version of this work was publicized, Xiong, Zhang, and Chen [65] presented a lattice-based batch IBE scheme in the plain model with security based on ℓ-succinct LWE.

In concurrent work [47], Jiang, Wee, and Zhu present a lattice-based, partially non-interactive, threshold signature scheme, which uses similar techniques to our threshold batch IBE construction in Section 6.

2 Technical Overview

2.1 Warm-up: Batch IBE from Attribute-Based Encryption

Key-policy attribute-based encryption (KP-ABE) [56,46] generalizes public-key encryption by associating each secret key sk_f with a function $f : \mathcal{X} \to \mathcal{Y}$. Ciphertexts are labeled with a public attribute $x \in \mathcal{X}$ and can be decrypted using sk_f if and only if $f(x) = 0$.

Batch IBE can be obtained from any KP-ABE scheme by considering the special class of functions f_S that take an identity tag as input and output 0 exactly when the tag belongs to a designated set $S = \{r_i\}_{i\in[\ell]}$. Then, a ciphertext associated with identity tag r can be decrypted by the key sk_{f_S} if and only if $r \in S$. The key sk_{f_S} plays the role of a pre-decryption key for the batch S. We formalize this black-box transformation in Section 8.

For this approach to yield a succinct batch IBE scheme, the KP-ABE key for f_S must be short. The KP-ABE scheme of Boneh et al. [19], Cini and Wee [31], the unbounded ABE of Cini–Wee [32], and the almost-optimal ABE of Wee [63] all have sufficiently succinct keys for this transformation.

2.2 Our Batch IBE System

An alternative approach to building practical batch IBE systems is to extend an efficient IBE scheme to the batch setting [4,30,59]. For this purpose, earlier batch IBE constructions (with epochs) [4,30] exploit the observation that IBE implies a signature scheme [18, §6]. For the pre-decryption key, they generate a signature on an accumulator value committing to a batch of identity tags. Encryption is then implemented via witness encryption for the following two relations: (i) the signature verification relation induced by the underlying IBE scheme, and (ii) the membership verification relation for a given identity tag within the accumulator. Concrete constructions instantiate the signature with BLS signatures [22] and the accumulator with KZG commitments [48]. Their witness encryption component requires linear constraint systems in which witness group elements are never paired with one another. Enforcing this linearity condition requires the use of a 'one-time BLS signature' (without random oracles) tied to a specific 'epoch', thereby necessitating encryption with respect to both identity tags and a pre-determined epoch number – an essential aspect for preserving security in their constructions.

In our batch IBE system (without epochs), we overcome the limitation of witness encryption over linear constraint systems by moving to lattices. At a high level, our scheme combines the IBE system in [27] with the shifted multi-preimage trapdoor sampler of Waters, Wee, and Wu [62], which plays the role of the accumulator. As designing an end-to-end batch IBE scheme building on the 'correct' components was the main challenge of this work, we next briefly look at the steps taken towards the final scheme.

2.2.1 Shifted Multi-preimage Trapdoor Sampler by Waters, Wee, and Wu. The sampler enables obtaining a trapdoor for a collection of correlated matrices $\mathbf{A}_1, \ldots, \mathbf{A}_\ell \in \mathbb{Z}_q^{n \times (md+m)}$ with marginally uniform distributions, without any hints for $\mathbf{A}_i$, $i \in [\ell]$. Concretely, for every $i \in [\ell]$, let $\mathbf{A}_i \coloneqq [\mathbf{A}_0 \mid \mathbf{B}_0 - u_i \otimes \mathbf{G}]$, where $\mathbf{A}_0 \in \mathbb{Z}_q^{n \times m}$, $\mathbf{B}_0 \in \mathbb{Z}_q^{n \times md}$ are uniformly-random matrices, $u_i \in \{0,1\}^d$ are distinct bit vectors, and $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$ is the gadget matrix. Here, $d \geq \lceil \log \ell \rceil$. The sampler obtains a trapdoor $\mathbf{T}_\mathbf{A}$ for the matrix

$$\mathbf{A} \coloneqq \begin{bmatrix} \mathbf{A}_1 & & & \mathbf{G} \\ & \mathbf{A}_2 & & \mathbf{G} \\ & & \ddots & \vdots \\ & & \mathbf{A}_\ell & \mathbf{G} \end{bmatrix} \in \mathbb{Z}_q^{n\ell \times (\ell md + \ell m + m)} \tag{1}$$

from the public trapdoor $\mathbf{T}_\mathbf{G}$ of the gadget matrix $\mathbf{G}$. Here, the matrices $\mathbf{A}_0$ and $\mathbf{B}_0$ constitute the common reference string (CRS) of the trapdoor sampler. In our batch IBE scheme, ℓ will determine the maximum batch size.

Given $\mathsf{td} \coloneqq (\mathbf{A}, \mathbf{T}_\mathbf{A})$, one can then sample a shifted preimage for any column of vectors $(\mathbf{h}_1, \ldots, \mathbf{h}_\ell) \in \mathbb{Z}_q^{n\ell}$:

$$\mathrm{col}(\mathbf{v}_1, \ldots, \mathbf{v}_\ell, \hat{\mathbf{c}}) \leftarrow \mathsf{SampleMultPre}(\mathsf{td}, \mathbf{h}_1, \ldots \mathbf{h}_\ell) \in \mathbb{Z}_q^{\ell(md+m)+m},$$

where $\mathbf{A}_i \mathbf{v}_i + \mathbf{G}\hat{\mathbf{c}} = \mathbf{h}_i$ for all $i \in [\ell]$. Crucially for the security of our batch IBE system, the trapdoor sampler satisfies *somewhere programmability*: given a random matrix $\mathbf{A}_{i^*}$ for index i^*, it enables generating a CRS, indistinguishable from a uniformly random one, such that for the sequence of matrices $\tilde{\mathbf{A}}_1, \ldots, \tilde{\mathbf{A}}_\ell$ derived from this CRS, it holds that $\tilde{\mathbf{A}}_{i^*} = \mathbf{A}_{i^*}$.

2.2.2 The Base IBE Scheme. Let $H_{\mathsf{ro}} \colon \mathcal{I} \to \mathbb{Z}_q^{n \times \overline{m}}$ be a hash function that maps identity tags to matrices. For the security proof, H_{ro} will be modeled as a random oracle. The setup algorithm generates a matrix-trapdoor tuple $(\mathbf{C} \in \mathbb{Z}_q^{n \times m}, \mathbf{T_C})$ and a random vector $\mathbf{u} \in \mathbb{Z}_q^n$, and sets the master public key as $\mathsf{pk} \coloneqq (\mathbf{C}, \mathbf{u})$ and the master secret key as $\mathsf{sk} \coloneqq (\mathsf{pk}, \mathbf{T_C})$. To generate the secret key for an identity tag r, it computes $\mathsf{sk}_r \leftarrow \mathsf{SampleLeft}(\mathbf{C}, H_{\mathsf{ro}}(r), \mathbf{T_C}, \mathbf{u}, \sigma)$, where $\mathsf{SampleLeft}$ outputs a discrete Gaussian vector $\mathbf{v}$ with width σ such that $[\mathbf{C} \mid H_{\mathsf{ro}}(r)] \cdot \mathbf{v} = \mathbf{u}$.

To encrypt a bit $\mathsf{m} \in \{0,1\}$ under the identity tag r, the encryption algorithm generates a dual-Regev ciphertext $\mathsf{ct} \coloneqq (\mathsf{ct}[1], \mathsf{ct}[2])$ with $\mathbf{u}$ as the public key and $[\mathbf{C} | H_{\mathsf{ro}}(r)]$ as the public parameter:

$$\begin{aligned} \mathsf{ct}[1] &\coloneqq \mathbf{u}^\top \mathbf{s} + \boldsymbol{\epsilon}_1 + \mathsf{m} \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \mathsf{ct}[2] &\coloneqq ([\mathbf{C} | H_{\mathsf{ro}}(r)])^\top \mathbf{s} + \boldsymbol{\epsilon}_2 \in \mathbb{Z}_q^{m+\overline{m}}, \end{aligned}$$

where $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, and $\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2$ are low-norm noise vectors.

Finally, decryption outputs $\mathsf{Round}(\mathsf{ct}[1] - \mathsf{sk}_r^\top \mathsf{ct}[2])$, where $\mathsf{Round}(x)$ outputs 0 if $x \bmod q$ is closer to 0 than to $\lfloor q/2 \rfloor$.

2.2.3 An Index-Dependent Batch IBE Scheme. The setup algorithm generates a matrix-trapdoor tuple $(\mathbf{C} \in \mathbb{Z}_q^{n \times m}, \mathbf{T_C})$, a random vector $\mathbf{u} \in \mathbb{Z}_q^n$, and the crs for the shifted multi-preimage trapdoor sampler. It sets $\mathsf{pk} \coloneqq (\mathsf{crs}, \mathbf{C}, \mathbf{u})$ and $\mathsf{sk} \coloneqq (\mathsf{pk}, \mathbf{T_C})$.

To encrypt a bit $\mathsf{m} \in \{0,1\}$ with respect to an index $i \in [\ell]$ and an identity tag r, our scheme generates a dual-Regev ciphertext $\mathsf{ct} \coloneqq (\mathsf{ct}[1], \mathsf{ct}[2], \mathsf{ct}[3])$ with $\mathbf{u}$ as the public key and the matrix $\big[\mathbf{C} \mid \mathbf{A}_i \mid H_{\mathsf{ro}}(r)\big]$ as the public parameter. Here, the matrix $\mathbf{A}_i$ is obtained from the trapdoor sampler's crs in the public key pk. The resulting ciphertext is defined as:

$$\begin{aligned} \mathsf{ct}[1] &\coloneqq \mathbf{u}^\top \mathbf{s} + \boldsymbol{\epsilon}_1 + \mathsf{m} \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \mathsf{ct}[2] &\coloneqq \mathbf{C}^\top \mathbf{s} + \boldsymbol{\epsilon}_2 \in \mathbb{Z}_q^m, \\ \mathsf{ct}[3] &\coloneqq \big[\mathbf{A}_i \mid H_{\mathsf{ro}}(r)\big]^\top \mathbf{s} + \boldsymbol{\epsilon}_3 \in \mathbb{Z}_q^{md+m+\overline{m}}, \end{aligned} \tag{2}$$

where $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, and $\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2, \boldsymbol{\epsilon}_3$ are low-norm noise vectors.

Now, let $r_1, \ldots, r_\ell$ be a batch of identity tags, where r_i corresponds to index i. The pre-decryption algorithm first samples a vector $\mathbf{v} = \mathrm{col}(\mathbf{v}_1^1, \ldots \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$ that is a preimage of the vector $(\mathbf{u}, \ldots, \mathbf{u}) \in \mathbb{Z}_q^{n\ell}$ under the matrix

$$\big[\mathbf{A} \mid \hat{\mathbf{A}}\big] \coloneqq \begin{bmatrix} \mathbf{A}_1 & & & \mathbf{G} & H_{\mathsf{ro}}(r_1) & & & \\ & \mathbf{A}_2 & & \mathbf{G} & & H_{\mathsf{ro}}(r_2) & & \\ & & \ddots & \vdots & & & \ddots & \\ & & \mathbf{A}_\ell & \mathbf{G} & & & & H_{\mathsf{ro}}(r_\ell) \end{bmatrix}.$$

such that

$$\big[\mathbf{A}_i \mid \mathbf{G} \mid H_{\mathsf{ro}}(r_i)\big] \begin{bmatrix} \mathbf{v}_i^1 \\ \hat{\mathbf{c}} \\ \mathbf{v}_i^2 \end{bmatrix} = \mathbf{u} \quad \text{for all } i = 1, \ldots, \ell.$$

It is possible to publicly sample this preimage $\mathbf{v}$ just given pk by first generating a trapdoor $\mathbf{T_A}$ for $\mathbf{A}$ as in WWW24 [62], and then calling

$$\mathsf{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T_A}, (\mathbf{u}, \ldots, \mathbf{u}), \sigma)$$

To enable decryption of all ℓ ciphertexts in the batch, the algorithm returns the pre-decryption key sbk defined as $\mathsf{sbk} \leftarrow \mathsf{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{c} := \mathbf{G}\hat{\mathbf{c}}, \sigma)$ so that $\mathbf{C} \cdot \mathsf{sbk} = \mathbf{G}\hat{\mathbf{c}}$.

In a direct analogy to the earlier batch IBE schemes [4,30,59], the vector $\mathbf{c}$ output by the WWW24 sampler can be viewed as a *succinct* commitment to a set of identity tags, and the values $\mathbf{v}_i := (\mathbf{v}_i^1, \mathbf{v}_i^2)$ as the opening proofs.

The reader may notice that $(\mathsf{sbk}, \mathbf{v}_i^1, \mathbf{v}_i^2)$ is an IBE secret key for the identity tag r_i. Indeed,

$$\left[\mathbf{C} \mid \mathbf{A}_i \mid H_{\mathsf{ro}}(r_i)\right] \cdot \begin{bmatrix} \mathsf{sbk} \\ \mathbf{v}_i^1 \\ \mathbf{v}_i^2 \end{bmatrix} = \mathbf{u}.$$

Note that for decryption to succeed, every decryptor must obtain the same preimage vector $\mathbf{v} := (\mathbf{v}_1^1, \ldots \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$. Hence, this vector is generated using the same sequence of random bits, obtained via a hash function (modeled as a random oracle), when the decryptors and key generator invoke $\mathsf{SampleLeft}$.

Although our pre-decryption keys resemble GPV signatures [43] (with the vector $\mathbf{c}$ replacing the target), it would be misleading to view our scheme as an extension of the GPV IBE. In the GPV IBE, the random oracle maps an identity tag r to a vector $H(r)$, and the secret key is a short preimage of $H(r)$ under a single public matrix. In contrast, in our scheme, the identity tags determine the matrices $H_{\mathsf{ro}}(r)$, over which the preimage is found for the target vector $\mathbf{c}$.

2.2.4 Removing the Index Dependency. The scheme described so far encrypts a message with respect to both an identity tag r and an index $i \in [\ell]$, and for decryption to succeed, all of the indices in a batch must be distinct. To remove this index dependency, we let the encryptors generate their indices u_i (which determine the matrices $\mathbf{A}_i$ in equation (2)) via an injective map $H(\cdot) : \mathcal{I} \to \{0,1\}^d$ applied to their identity tags r_i. Concretely, the encryptor uses $\mathbf{A}_{r_i}$ in place of $\mathbf{A}_i$ that was generated with $u_i = i$, where

$$\mathbf{A}_{r_i} := \left[\mathbf{A}_0 \mid \mathbf{B}_0 - u_i \otimes \mathbf{G}\right] \qquad \text{and} \qquad u_i := H(r_i) \in \{0,1\}^d$$

The rest of the encryption algorithm is unchanged.

The algorithm provided in WWW24 [62] for generating the trapdoor $\mathbf{T}_{\mathbf{A}}$ for the matrix $\mathbf{A}$ (equation (1)) has an exponential upper bound in the label length d. This implies an exponential runtime for our pre-decryption and decryption algorithms, as our label space $\{0,1\}^d$ is as large as the identity space. In Appendix B, we adapt the WWW24 construction in [62] to work with d-bit labels while keeping the runtime of trapdoor generation polynomial in $(n, m, \ell, d, \log q)$. Our key observation is that trapdoor generation uses homomorphic computation over matrix encodings, and in the case of [62], the function being evaluated is always an indicator function, which enables a polynomial running time despite functions having an exponential range $\{0,1\}^d$.

2.2.5 Security. In the selective IBE security game, the adversary specifies a challenge identity r^* before observing pk. The challenger is then responsible for answering pre-decryption key queries for batches of identity tags excluding r^*. Importantly, it must perform these simulations without knowledge of the pre-decryption keys for any batch containing r^*, and without knowledge of the trapdoors for the matrices $[\mathbf{A}_{r^*} \mid H_{\mathsf{ro}}(r^*)]$ and $\mathbf{C}$. This is crucial for the reduction to the LWE problem.

The challenger can respond to pre-decryption key queries without a trapdoor for $\mathbf{C}$ by sampling a random, low-norm key sbk. However, it still cannot sample the preimages $\mathbf{v}_i := \mathrm{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$ for the matrices $[\mathbf{A}_{r_i} \mid H_{\mathsf{ro}}(r_i)]$, $r_i \neq r^*$. To enable the simulation of these preimages, we use the somewhere programmability of the trapdoor sampler, along with the programmability of the random oracle H_{ro}. More specifically, the challenger selects $\mathbf{A}_{r^*}$ at the beginning of the protocol and generates a matching crs for the trapdoor sampler. It then programs $H_{\mathsf{ro}}(r^*)$ as $\mathbf{A}_{r^*}\tilde{\mathbf{R}}^*$, where $\tilde{\mathbf{R}}^* \leftarrow \{1, -1\}^{t \times \overline{m}}$, so that $[\mathbf{A}_{r^*} \mid H_{\mathsf{ro}}(r^*)] = [\mathbf{A}_{r^*} \mid \mathbf{A}_{r^*}\tilde{\mathbf{R}}^*]$. By the leftover hash lemma, the matrix $\mathbf{A}_{r^*}\tilde{\mathbf{R}}^*$ is statistically close to a uniformly-random matrix, consistent with H_{ro} being a random oracle. For each non-challenge identity $r_i \neq r^*$, the challenger programs $H_{\mathsf{ro}}(r_i)$ via $\mathsf{TrapGen}$, obtaining a trapdoor $\mathbf{T}_{r_i}$ for $H_{\mathsf{ro}}(r_i)$. It can then use $\mathsf{SampleLeft}$ with trapdoor $\mathbf{T}_{r_i}$ (treating $H_{\mathsf{ro}}(r_i)$ as the left matrix) to find $\mathbf{v}_i$ such that $[\mathbf{A}_{r_i} \mid H_{\mathsf{ro}}(r_i)] \cdot \mathbf{v}_i = \mathbf{u} - \mathbf{c}$. Finally, it programs the random oracle that fixes the random coins of $\mathsf{SampleLeft}$, so that calling $\mathsf{SampleLeft}$ with the programmed output results in the same sequence of vectors $(\mathbf{v}_1^1, \ldots \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$ as the ones simulated by the challenger.

To ensure that the challenger identifies the random bits that cause $\mathsf{SampleLeft}$ to output $(\mathbf{v}_1^1, \ldots \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$, our scheme uses an *explainable* version of $\mathsf{SampleLeft}$. Recall that a randomized algorithm such as $\mathsf{SampleLeft}$ takes its usual input inp along with a string of random bits rb, and deterministically computes its output using this rb. We construct an algorithm $\mathsf{ExplainSL}(\mathsf{inp}, \mathbf{v}) \rightarrow \mathsf{rb}'$ that takes as input inp along with a vector $\mathbf{v}$ from the support of $\mathsf{SampleLeft}$. The algorithm outputs a string rb' such that $\mathsf{SampleLeft}(\mathsf{inp}; \mathsf{rb}')$ outputs $\mathbf{v}$. We refer to rb' as the explanation for $\mathbf{v}$. This is discussed further in Section 4.

We present our batch IBE construction, its correctness and security proofs in Section 5.

2.3 Thresholdizing the Pre-decryption Key

When the number of parties N is small, the batch IBE scheme can be thresholdized non-interactively by applying the transformation in [24, §22.3.1]. This requires each encryptor to run a τ-of-N secret sharing on its message m and to encrypt each message share using the public key of a different party, thus increasing the ciphertext, pre-decryption key, and public parameter sizes by a factor of N.

To avoid the larger parameter sizes, we can use the protocol in [12, §3] to obtain a non-interactive threshold batch IBE construction. The protocol secret shares the trapdoor for the matrix $\mathbf{C}$ among a set of N parties and operates in an offline/online mode. The offline mode runs once every w calls to the thresholdized Gaussian preimage sampling operation, for some parameter $w \geq 1$. Note that in our scheme, w corresponds to the number of pre-decryption queries that can be supported by one invocation of the offline mode. In the offline mode, a trusted party generates w 'instances' of the components of the Gaussian pre-image vectors (in $\mathbb{Z}_q^m$), and sends a share of these instances to the N parties. In total, each party stores $O(qw)$ such components. Then, in the online phase, every party sends only a share of the Gaussian pre-image vector (pre-decryption key) in $\mathbb{Z}_q^m$ (for up to w calls to the pre-decryption algorithm). Although it is possible to thresholdize the offline mode, this requires MPC operations with multiple rounds of interaction. A recent work [7] proposes a new method for distributing lattice trapdoors non-interactively, which can also be used in our batch IBE scheme.

As the techniques above have their limitations, we next present an efficient two-round threshold Gaussian preimage sampling protocol, which can be used to thresholdize our batch IBE system

as well as GPV signatures [43]. The protocol relies on the masking technique from threshold Raccoon [36] and the noise-flooding techniques as used in prior lattice-based threshold signatures [6]. We present our threshold Gaussian preimage sampler, the full TBIBE system, and state the correctness and security theorems for the TBIBE system using the threshold sampler in Section 6. We present the threshold GPV signatures using the threshold sampler, and prove its security in Section 7.

Threshold Gaussian Preimage Sampling Protocol. The protocol starts by distributing the trapdoor $\mathbf{T_C}$ via τ-of-N Shamir secret sharing. Each party i also receives the pairwise PRF seeds $\mathsf{seed}_{i,j}$ and $\mathsf{seed}_{j,i}$ for $j \neq i$, which are used to mask individual pre-decryption key shares via the row and column masks $\mathbf{m}_i$ and $\mathbf{m}_i^*$:

$$\mathbf{m}_i = \sum_{j \in \mathsf{act}, j \neq i} \mathsf{PRF}(\mathsf{seed}_{i,j}) \qquad \mathbf{m}_i^* = \sum_{j \in \mathsf{act}, j \neq i} \mathsf{PRF}(\mathsf{seed}_{j,i})$$

Note that a given $\mathsf{seed}_{i,j}$ is received by both party i and party j.

The threshold Gaussian sampler proceeds in two rounds. In round 1 of an iteration $\mathsf{sid} \in \mathbb{N}$, each party $i \in \mathsf{act}$ samples $\mathbf{p}_i \leftarrow \mathcal{D}_{\mathbb{Z}^m, \sigma_{\mathsf{flood}}}$, and publishes the exposed syndrome $\mathbf{e}_i = \mathbf{C} \cdot \mathbf{p}_i$, along with a row mask $\mathbf{m}_i$.

Round 2 requires all parties to agree on an active set act of parties for reconstruction. When used for encrypted mempools, he existing consensus protocol can be used to ensure agreement on act. In round 2, all parties compute $\mathbf{y}' = \mathbf{G}^{-1}(\mathbf{c} - \sum_{i \in \mathsf{act}} \mathbf{e}_i)$ and publish

$$\mathbf{sbk}_i = \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} \mathbf{y}' + \mathbf{p}_i + \mathbf{m}_i^\star,$$

where $\mathbf{m}_i^\star$ is the corresponding column mask. The combiner algorithm outputs

$$\mathbf{sbk} = \sum_{i \in \mathsf{act}} (\mathbf{sbk}_i - \mathbf{m}_i) = \mathbf{T_C} \mathbf{y}' + \sum_{i \in \mathsf{act}} \mathbf{p}_i,$$

which follows from $\sum_{i \in \mathsf{act}} \mathbf{m}_i = \sum_{i \in \mathsf{act}} \mathbf{m}_i^*$, and $\sum_{i \in \mathsf{act}} \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} = \mathbf{T_C}$ for Lagrange coefficients $\lambda_{i,\mathsf{act}}$. Therefore, $\mathbf{C} \cdot \mathbf{sbk} = \mathbf{c}$.

In threshold batch IBE, the first-round syndromes are bound by a transcript tag $\mathsf{sid}_{\mathsf{sp}} := H_{\mathsf{tr}}(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i \in \mathsf{act}})$. The parties then use $\mathsf{sid}_{\mathsf{sp}}$ as an additional input to the random oracle that fixes the coins of $\mathsf{SampleLeft}$, and thus the target $\mathbf{c}$. This is important for the security proof, as the simulator must know the first-round syndromes before programming the target $\mathbf{c}$.

The security proof for the threshold batch IBE protocol and GPV signatures rests on two lemmas. First, by a mask-switching lemma, after replacing $\mathsf{PRF}(\mathsf{seed}_{i,j})$ for honest i, j with uniform masks, the simulator can sample the pre-decryption key shares sbk_i uniformly at random for all but one honest party $h \in \mathsf{act}$, while sbk_h is fixed by the other honest shares:

$$\mathsf{sbk}_h := \mathbf{T_C} \mathbf{y}' + \sum_{i \in \mathcal{H}} \mathbf{p}_i - \sum_{i \in \mathcal{C}} \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} \mathbf{y}' - \sum_{i \in \mathcal{H} \setminus \{h\}} \mathsf{sbk}_i + \Delta,$$

where Δ depends on honest parties' seeds (shared with the adversary), and $\mathcal{H}$ and $\mathcal{C}$ denote the honest and corrupt parties. Here, the fact that sbk_h does not depend on the adversary's shares sbk_i, $i \in \mathcal{C}$, is what enables a two-round protocol with no need for the parties to *commit* to these shares in advance. Second, using a Rényi-divergence smudging lemma, the simulator replaces the term $\mathbf{T_C} \mathbf{y}' + \mathbf{p}_h$ by a centered Gaussian over the same coset. This removes the explicit dependence of the honest pre-decryption key shares on the trapdoor $\mathbf{T_C}$ as well as its shares. Finally, after smudging, the remaining hybrids follow largely from the non-threshold proof.

3 Preliminaries

Notation. We use λ as the security parameter. We call a function $f(\cdot)$ negligible (in λ), denoted by $\mathsf{negl}(\lambda)$, if $f(\lambda) = o(1/\mathsf{poly}(\lambda))$ for every polynomial $\mathsf{poly}(.)$. We adopt the short-hands $\mathcal{A}$ and $\mathcal{B}$ for the adversary and Ch for the challenger. We denote computationally indistinguishable games by $\approx_c$ and statistically indistinguishable games by $\approx_s$. We denote by $[n]$ the set $\{1, \ldots, n\}$ and by $(x_{a_i})_{a_i \in \{a_1,\ldots,a_k\}}$ the sequence $(x_{a_1}, \ldots, x_{a_k})$ such that $a_1 < \ldots < a_k$. We use $x \coloneqq y$ to denote assignment. The notation $a \leftarrow S$ defines a fresh uniform random variable on the set S, unless the distribution over S is specified. For a randomized algorithm $\mathcal{A}$ we denote by $y \leftarrow \mathcal{A}(x)$ the random variable obtained by running $\mathcal{A}$ on input x. We write $y \coloneqq \mathcal{A}(x; r)$ when y is the result of invoking algorithm $\mathcal{A}$ on input x with random tape r.

3.1 Batch IBE

Definition 1. *A* **batch identity-based encryption** *(*BIBE*) scheme instantiated with a maximum batch size of $\ell \in \mathbb{N}$, a message space $\mathcal{M}$ and an identity space $\mathcal{I}$ consists of four PPT algorithms:*

- $\text{BIBE.}\textsc{Setup}(1^\lambda, 1^\ell) \to (\mathsf{pk}, \mathsf{sk})$: *The setup algorithm takes the security parameter λ and batch size ℓ, and outputs a public key pk and secret key sk.*
- $\text{BIBE.}\textsc{Enc}(\mathsf{pk}, r, \mathsf{m}) \to \mathsf{ct}$: *The encryption algorithm takes as input a public key pk, an identity tag r, and a message $\mathsf{m} \in \mathcal{M}$, and outputs ciphertext ct.*
- $\text{BIBE.}\textsc{PreDec}(\mathsf{sk}, (r_i)_{i\in[\ell]}) \to \mathsf{sbk}$: *The pre-decryption algorithm takes as input a secret key sk and ℓ identity tags, and outputs pre-decryption key sbk.*
- $\text{BIBE.}\textsc{Dec}(\mathsf{pk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) \to (\mathsf{m}_i)_{i\in[\ell]}$: *The decryption algorithm takes the key sbk and ℓ ciphertexts, and outputs the corresponding messages.*

A BIBE scheme must be correct, secure, and efficient, with a succinct pre-decryption key sbk in the batch size ℓ.

Definition 2 (Correctness of BIBE). *For all $\lambda \in \mathbb{N}$, $\ell \in \mathbb{N}$, $(\mathsf{m}_i)_{i\in[\ell]} \in \mathcal{M}^\ell$ and $(r_i)_{i\in[\ell]} \in \mathcal{I}^\ell$ such that $r_i \neq r_j$ for $i \neq j$, it holds that*

$$\Pr\left[\text{BIBE.}\textsc{Dec}(\mathsf{pk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) = (\mathsf{m}_i)_{i\in[\ell]} \middle| \begin{array}{c} (\mathsf{pk},\mathsf{sk}) \leftarrow \textsc{Setup}(1^\lambda, 1^\ell) \\ \mathsf{ct}_i \leftarrow \text{BIBE.}\textsc{Enc}(\mathsf{pk}, r_i, \mathsf{m}_i) \ \forall i \in [\ell] \\ \mathsf{sbk} \leftarrow \text{BIBE.}\textsc{PreDec}(\mathsf{sk}, (r_i)_{i\in[\ell]}) \end{array}\right] > 1 - \mathsf{negl}(\lambda)$$

Definition 3 (Security of BIBE). *The security game $\mathsf{Game}^{\text{BIBE}}_{\mathcal{A},b}(1^\lambda, 1^\ell)$ is defined in Figure 1 with respect to the adversary $\mathcal{A}$. A* BIBE *scheme is selectively secure in the random oracle model if there exists a negligible function $\mathsf{negl}(.)$ such that for all $\ell = \mathsf{poly}(\lambda)$ and PPT adversaries $\mathcal{A}$,*

$$\text{Adv}^{\text{BIBE}}_{\mathcal{A}}(1^\lambda, 1^\ell) = \left|\Pr[\mathsf{Game}^{\text{BIBE}}_{\mathcal{A},0}(1^\lambda, \ell) = 1] - \Pr[\mathsf{Game}^{\text{BIBE}}_{\mathcal{A},1}(1^\lambda, \ell) = 1]\right| < \mathsf{negl}(\lambda)$$

The Security Game $\mathsf{Game}^{\mathbf{BIBE}}_{\mathcal{A},b}(1^\lambda, \ell)$

Challenge identity: The adversary $\mathcal{A}$ commits to the challenge identity $r^* \in \mathcal{I}$ and sends it to the challenger Ch.

Setup: Ch takes as input the security parameter λ and the maximum batch size ℓ. It runs $(\mathsf{pk}, \mathsf{sk}) \leftarrow \textsc{Setup}(1^\lambda, 1^\ell)$ and sends pk to $\mathcal{A}$. The rest of the game proceeds in rounds as follows:

Pre-decryption queries: $\mathcal{A}$ may issue an arbitrary number of pre-challenge queries for computing the pre-decryption key. Upon receiving a pre-decryption query for an ordered identity list $(r_i)_{i\in[\ell]}$, Ch checks if $r^* \in (r_i)_{i\in[\ell]}$. If not, it computes the pre-decryption key $\mathsf{sbk} \leftarrow \text{BIBE.}\textsc{PreDec}(\mathsf{sk}, (r_i)_{i\in[\ell]})$ and sends sbk to $\mathcal{A}$. Otherwise, it aborts the game.

Challenge round: At any point in time, $\mathcal{A}$ may decide that the current round is the challenge round. In this case,

- ▷ $\mathcal{A}$ sends two messages $\mathsf{m}_0, \mathsf{m}_1 \in \mathcal{M}$ on which it would like to be challenged.
- ▷ Ch computes $\mathsf{ct}_b \leftarrow \text{BIBE.}\textsc{Enc}(\mathsf{pk}, r^*, \mathsf{m}_b)$ and sends ct_b to $\mathcal{A}$.

Post-challenge queries: The adversary can again send an arbitrary number of key computation queries as in the pre-challenge case. If it asks for the pre-decryption key corresponding to a list of identities $(r_i)_{i\in[\ell]}$ such that $r^* \in (r_i)_{i\in[\ell]}$, then Ch aborts the game.

Random oracle queries: Upon receiving a query containing a new input x, Ch samples a value y from the output distribution of the random oracle, stores the tuple (x, y), and returns y. It returns the same value y upon receiving the same input x in a future query.

Output: At any point in time, $\mathcal{A}$ can decide to output a bit $b' \in \{0, 1\}$. The game then halts with the same output b'.

Figure 1: The BIBE security game.

A fully secure BIBE scheme is defined in a similar manner, with the distinction that the adversary is allowed to select the challenge identity r^* after observing the public key pk and issuing a polynomial number of pre-challenge queries.

3.2 Threshold Batch IBE

Definition 4. *A* **threshold batch IBE** *(*TBIBE*) system instantiated with a committee of $N \in \mathbb{N}$ parties, a threshold $\tau \in [N]$, a maximum batch size $\ell \in \mathbb{N}$, a message space $\mathcal{M}$, and an identity space $\mathcal{I}$ consists of five PPT algorithms:*

- TBIBE.$\textsc{Setup}(1^\lambda, 1^\ell, 1^N, 1^\tau) \to (\mathsf{pk}, \mathsf{sk}_1, \ldots \mathsf{sk}_N)$: *The setup algorithm takes the security parameter λ, maximum batch size ℓ, number of parties N, and threshold τ, and outputs a public key pk and N secret keys $(\mathsf{sk}_1, \ldots, \mathsf{sk}_N)$.*
- TBIBE.$\textsc{Enc}(\mathsf{pk}, r, \mathsf{m}) \to \mathsf{ct}$: *The encryption algorithm takes a public key pk, an identity $r \in \mathcal{I}$, and a message $\mathsf{m} \in \mathcal{M}$, and outputs a ciphertext ct.*
- TBIBE.$\textsc{PreDec}(\mathsf{sk}_j, (r_i)_{i\in[\ell]}) \to \mathsf{sbk}_j$: *The pre-decryption algorithm takes as input a secret key sk_j for a party $j \in [N]$ and ℓ identities, and outputs a pre-decryption key sbk_j for that party.*
- TBIBE.$\textsc{Comb}(\mathsf{pk}, \mathsf{act} \subseteq [N], (r_i)_{i\in[\ell]}, (\mathsf{sbk}_j)_{j\in\mathsf{act}}) \to \mathsf{sbk}$: *The combiner algorithm takes as input a subset $\mathsf{act} \subseteq [N]$ of size τ, identities $(r_i)_{i\in[\ell]}$, and the pre-decryption keys $(\mathsf{sbk}_j)_{i\in\mathsf{act}}$, and outputs a single* short *pre-decryption key sbk.*
- TBIBE.$\textsc{Dec}(\mathsf{pk}, \mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) \to (\mathsf{m}_i)_{i\in[\ell]}$: *The decryption algorithm takes as input a combined pre-decryption key sbk and ℓ ciphertexts, and outputs the corresponding messages.*

When pre-decryption is a two-round operation, we express the two algorithms by TBIBE.PredecOne$(\mathsf{sk}_j, \mathsf{act}, \mathsf{sid}, ($ *and* TBIBE.PredecTwo*, where* act *is the active, agreed-upon set of signers,* sid *is an identifier unique to each signing query, and the input to* TBIBE.PredecTwo *contains the output of* TBIBE.PredecOne *from the active set* act*. In the two-round case,* TBIBE.Comb *takes as input the round-1 messages and the round-2 outputs of the active parties.*

The Security Game $\mathsf{Game}^{\mathbf{TBIBE}}_{\mathcal{A},b}(1^\lambda, 1^\ell, 1^N, 1^\tau)$

Challenge identity: The adversary $\mathcal{A}$ commits to the challenge identity r^* and sends it to the challenger Ch.

Setup: $\mathcal{A}$ sends polynomially bounded parameters $N, \tau \leq N$ and a subset $S \subseteq [N]$ of size at most $\tau - 1$ to Ch. Ch also takes as input the security parameter λ, batch size ℓ, and runs $(\mathsf{pk}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N) \leftarrow$ TBIBE.Setup$(1^\lambda, 1^\ell, 1^N, 1^\tau)$. It sends pk and $(\mathsf{sk}_i)_{i\in S}$ to $\mathcal{A}$.

Pre-challenge queries: $\mathcal{A}$ may issue an arbitrary number of pre-challenge queries for computing the pre-decryption key:

- $\mathcal{A}$ sends a list $(r_i)_{i\in[\ell]}$ of up to ℓ identities to Ch.
- **One-round pre-decryption:** If $r^* \notin (r_i)_{i\in[\ell]}$, Ch computes the key shares of the honest parties $j \in [N]\setminus S$, $\mathsf{sbk}_j \leftarrow$ TBIBE.PreDec$(\mathsf{sk}_j, (r_i)_{i\in[\ell]})$, and sends $(\mathsf{sbk}_j)_{j\in[N]\setminus S}$ to $\mathcal{A}$. Otherwise, Ch aborts.
- **Two-round pre-decryption:** Upon receiving a query with the active set act and unique identifier sid, Ch first replies with
$$\mathsf{sbk}^1_j \leftarrow \text{TBIBE.PredecOne}(\mathsf{sk}_j, \mathsf{act}, \mathsf{sid}, (r_i)_{i\in[\ell]})$$
for $j \in [N]\setminus S$. It then collects $(\mathsf{sbk}^1_k)_{k\in S}$, runs
$$\mathsf{sbk}_j \leftarrow \text{TBIBE.PredecTwo}(\mathsf{sk}_j, \mathsf{act}, \mathsf{sid}, (r_i)_{i\in[\ell]}, (\mathsf{sbk}^1_k)_{k\in\mathsf{act}}),$$
for $j \in [N]\setminus S$, and sends $(\mathsf{sbk}_j)_{j\in[N]\setminus S}$ to $\mathcal{A}$.

Challenge round: At any point in time, the adversary may decide that the current round is the challenge round. In this case, $\mathcal{A}$ sends two messages $\mathsf{m}_0, \mathsf{m}_1 \in \mathcal{M}$ on which it would like to be challenged. Ch computes $\mathsf{ct}_b \leftarrow$ TBIBE.Enc$(\mathsf{pk}, r^*, \mathsf{m}_b)$ and sends ct_b to $\mathcal{A}$.

Post-challenge queries: $\mathcal{A}$ may again issue an arbitrary number of key computation queries as in the pre-challenge case. If it asks for the pre-decryption key shares corresponding to a list of identities $(r_i)_{i\in[\ell]}$ such that $r^* \in (r_i)_{i\in[\ell]}$, then Ch aborts the game.

Random oracle queries: Upon receiving a query containing a new input x, Ch samples a value y from the output distribution of the random oracle, stores the tuple (x, y), and returns y. It returns the value y upon receiving the same input x in a future query.

Output: At any point in time, $\mathcal{A}$ can decide to output a bit $b' \in \{0, 1\}$. The game then halts with the same output b'.

Figure 2: The TBIBE security game.

Definition 5 (Correctness of TBIBE). *For all* $\lambda, \ell \in \mathbb{N}$, $\mathsf{act} \subseteq [N]$, $(\mathsf{m}_i)_{i\in[\ell]} \in \mathcal{M}^\ell$ *and* $(r_i)_{i\in[\ell]} \in \mathcal{I}^\ell$*, the following probability is greater than* $1 - \mathsf{negl}(\lambda)$*:*

$$\Pr\left[\text{TBIBE.Dec}(\mathsf{pk}, \mathsf{sbk}, (ct_i)_{i\in[\ell]}) = (\mathsf{m}_i)_{i\in[\ell]} \;\middle|\; \begin{array}{c}(\mathsf{pk},(\mathsf{sk})_{i\in[N]}) \leftarrow \text{TBIBE.Setup}(1^\lambda,1^\ell,1^N,1^\tau),\\ \mathsf{ct}_i \leftarrow \text{TBIBE.Enc}(\mathsf{pk}, r_i, \mathsf{m}_i),\\ \mathsf{sbk}_j \leftarrow \text{TBIBE.PreDec}(\mathsf{sk}_j, (r_i)_{i\in[\ell]})\ \forall i\in[\ell]\\ \mathsf{sbk} \leftarrow \text{TBIBE.Comb}(\mathsf{pk}, \mathsf{act}, (r_i)_{i\in[\ell]}, (\mathsf{sbk}_j)_{j\in\mathsf{act}})\end{array}\right]$$

Definition 6 (Security of TBIBE). *The game* $\mathsf{Game}^{\text{TBIBE}}_{\mathcal{A},b}(1^\lambda, 1^\ell, 1^N, 1^\tau)$ *is defined in Figure 2 with respect to the adversary* $\mathcal{A}$*. A* TBIBE *scheme is selectively secure in the random oracle model*

if for all $\ell, N = \mathsf{poly}(\lambda)$, $\tau < N$, *and PPT adversaries* $\mathcal{A}$, *there exists some negligible function* $\mathsf{negl}(.)$ *such that*

$$\mathrm{Adv}^{\mathrm{TBIBE}}_{\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^\tau) = \Big|\Pr[\mathsf{Game}^{\mathrm{TBIBE}}_{\mathcal{A},0}(1^\lambda, 1^\ell, 1^N, 1^\tau) = 1] - \Pr[\mathsf{Game}^{\mathrm{TBIBE}}_{\mathcal{A},1}(1^\lambda, 1^\ell, 1^N, 1^\tau) = 1]\Big| < \mathsf{negl}(\lambda)$$

3.3 Threshold Signatures

Definition 7. *A* **threshold signature** (TSIG) *system instantiated with a committee of* $N \in \mathbb{N}$ *parties, a threshold* $\tau \in [N]$, *and a message space* $\mathcal{M}$ *consists of three PPT algorithms:*

- $\mathrm{TSIG.Setup}(1^\lambda, 1^N, 1^\tau) \to (\mathsf{pk}, \mathsf{pkc}, \mathsf{sk}_1, \ldots \mathsf{sk}_N)$: *The setup algorithm takes the security parameter* λ, *number of parties* N, *and threshold* τ, *and outputs a public key* pk, *a combiner key* pkc, *and* N *secret keys* $(\mathsf{sk}_1, \ldots, \mathsf{sk}_N)$.
- $\mathrm{TSIG.Sign}(\mathsf{sk}_i, \mathsf{m}) \to \sigma_i$ *The signing algorithm takes a secret key* sk_i, *and a message* $\mathsf{m} \in \mathcal{M}$, *and outputs a signature share* σ_i.
- $\mathrm{TSIG.Comb}(\mathsf{pkc}, \mathsf{act}, \mathsf{m}, (\sigma_i)_{i\in\mathsf{act}}) \to \sigma$ *The combiner algorithm takes a public combiner key* pkc, *a subset* $\mathsf{act} \subseteq [N]$ *of size* τ, *a message* $\mathsf{m} \in \mathcal{M}$, *and the signatures* $(\sigma_i)_{i\in\mathsf{act}}$, *and outputs a combined signature* σ.
- $\mathrm{TSIG.Ver}(\mathsf{pk}, \sigma, \mathsf{m}) \to 0/1$ *The verification algorithm takes a public key* pk, *a signature* σ, *and a message* m, *and outputs accept or reject.*

When signing is a two-round operation, the algorithms are denoted by $\mathrm{TSIG.SignOne}(\mathsf{sk}_i, \mathsf{act}, \mathsf{sid}, \mathsf{m})$ *and* $\mathrm{TSIG.SignTwo}$, *where* act *is the active, agreed-upon set of signers,* sid *is an identifier unique to each signing query, and the input to* $\mathrm{TSIG.SignTwo}$ *contains the output of* $\mathrm{TSIG.SignOne}$ *from the active set* act.

Definition 8 (Correctness of TSIG). *For all* $\lambda, \ell \in \mathbb{N}$, $T \subseteq [N]$, *and* $\mathsf{m} \in \mathcal{M}$, *the following probability is greater than* $1 - \mathsf{negl}(\lambda)$:

$$\Pr\left[\mathrm{TSIG.Ver}(\mathsf{pk}, \sigma, \mathsf{m}) = 1 \;\middle|\; \begin{array}{c}(\mathsf{pk},\mathsf{pkc},(\mathsf{sk}_i)_{i\in[N]}) \leftarrow \mathrm{TSIG.Setup}(1^\lambda,1^N,1^\tau),\\ \sigma_i \leftarrow \mathrm{TSIG.Sign}(\mathsf{sk}_i,\mathsf{m}),\\ \sigma \leftarrow \mathrm{TSIG.Comb}(\mathsf{pk},\mathsf{pkc},(\sigma_i)_{i\in T})\end{array}\right]$$

Definition 9 (TS-UF-1 of TSIG). *The game* $\mathsf{Game}^{\mathrm{TSIG}}_{\mathcal{A}}(1^\lambda, 1^N, 1^\tau)$ *for threshold signature unforgeability is defined in Figure 3 with respect to the adversary* $\mathcal{A}$. *A* TSIG *scheme is TS-UF-1 secure if for all* $N = \mathsf{poly}(\lambda)$, $\tau < N$, *and PPT adversaries* $\mathcal{A}$, *there exists some negligible function* $\mathsf{negl}(.)$ *such that*

$$\mathrm{Adv}^{\mathrm{TSIG}}_{\mathcal{A}}(1^\lambda, 1^N, 1^\tau) \coloneqq \Pr[\mathsf{Game}^{\mathrm{TSIG}}_{\mathcal{A}}(1^\lambda, 1^N, 1^\tau) = 1] < \mathsf{negl}(\lambda)$$

The Security Game $\mathsf{Game}_{\mathcal{A}}^{\mathbf{TSIG}}(1^\lambda, 1^N, 1^\tau)$

Setup: $\mathcal{A}$ sends polynomially bounded parameters N, $\tau \leq N$ and a subset $S \subseteq [N]$ of size at most $\tau - 1$ to Ch. Ch also takes as input the security parameter λ and runs $(\mathsf{pk}, \mathsf{pkc}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N) \leftarrow \text{TSIG.}\textsc{Setup}(1^\lambda, 1^N, 1^\tau)$. It sends pk and $(\mathsf{sk}_i)_{i \in S}$ to $\mathcal{A}$, and initializes the table $T(\mathsf{m}) := \emptyset$ for $(\mathsf{m}, \mathsf{sid}) \in \mathcal{M}$.

Signature queries: $\mathcal{A}$ may issue an arbitrary number of signature queries for the parties $j \in [N]$ with a message m and optionally a session id sid. Different messages must have different sids unique to the message.

- ▷ **One-round signing:** Ch replies with $\sigma_j \leftarrow \text{TSIG.}\textsc{Sign}(\mathsf{sk}_j, \mathsf{m})$, and sets $T(\mathsf{m}) := T(\mathsf{m}) \cup \{j\}$.
- ▷ **Two-round signing:** Upon receiving a query with the active set act and the unique identifier sid, Ch first replies with

$$\sigma_j^1 \leftarrow \text{TSIG.}\textsc{SignOne}(\mathsf{sk}_j, \mathsf{act}, \mathsf{sid}, \mathsf{m})$$

for $j \in [N] \setminus S$. It then collects $(\sigma_k^1)_{k \in S}$, computes

$$\sigma_j \leftarrow \text{TSIG.}\textsc{SignTwo}(\mathsf{sk}_j, \mathsf{act}, \mathsf{sid}, \mathsf{m}, (\sigma_k^1)_{k \in \mathsf{act}}),$$

returns σ_j, and sets $T(\mathsf{m}) := T(\mathsf{m}) \cup \{j\}$.

Forgery: $\mathcal{A}$ returns a message-signature tuple (m, σ). The game outputs 1 (*i.e.*, $\mathcal{A}$ wins the game) if $\text{TSIG.}\textsc{Ver}(\mathsf{pk}, \mathsf{m}, \sigma) = 1$ and $|T(\mathsf{m})| < \tau - |S|$.

Figure 3: The TSIG security game.

3.4 Background on Lattices

This section focuses on a few algorithms used by our construction. More detailed preliminaries on lattices are presented in Section A.2.

We assume all vectors are column vectors. Given vectors $\mathbf{v}_1, \ldots \mathbf{v}_\ell \in \mathbb{Z}^m$ and the matrices $\mathbf{A}_1, \ldots, \mathbf{A}_\ell \in \mathbb{Z}^{n \times m}$, we define the following short-hand notations:

$$\text{col}(\mathbf{v}_1, \ldots \mathbf{v}_\ell) := \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \ldots \\ \mathbf{v}_\ell \end{bmatrix} \in \mathbb{Z}^{\ell m} \qquad \text{diag}(\mathbf{A}_1, \ldots, \mathbf{A}_\ell) := \begin{bmatrix} \mathbf{A}_1 & & & \\ & \mathbf{A}_2 & & \\ & & \ldots & \\ & & & \mathbf{A}_\ell \end{bmatrix} \in \mathbb{Z}^{\ell n \times \ell m}$$

We use the ℓ_2-norm for vectors and matrices unless stated otherwise, *i.e.*, $\|\mathbf{v}\| = \|\mathbf{v}\|_2 = \sqrt{\sum_i |v_i|^2}$ for a vector $\mathbf{v} = \text{col}(v_1, \ldots, v_n) \in \mathbb{Z}^n$, and $\|\mathbf{A}\| = \max_{i \in [m]} \|\mathbf{v}_i\|$ for a matrix $\mathbf{A} = [\mathbf{v}_1, \ldots, \mathbf{v}_m] \in \mathbb{Z}^{n \times m}$, where $\mathbf{v}_i \in \mathbb{Z}^n$, $i \in [m]$. We denote by $\|\mathbf{A}\|_{\mathsf{GS}}$ the ℓ_2-norm of the Gram-Schmidt orthogonalization of the matrix $\mathbf{A}$, and by $\mathbf{I}_n \in \mathbb{Z}_q^{n \times n}$ the n-by-n identity matrix. Note that for any matrix $\mathbf{T} \in \mathbb{Z}_q^{m \times \tilde{m}}$, it holds that $\|\mathbf{T}\|_{\mathsf{GS}} \leq \|\mathbf{T}\| \leq \sqrt{m}\|\mathbf{T}\|_\infty$. We denote the statistical distance between two distributions P and Q by $\Delta(P, Q)$.

3.4.1 The Gadget Matrix and Trapdoors.

For $n, q \in \mathbb{N}$, the gadget matrix $\mathbf{G}_n$ is defined as follows [50]:

$$\mathbf{G}_n := \mathbf{I}_n \otimes \mathbf{g}^\top \in \mathbb{Z}_q^{n \times \tilde{m}},$$

where $\mathbf{g}^\top := [1, 2, \ldots, 2^{\lfloor \log(q) \rfloor}]$ and $\tilde{m} := n(\lfloor \log(q) \rfloor + 1)$.

Lemma 1 (Adapted from [50], Theorem 3.9 [62]). *Let $n, m, \tilde{m}, q$ be lattice parameters with $\tilde{m}$ as defined above. Then, there exist efficient algorithms* $(\mathsf{TrapGen}, \mathsf{SamplePre})$ *such that;*

– $\mathsf{TrapGen}(1^n, q, m) \to (\mathbf{A}, \mathbf{T_A})$: *Given a lattice dimension n, modulus q and number of samples m,* $\mathsf{TrapGen}$ *outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a gadget trapdoor $\mathbf{T_A} \in \mathbb{Z}_q^{m \times \tilde{m}}$.*
– $\mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma) \to \mathbf{v}$: *Given a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a trapdoor $\mathbf{T_A} \in \mathbb{Z}_q^{m \times \tilde{m}}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$ and a Gaussian width parameter σ,* $\mathsf{SamplePre}$ *outputs a vector $\mathbf{v} \in \mathbb{Z}_q^m$.*

For all $m \geq O(n \log(q))$, the algorithms satisfy the following:

– ***Trapdoor distribution:*** *Given $(\mathbf{A}, \mathbf{T_A}) \leftarrow \mathsf{TrapGen}(1^n, q, m)$ and $\mathbf{A}' \leftarrow \mathbb{Z}_q^{n \times m}$, it holds that $\Delta(\mathbf{A}, \mathbf{A}') \leq 2^{-n}$. Moreover, $\mathbf{AT_A} = \mathbf{G}$ and $\|\mathbf{T_A}\|_\infty = 1$.*
– ***Preimage sampling:*** *For all trapdoors $\mathbf{T_A} \in \mathbb{Z}_q^{m \times \tilde{m}}$, parameters $\sigma > 0$, and target vectors $\mathbf{u} \in \mathbb{Z}_q^n$ in the column span of $\mathbf{A}$, $\mathbf{v} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma)$ satisfies $\mathbf{Av} = \mathbf{u}$.*
– ***Preimage distribution:*** *Suppose $\mathbf{T_A}$ is a gadget trapdoor for $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ (*i.e.*, $\mathbf{AT_A} = \mathbf{G}$). Then, for all $\sigma \geq \sqrt{m\tilde{m}} \|\mathbf{T_A}\|_\infty \cdot \omega(\sqrt{\log(n)})$ and target vectors $\mathbf{u} \in \mathbb{Z}_q^n$, the statistical distance between the following distributions is at most 2^{-n}: $\{\mathbf{v} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma)\}$ and $\{\mathbf{v} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{u})\}$.*

Definition 10 (Learning with Errors, from Def. 9 [5]). *Given a prime q, a positive integer n, and a distribution χ over $\mathbb{Z}_q$, a $(\mathbb{Z}_q, n, \chi)$-LWE problem instance is associated with a challenge oracle $\mathcal{O}$ that is either a noisy pseudo-random sampler $\mathcal{O}_s$ for a random secret key $s \in \mathbb{Z}_q^n$, or a truly random sampler $\mathcal{O}_\$$, behaving as follows:*

– *$\mathcal{O}_s$: outputs samples of the form $(\mathbf{u}_i, \mathbf{v}_i) := (\mathbf{u}_i, \mathbf{u}_i^\top \mathbf{s} + \mathbf{x}_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, where $\mathbf{u}_i \leftarrow \mathbb{Z}_q^n$, $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ is a uniformly-random invariant, and $\mathbf{x}_i \in \mathbb{Z}_q$ is a fresh sample from χ.*
– *$\mathcal{O}_\$$: outputs truly uniform random samples from $\mathbb{Z}_q^n \times \mathbb{Z}_q$.*

An algorithm $\mathcal{A}$ decides the $(\mathbb{Z}_q, n, \chi)$-LWE problem if for some $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, the advantage of $\mathcal{A}$, defined as $\mathsf{Adv}_{\mathcal{A}}^{\mathsf{LWE}}(1^\lambda) = |\Pr[\mathcal{A}^{\mathcal{O}_s} = 1] - \Pr[\mathcal{A}^{\mathcal{O}_\$} = 1]|$ is non-negligible in λ.

The $(\mathbb{Z}_q, n, \chi)$-LWE assumption states that no PPT algorithm decides the $(\mathbb{Z}_q, n, \chi)$-LWE problem. For some $\alpha \in (0, 1)$ and a prime q, we set the noise distribution χ to be Ψ_α, the distribution over $\mathbb{Z}_q$ of the random variable $\lfloor qX \rceil \bmod q$, where X is a normal random variable with mean 0 and standard deviation $\alpha/\sqrt{2\pi}$. Then, as in [5], we assume the hardness of approximating the SIVP and GapSVP problems in lattices of dimension n to within approximation factors polynomial in n. Given these assumptions and [5, Theorem 11], which in turn follows from Regev's reduction [54], we conclude that deciding the $(\mathbb{Z}_q, n, \Psi_\alpha)$-LWE problem is hard, when n/α is a polynomial in n and $q > 2\sqrt{n}/\alpha$.

3.5 Shifted Multi-Preimage Trapdoor Sampler

Our BIBE system makes use of the following shifted multi-preimage trapdoor sampler, introduced and instantiated by Waters, Wee, and Wu [62].

Definition 11 (Definitions 4.2 and 4.4 from [62]). *Let ℓ be a dimension parameter, and n, t, q, σ be functions of λ and ℓ. An (n, t, q, σ)-shifted multi-preimage trapdoor sampler consists of the following efficient algorithms:*

– $\mathsf{Gen}(1^\lambda, 1^\ell) \to \mathsf{crs}$*: The randomized generation algorithm outputs a CRS.*
– $\mathsf{Expand}(1^\lambda, 1^\ell, \mathsf{crs}, (u_i)_{i \in [\ell]}) \to (\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td})$*: The deterministic expand algorithm takes up to ℓ distinct bit vectors $(u_i)_{i \in [\ell]}$, and outputs matrices $\mathbf{A}_1, \ldots, \mathbf{A}_\ell \in \mathbb{Z}_q^{n \times t}$ and a matrix-trapdoor tuple* td*.*

– $\mathsf{SampleMultPre}(\mathsf{td}, \mathbf{t}_1, \ldots, \mathbf{t}_\ell) \to (\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c})$*: Given a matrix-trapdoor tuple* td *and target vectors* $\mathbf{t}_1, \ldots, \mathbf{t}_\ell$*, the randomized sampling algorithm outputs a shift* $\mathbf{c} \in \mathbb{Z}_q^n$ *and the preimages* $\boldsymbol{\pi}_i \in \mathbb{Z}_q^t$.

The sampler must satisfy:

– **Correctness:** For all $\lambda, \ell \in \mathbb{N}$, all crs in the support of $\mathsf{Gen}(1^\lambda, 1^\ell)$, ℓ distinct bit vectors $(u_i)_{i\in[\ell]}$, and all $\mathbf{t}_1, \ldots, \mathbf{t}_\ell \in \mathbb{Z}_q^n$, given $(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td}) \coloneqq \mathsf{Expand}(1^\lambda, 1^\ell, \mathsf{crs}, (u_i)_{i\in[\ell]})$, it holds that

$$\Pr\left[\mathbf{A}_i\boldsymbol{\pi}_i = \mathbf{t}_i + \mathbf{c}\ \forall i \in [\ell] \colon (\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c}) \leftarrow \mathsf{SampleMultPre}(\mathsf{td}, \mathbf{t}_1, \ldots, \mathbf{t}_\ell)\right] = 1.$$

– **Preimage distribution:** For all polynomials $\ell = \ell(\lambda)$, except with negligible probability over $\mathsf{crs} \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell)$, and $(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td}) \coloneqq \mathsf{Expand}(1^\lambda, 1^\ell, \mathsf{crs})$, the statistical distance between the following distributions is $\mathsf{negl}(\lambda)$:

$$\{(\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c}) \leftarrow \mathsf{SampleMultPre}(\mathsf{td}, \mathbf{t}_1, \ldots, \mathbf{t}_\ell)\} \text{ and}$$
$$\left\{(\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c}) \middle| \begin{array}{c} \mathbf{c}\leftarrow\mathbb{Z}_q^n, \\ \boldsymbol{\pi}_i\leftarrow(\mathbf{A}_i^{-1})_\sigma(\mathbf{t}_i+\mathbf{c})\ \forall i\in[\ell] \end{array}\right\}$$

– **Somewhere programmable:** There exists an efficient algorithm $\mathsf{GenProg}$ such that for all polynomials $\ell = \ell(\lambda)$, and ℓ distinct bit vectors $(u_i)_{i\in[\ell]}$, the following holds:
- $\mathsf{GenProg}(1^\lambda, 1^\ell, u_i, \mathbf{A}_i)$ takes a bit vector u_i and matrix $\mathbf{A}_i^* \in \mathbb{Z}_q^{n\times t}$, and outputs a common reference string $\tilde{\mathsf{crs}}$.
- $\forall i \in [\ell]$, $\forall \mathbf{A}_i^* \in \mathbb{Z}_q^{n\times t}$:

$$\Pr\left[\mathbf{A}_i^* = \tilde{\mathbf{A}}_i \middle| \begin{array}{c} \tilde{\mathsf{crs}}:=\mathsf{GenProg}(1^\lambda,1^\ell,u_i,\mathbf{A}_i^*), \\ (\tilde{\mathbf{A}}_1,\ldots,\tilde{\mathbf{A}}_\ell,\mathsf{td}):=\mathsf{Expand}(1^\lambda,1^\ell,\tilde{\mathsf{crs}},(u_i)_{i\in[\ell]}) \end{array}\right] = 1.$$

- The statistical distance between the following distributions is $\mathsf{negl}(\lambda)$:

$$\{\mathsf{crs} \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell)\} \text{ and } \left\{\tilde{\mathsf{crs}} \middle| \begin{array}{c} \mathbf{A}_i\leftarrow\mathbb{Z}_q^{n\times t}, \\ \tilde{\mathsf{crs}}:=\mathsf{GenProg}(1^\lambda,1^\ell,u_i,\mathbf{A}_i) \end{array}\right\}$$

– **Local expansion:** There exists an efficient deterministic algorithm $\mathsf{ExpandLocal}$ such that:
- $\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, u_i)$ takes a bit vector u_i and outputs $\mathbf{A}_i \in \mathbb{Z}_q^{n\times t}$.
- For all $\lambda, \ell \in \mathbb{N}$ and all crs, given

$$(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td}) \coloneqq \mathsf{Expand}(1^\lambda, 1^\ell, \mathsf{crs}, (u_i)_{i\in[\ell]}),$$

  it holds that for all $i \in [\ell]$, $\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, u_i) = \mathbf{A}_i$. Moreover, $\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, u_i)$ only reads $\mathsf{poly}(\lambda, \log(\ell))$ bits of crs and runs in time $\mathsf{poly}(\lambda, \log(\ell))$.

The properties of the shifted multi-preimage trapdoor sampler, and the construction by Waters, Wee, and Wu [62, Construction 4.6] are described in Appendix B.

In the BIBE system of Section 5, during each call to the pre-decryption algorithm, the shifted multi-preimage sampler is invoked on at most ℓ distinct bit strings $\tilde{h}_1, \ldots, \tilde{h}_\ell \in \{0,1\}^d$. However, these bit strings may have length $d = \mathsf{poly}(\lambda)$, whereas in [62], $d = \lceil \log(\ell) \rceil = O(\log(\lambda)$. By [62, Theorem 3.10], runtime of Expand is bounded by $2^\ell \cdot \mathsf{poly}(n, m, \ell, \log(q))$. Hence, naively invoking [62, Theorem 3.10] with $d = \mathsf{poly}(\lambda)$ would imply an exponential running time for Expand. However, in Section B.4, we show how to obtain a polynomial running time in λ via a careful modification of the Expand algorithm presented in [62]. Given the modified algorithm, we restate the original security theorem [62, Theorem 4.7] below. in terms of both d and ℓ.

Lemma 2. *Let $n \geq \lambda$, $d = d(\lambda)$, $n = n(\lambda, \ell, d)$, $q = q(\lambda, \ell, d)$, $m = 3n\lceil \log q \rceil$, and $t = \tilde{m}(d+1)$. Then, for every $\sigma \geq (\ell t + \tilde{m}) \log(\ell n)$, the d-bit label variant of [62, Construction 4.6] in Appendix B, instantiated on any distinct labels $u_1, \ldots, u_\ell \in \{0,1\}^d$, is an (n, t, q, σ)-shifted multi-preimage trapdoor sampler with perfect somewhere programmability, transparent setup, local expansion, and simulatable openings. The CRS size is $nt \log(q) = n\tilde{m}(d+1)\log(q)$. Moreover, Expand runs in time $\mathsf{poly}(n, m, \ell, d, \log(q))$.*

4 Explainable Discrete Gaussian Sampler SampleLeft

Our BIBE system in Section 5 relies on a discrete Gaussian sampling algorithm called $\mathsf{SampleLeft}$. The security proof requires this algorithm to be *explainable* [49]. We first define what explainable $\mathsf{SampleLeft}$ means (Definition 12), and then construct such an algorithm (Theorem 1). As defined in [5, §4.1], $\mathsf{SampleLeft}$ is a randomized algorithm that takes as input a tuple $\mathsf{inp} := (\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma)$, and outputs a vector $\mathbf{v}$ such that $\mathbf{F}\mathbf{v} = \mathbf{u}$, where $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$. Informally, we say that the algorithm is explainable if there is an efficient helper algorithm, denoted $\mathsf{ExplainSL}$, that takes as input $(\mathsf{inp}, \mathbf{v})$, and outputs some random bits rb such that $\mathsf{SampleLeft}(\mathsf{inp}; \mathsf{rb}) = \mathbf{v}$. We refer to rb as the explanation for $\mathbf{v}$. Our definition of explainable $\mathsf{SampleLeft}$ builds on the definition by Champion, Hsieh and Wu [28, Def. 4.1]. The construction uses the explainable $\mathsf{SamplePre}$ algorithm from [28, Sec. 7], which in turn uses the explainable discrete Gaussian sampler by Lu and Waters [49].

Definition 12 (Explainable $\mathsf{SampleLeft}$). *Let n, m, m_1, q denote the lattice parameters. A $(\rho, \sigma_{\mathsf{loss}})$-explainable $\mathsf{SampleLeft}$ sampler with randomness length $\rho(\lambda, n, m, m_1, q)$ and Gaussian width loss $\sigma_{\mathsf{loss}}(\lambda, n, m, m_1, q)$ is a pair of efficient algorithms $\Pi_{\mathsf{SL}} = (\mathsf{SampleLeft}, \mathsf{ExplainSL})$ with the following syntax:*

- $\mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \mathsf{rb}) \to \mathbf{v}$*: Given matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}$, a gadget trapdoor $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times \tilde{m}}$ for the matrix $\mathbf{A}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$, a Gaussian width parameter σ, and random bits $\mathsf{rb} \in \{0,1\}^{\rho(\lambda, n, m, m_1, q)}$, the sampling algorithm deterministically outputs a vector $\mathbf{v} \in \mathbb{Z}_q^{m+m_1}$.*
- $\mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathbf{v}, \sigma) \to \mathsf{rb}$*: Given a precision parameter κ, matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}$, $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times \tilde{m}}$, and vector $\mathbf{u} \in \mathbb{Z}_q^n$ as defined above, a preimage $\mathbf{v} \in \mathbb{Z}_q^{m+m_1}$, and a Gaussian width parameter σ, the explain algorithm outputs a bit string $\mathsf{rb} \in \{0,1\}^{\rho(\lambda, n, m, m_1, q)}$.*

We require the following properties to hold, except with negligible probability, for all $\lambda \in \mathbb{N}$, matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}, \mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times \tilde{m}}$ such that $\mathbf{A}\mathbf{T}_\mathbf{A} = \mathbf{G}$, target vectors $\mathbf{u} \in \mathbb{Z}_q^n$, where $\|\mathbf{u}\|_\infty \leq 2^\lambda$, and all Gaussian width parameters σ, where $\|\mathbf{T}_\mathbf{A}\|_\infty \cdot \sigma_{\mathsf{loss}}(\lambda, n, m, m_1, q) \leq \sigma \leq 2^\lambda$:

Correctness: *The following distributions are statistically close:*

$$\{\mathbf{v} \leftarrow \mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \mathsf{rb});\ \mathsf{rb} \leftarrow \{0,1\}^\rho\} \ \text{ and } \ \left\{\mathbf{v} \leftarrow [\mathbf{A} \mid \mathbf{M}_1]_\sigma^{-1}(\mathbf{u})\right\}$$

Moreover, for all $\mathbf{v}$ in the support of $\mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \mathsf{rb})$, it holds that $[\mathbf{A} \mid \mathbf{M}_1]\mathbf{v} = \mathbf{u}$.

Explainability: *For all precision parameters $\kappa \in \mathbb{N}$, the statistical distance between the following distributions is bounded by $1/\kappa + \mathsf{negl}(\lambda)$:*

- $\mathsf{DSampleLeft}$*: Sample $\mathsf{rb} \leftarrow \{0,1\}^\rho$ and $\mathbf{v} \leftarrow \mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \mathsf{rb})$. Output $(\mathbf{v}, \mathsf{rb})$.*

– $\mathsf{DExplainLeft}$: *Sample* $\mathsf{rb}' \leftarrow \{0,1\}^\rho$ *and* $\mathbf{v} \leftarrow \mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \mathsf{rb}')$. *Compute* $\mathsf{rb} \leftarrow \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \mathbf{M}_1, \mathbf{T}_\mathbf{A}, \mathbf{u}, \mathbf{v}, \sigma)$ *and output* $(\mathbf{v}, \mathsf{rb})$.

In Appendix C, we build an explainable $\mathsf{SampleLeft}$ sampler called Π_{SL} using the explainable discrete Gaussian preimage sampler (*i.e.*, the explainable $\mathsf{SamplePre}$ sampler) by Champion, Hsieh and Wu [28, Section 7]. The algorithm satisfies the following theorem, whose proof is also presented in Appendix C.

Theorem 1. *There exists a* $(\rho, \sigma_{\mathsf{loss}})$*-explainable* $\mathsf{SampleLeft}$ *sampler* Π_{SL} *for*

$$\rho = \mathsf{poly}(\lambda, n, m, m_1, \log(q))$$
$$\sigma_{\mathsf{loss}}(\lambda, n, m, m_1, q) = 18(m+m_1)^{3/2} \log((m+m_1)\lambda) \log(\log(q)).$$

As in the case of explainable $\mathsf{SamplePre}$ [28, Sec. 7] and the explainable discrete Gaussian sampler [49], our $\mathsf{ExplainSL}$ algorithm runs in time $\mathsf{poly}(\lambda, \kappa)$. In our security proof, $\mathsf{ExplainSL}$ will be used by the LWE adversary $\mathcal{A}$ to simulate the pre-decryption keys of the BIBE system in Section 5. Although a PPT $\mathcal{A}$ cannot use κ such that $1/\kappa$ is negligible in λ, a non-negligible value ensuring a polynomial-time $\mathsf{ExplainSL}$ is nevertheless sufficient for the security reduction. Indeed, suppose there is an adversary $\mathcal{B}$ that wins the security game for the BIBE scheme (Figure 1) with non-negligible probability ϵ. Then, tuning κ such that $1/\kappa = \epsilon/2$ ensures that $\mathcal{B}$ still cannot distinguish between the real game and the one with simulated pre-decryption keys except with probability $\mathsf{negl}(\lambda) + \epsilon/2$, which is sufficient for $\mathcal{A}$ to win the LWE game with non-negligible probability $\epsilon/2 - \mathsf{negl}(\lambda)$.

5 Our Batch IBE System

In this section, we present our full epochless batch IBE (BIBE) system. It uses the parameters $\ell, q, n, m, t, d, \overline{m}, \sigma_1, \sigma_2$, where ℓ denotes the maximum batch size and the rest are specified in Section 5.1. Let $\Pi_{\mathsf{SL}} = (\mathsf{SampleLeft}, \mathsf{ExplainSL})$ denote the explainable $\mathsf{SampleLeft}$ algorithm in Theorem 1 with randomness length ρ and Gaussian width loss σ_{loss}. The BIBE scheme makes use of three hash functions H, H_{ro}, and H_{sp}, with the following properties.

Definition 13 (The hash function H). *The hash function* $H \colon \mathcal{I} \to \{0,1\}^d$ *is an injective function that maps an identity tag* $r \in \mathcal{I}$ *to an index value in* $\{0,1\}^d$.

In the construction below, the size of the index space is set to be 2^d, where $d = \mathsf{poly}(\lambda)$. In practice, this enables generating the index values from the identity tags using a hash function H, while ensuring that indices are distinct except with probability $1/\mathsf{poly}(\log \ell)$. Here, H is also helpful when the identity tag r is a long structured string (*e.g.*, a transaction description). When using a small identity space (*e.g.*, $d = \lceil \log \ell \rceil$), our scheme would be re-using the same sequence of matrices $(A_1, \ldots, A_\ell)$ for each batch, which can be calculated in advance, removing the need for H (however, in this case, it would suffer from index collisions).

Definition 14 (The hash function H_{ro}). *The hash function* $H_{\mathsf{ro}} \colon \mathcal{I} \to \mathbb{Z}_q^{n \times \overline{m}}$ *is a random oracle that maps an identity tag* $r \in \mathcal{I}$ *to a matrix in* $\mathbb{Z}_q^{n \times \overline{m}}$.

Definition 15 (The hash function H_{sp}). *Let* $\rho_{\mathsf{SL}} \coloneqq \rho(\lambda, \ell n, \ell t + \tilde{m}, \ell\overline{m}, q)$ *denote the randomness length of the explainable* $\mathsf{SampleLeft}$ *sampler from Theorem 1 when applied to matrices* $\mathbf{A} \in \mathbb{Z}_q^{\ell n \times (\ell t + \tilde{m})}$ *and* $\hat{\mathbf{A}} \in \mathbb{Z}_q^{\ell n \times \ell\overline{m}}$. *The hash function*

$$H_{\mathsf{sp}} \colon \mathbb{Z}_q^{\ell n \times (\ell t + \tilde{m})} \times \mathbb{Z}_q^{(\ell t + \tilde{m}) \times \ell\tilde{m}} \times \mathcal{I}^\ell \times \mathbb{Z}_q^n \times \mathbb{Z}^+ \to \{0,1\}^{\rho_{\mathsf{SL}}}$$

takes as input a matrix $\mathbf{A} \in \mathbb{Z}_q^{\ell n \times (\ell t + \tilde{m})}$*, a gadget trapdoor* $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{(\ell t+\tilde{m}) \times \ell \tilde{m}}$ *for* $\mathbf{A}$*, identity tags* $(r_1, \ldots, r_\ell) \in \mathcal{I}^\ell$*, a vector* $\mathbf{u} \in \mathbb{Z}_q^n$*, and a Gaussian width parameter* σ*. It outputs the random coins used by* $\mathsf{SampleLeft}$*.*

H_{sp} ensures that the decryptors use the same randomness when they call $\mathsf{SampleLeft}$ as within the pre-decryption algorithm (see Section 2.2.5). In the security proof, H_{ro} and H_{sp} will be modeled as random oracles.

We also let Π_{samp} be a (n, t, q, σ_1)-shifted multi-preimage trapdoor sampler with local expansion from [62, Constr. 4.6] (*cf.* Section 3.5), denoted by $\Pi_{\mathsf{samp}} = (\mathsf{Gen}, \mathsf{GenTD}, \mathsf{Expand}, \mathsf{ExpandLocal})$.

We can now describe the algorithms of our batch IBE scheme.

BIBE.Setup$(1^\lambda, 1^\ell)$**.** The setup algorithm first computes $\mathsf{crs}_{\mathsf{samp}} \leftarrow \mathsf{Gen}(1^\lambda, 1^d)$. It then samples a uniformly-random vector $\mathbf{u} \leftarrow \mathbb{Z}_q^n$ and a tuple $(\mathbf{C}, \mathbf{T}_{\mathbf{C}}) \leftarrow \mathsf{TrapGen}(1^n, q, m)$. The algorithm outputs

$$\mathsf{pk} \coloneqq (\mathsf{crs}_{\mathsf{samp}}, \mathbf{C}, \mathbf{u}) \qquad \mathsf{sk} \coloneqq (\mathsf{pk}, \mathbf{T}_{\mathbf{C}}).$$

BIBE.Enc$(\mathsf{pk}, r \in \mathcal{I}, \mathsf{m} \in \{0,1\})$**.** The encryption algorithm first samples a uniformly-random vector $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ and sets the index value as $\tilde{h} \coloneqq H(r) \in \{0,1\}^d$. It then computes $\mathbf{A}_r \coloneqq \mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}_{\mathsf{samp}}, \tilde{h}) \in \mathbb{Z}_q^{n \times t}$. It next selects the noise vectors $\boldsymbol{\epsilon}_1 \leftarrow \chi$, $\boldsymbol{\epsilon}_2 \leftarrow \chi^m$, and $\boldsymbol{\epsilon}_{3,\mathsf{pre}} \leftarrow \chi^t$, along with a matrix $\tilde{\mathbf{R}} \leftarrow \{-1, 1\}^{t \times \overline{m}}$. Finally, it sets $\boldsymbol{\epsilon}_3 \coloneqq \mathrm{col}(\boldsymbol{\epsilon}_{3,\mathsf{pre}}, \tilde{\mathbf{R}}^\top \boldsymbol{\epsilon}_{3,\mathsf{pre}})$, $\mathbf{F} \coloneqq [\mathbf{A}_r | H_{\mathsf{ro}}(r)] \in \mathbb{Z}_q^{n \times (t+\overline{m})}$, and outputs the ciphertext ct as

$$\mathsf{ct} \coloneqq \big(r, \mathsf{ct}[1], \mathsf{ct}[2], \mathsf{ct}[3]\big) \qquad \text{where} \qquad \begin{cases} \mathsf{ct}[1] \coloneqq \mathbf{u}^\top \mathbf{s} + \boldsymbol{\epsilon}_1 + \mathsf{m} \cdot \lfloor q/2 \rfloor \in \mathbb{Z}_q \\ \mathsf{ct}[2] \coloneqq \mathbf{C}^\top \mathbf{s} + \boldsymbol{\epsilon}_2 \in \mathbb{Z}_q^m \\ \mathsf{ct}[3] \coloneqq \mathbf{F}^\top \mathbf{s} + \boldsymbol{\epsilon}_3 \in \mathbb{Z}_q^{t+\overline{m}} \end{cases}$$

BIBE.PreDec$(\mathsf{sk}, (r_i)_{i \in [\ell]})$**.** The pre-decryption algorithm first sets $\tilde{h}_i \coloneqq H(r_i) \in \{0,1\}^d$ for each $i \in [\ell]$. It requires that $\tilde{h}_1, \ldots, \tilde{h}_\ell$ are distinct vectors; otherwise, it aborts. It then generates the matrices

$$\big(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td} \coloneqq (\mathbf{A}, \mathbf{T}_{\mathbf{A}})\big) = \mathsf{Expand}\big(1^\lambda, 1^\ell, 1^d, \mathsf{crs}_{\mathsf{samp}}, (\tilde{h}_i)_{i \in [\ell]}\big),$$

using the deterministic Expand algorithm, where

$$\mathbf{A} \coloneqq \begin{bmatrix} \mathbf{A}_1 & & & \mathbf{G} \\ & \mathbf{A}_2 & & \mathbf{G} \\ & & \ddots & \vdots \\ & & & \mathbf{A}_\ell \; \mathbf{G} \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times (\ell t + \tilde{m})}, \tag{3}$$

and constructs $\mathbf{h} \coloneqq (\mathbf{u}, \ldots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$ and

$$\hat{\mathbf{A}} \coloneqq \begin{bmatrix} H_{\mathsf{ro}}(r_1) & & & \\ & H_{\mathsf{ro}}(r_2) & & \\ & & \ddots & \\ & & & H_{\mathsf{ro}}(r_\ell) \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times \ell \overline{m}}. \tag{4}$$

Using these matrices, it calls the hash function H_{sp} to obtain the random coins for the $\mathsf{SampleLeft}$ algorithm,

$$\mathsf{rb} \coloneqq H_{\mathsf{sp}}\big(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1\big) \in \{0,1\}^{\rho}.$$

and invokes

$$\mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2) \coloneqq \mathsf{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \mathsf{rb}).$$

Finally, it samples and outputs the following vector as the pre-decryption key:

$$\mathbf{sbk} \leftarrow \mathsf{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{c} \coloneqq \mathbf{G}\hat{\mathbf{c}}, \sigma_2) \in \mathbb{Z}_q^m$$

so that $\mathbf{C} \cdot \mathbf{sbk} = \mathbf{G} \cdot \hat{\mathbf{c}}$.

BIBE.Dec$(\mathsf{pk}, \mathbf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]})$**.** For each $i \in [\ell]$ the decryption algorithm parses the ciphertexts as $(r_i, \mathsf{ct}_i[1], \mathsf{ct}_i[2], \mathsf{ct}_i[3]) \leftarrow \mathsf{ct}_i$. It computes $\tilde{h}_i \coloneqq H(r_i) \in \{0,1\}^d$ for $i \in [\ell]$, and aborts if $\tilde{h}_1, \ldots, \tilde{h}_\ell$ are not distinct vectors. Next, it generates

$$(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td} \coloneqq (\mathbf{A}, \mathbf{T}_{\mathbf{A}})) \coloneqq \mathsf{Expand}\big(1^\lambda, 1^\ell, 1^d, \mathsf{crs}_{\mathsf{samp}}, (\tilde{h}_i)_{i\in[\ell]}\big)$$

and constructs the matrices $\mathbf{A}$ and $\hat{\mathbf{A}}$ as in equations (3) and (4). It then calls the hash function H_{sp} and the algorithm $\mathsf{SampleLeft}$ to obtain

$$\begin{aligned} \mathsf{rb} &\coloneqq H_{\mathsf{sp}}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1) \in \{0,1\}^{\rho} \\ \mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2) &\coloneqq \mathsf{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \mathsf{rb}). \end{aligned}$$

Finally, for each $i \in [\ell]$, it sets $\mathbf{v}_i \coloneqq \mathrm{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$ and outputs m_i' as the message:

$$\mathsf{m}_i' \coloneqq \mathsf{round}\left(\mathsf{ct}_i[1] - \mathbf{sbk}^\top \mathsf{ct}_i[2] - \mathbf{v}_i^\top \mathsf{ct}_i[3]\right).$$

Here, $\mathsf{round}(x)$ outputs 1 if $|x - \lfloor q/2 \rfloor| < \lfloor q/4 \rfloor$, and 0 otherwise.

Remark 1. Both BIBE.PreDec and BIBE.Dec can work with smaller batches of size less than ℓ. The algorithms remain unchanged in this case, except for working with smaller matrices.

5.1 Correctness and Parameter Sizes

Theorem 2. *The* BIBE *scheme is correct, as in Definition 2, whenever the indices are distinct,* $n \geq \lambda$, $m \geq 3n\lceil \log q \rceil$, $t \geq \overline{m} \geq 3n\lceil \log q \rceil$, $t = \tilde{m}(d+1)$,

$$\begin{aligned} &\sigma_2 \geq m \cdot \omega(\sqrt{\log(n)}) \qquad\qquad \alpha = O([\ell^{3/2} t^3 \cdot \omega(\log(2\ell t + \tilde{m})\lambda \log(t))]^{-1}) \\ &\sigma_1 \geq \max\left(18(2\ell t + \tilde{m})^{3/2} \log((2\ell t + \tilde{m})\lambda) \log(\log(q)),\quad 2\overline{m}^{3/2} \cdot \omega(\log(\overline{m} + t))\right) \\ &q = O(\ell^{3/2} t^{7/2}) \cdot \omega(\log(2\ell t + \tilde{m})\lambda \log(t)) \end{aligned}$$

where q is rounded up to the nearest larger prime.

The parameters q and α indirectly bound the norm $\|\mathbf{sbk}\|_2$ by some $B_{\mathsf{th}} < q/2$; so that the norm of the centered lift of $\mathbf{sbk}$ remains bounded by $q/4$, thus enabling successful decryption.

Proof. By assumption, the indices $\tilde{h}_i \coloneqq H(r_i)$ are distinct. Therefore, PreDec does not abort. Now, since $n \geq \lambda$, $q > 2$, $m \geq 3n\lceil \log q \rceil$, $t = \tilde{m}(d+1)$,

$$\sigma_1 \geq 18(2\ell t + \tilde{m})^{3/2} \log\left((2\ell t + \tilde{m})\lambda\right) \log\left(\log\left(q\right)\right),$$

and $\sigma_2 \geq m \cdot \omega(\sqrt{\log(n)})$, by Lemma 10, we make the following observations:

1. The d-bit label variant of [62, Construction 4.6] in Appendix B is an (n, t, q, σ_1)-shifted multi-preimage trapdoor sampler with perfect somewhere programmability and local expansion. Therefore, by the local expansion property, for all $\lambda, \ell \in \mathbb{N}$ and for all crs, given $(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td}) \coloneqq \mathsf{Expand}(1^\lambda, 1^d, \mathsf{crs}, (\tilde{h}_i)_{i\in[\ell]})$, it holds that $\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, \tilde{h}_i) = \mathbf{A}_i$ for all $i \in [\ell]$.
2. $m \geq O(n \log(q))$ and $\sigma_2 \geq m\|\mathbf{T_C}\|_\infty \cdot \omega(\sqrt{\log(n)})$. Then, the outputs of the algorithms $\mathsf{TrapGen}$ and $\mathsf{SamplePre}$ in BIBE.Setup and BIBE.PreDec satisfy Lemma 1.
3. $\rho = \mathsf{poly}(\lambda, n, \ell t + \tilde{m}, \ell t, \log(q))$ and $\sigma_1 \geq \|\mathbf{T_A}\|_\infty \cdot \sigma_{\mathsf{loss}}$, where

$$\sigma_{\mathsf{loss}} = 18(2\ell t + \tilde{m})^{3/2} \log\left((2\ell t + \tilde{m})\lambda\right) \log\left(\log\left(q\right)\right).$$

   Therefore, by Theorem 1, $\Pi_{\mathsf{SL}} = (\mathsf{SampleLeft}, \mathsf{ExplainSL})$ is correct and explainable when used with the Gaussian width parameter σ_1.
4. For all $i \in [\ell]$ and $r \in \mathcal{I}$, the matrices $\mathbf{A}_i$ and $H_{\mathsf{ro}}(r)$ are full-rank (rank n), and there exists a value w such that $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(H_{\mathsf{ro}}(r)) \geq w$. Moreover, $\sigma_1 \geq (q/w)\log(2\ell t)$.

Let $\mu \coloneqq \max(t, \overline{m})$. For a pre-decryption key $\mathbf{sbk}$ and ciphertext ct_i generated via the algorithms BIBE.PreDec and BIBE.Enc, by items (1)-(2)-(3) (*i.e.*, the properties of the shifted multi-preimage trapdoor sampler, Lemma 1 and the correctness of Π_{SL} by Theorem 1), it holds that $\mathbf{F}_{r_i} \cdot \mathbf{v}_i + \mathbf{C} \cdot \mathbf{sbk} = \mathbf{u}$, where $\mathbf{F}_{r_i} = [\mathbf{A}_i | H_{\mathsf{ro}}(r_i)]$. Consequently, for $i \in [\ell]$,

$$\begin{aligned}
\mathsf{ct}_i[1] - &\left(\mathbf{sbk}^\top \mathsf{ct}_i[2] + \mathbf{v}_i^\top \mathsf{ct}_i[3]\right) \\
&= \mathbf{u}^\top \mathbf{s}_i + \boldsymbol{\epsilon}_{1,i} + \mathsf{m}_i \cdot \lfloor q/2 \rfloor - \left(\mathbf{sbk}^\top \mathbf{C}^\top \mathbf{s}_i + \mathbf{sbk}^\top \boldsymbol{\epsilon}_{2,i} + \mathbf{v}_i^\top \mathbf{F}_{r_i}^\top \mathbf{s}_i + \mathbf{v}_i^\top \boldsymbol{\epsilon}_{3,i}\right) \\
&= \left(\mathbf{u} - \mathbf{C} \cdot \mathbf{sbk} - \mathbf{F}_{r_i} \cdot \mathbf{v}_i\right)^\top \mathbf{s}_i + \boldsymbol{\epsilon}_{1,i} - \left(\mathbf{sbk}^\top \boldsymbol{\epsilon}_{2,i} + \mathbf{v}_i^\top \boldsymbol{\epsilon}_{3,i}\right) + \mathsf{m}_i \cdot \lfloor q/2 \rfloor \\
&= \left(\boldsymbol{\epsilon}_{1,i} - \mathbf{sbk}^\top \boldsymbol{\epsilon}_{2,i} - \mathbf{v}_i^\top \boldsymbol{\epsilon}_{3,i}\right) + \mathsf{m}_i \cdot \lfloor q/2 \rfloor
\end{aligned}$$

Recall that $\boldsymbol{\epsilon}_{3,i} \coloneqq \mathrm{col}(\boldsymbol{\epsilon}_{3,i,\mathsf{pre}}, \tilde{\mathbf{R}}_i^\top \boldsymbol{\epsilon}_{3,i,\mathsf{pre}})$. Therefore,

$$\boldsymbol{\epsilon}_{1,i} - \mathbf{sbk}^\top \boldsymbol{\epsilon}_{2,i} - \mathbf{v}_i^\top \boldsymbol{\epsilon}_{3,i} = \boldsymbol{\epsilon}_{1,i} - \mathbf{sbk}^\top \boldsymbol{\epsilon}_{2,i} - (\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \boldsymbol{\epsilon}_{3,i,\mathsf{pre}}$$

Next, we observe that by the correctness of Π_{SL} at item (3) (*i.e.*, Theorem 1), the output of $\mathsf{SampleLeft}$ is statistically indistinguishable from a sample from $\mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})$, where $\mathbf{h} \coloneqq \mathrm{col}(\mathbf{u}, \ldots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$ and $\mathbf{F} = [\mathbf{A}|\hat{\mathbf{A}}]$ (see equations (3) and (4) for $\mathbf{A}$ and $\hat{\mathbf{A}}$). Therefore, by item (4), we can invoke Lemma 18 for $(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$, and infer that $\mathbf{v}_i$ is a Gaussian sample with width parameter σ_1. Combining this with Lemma 16, we observe that for all $i \in [\ell]$, $\|\mathbf{v}_i^1\| \leq \sqrt{t}\,\sigma_1$ and $\|\mathbf{v}_i^2\| \leq \sqrt{\overline{m}}\,\sigma_1$ except with negligible probability. Then, by Lemma 15, and since $\tilde{\mathbf{R}}_i \in \{-1, 1\}^{t \times \overline{m}}$,

$$\|\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2\| \leq \|\mathbf{v}_i^1\| + \|\tilde{\mathbf{R}}_i \mathbf{v}_i^2\| \leq O(\sqrt{t}\,\sigma_1 + \sqrt{\mu\overline{m}}\,\sigma_1) \leq O(\mu\sigma_1)$$

except with negligible probability.

By item (2), $\mathbf{sbk}$ is a Gaussian sample with width parameter σ_2. Combining this with Lemma 16, we have $\|\mathbf{sbk}\| \le \sqrt{m}\,\sigma_2$ except with negligible probability. Hence, by Lemma 26,

$$\begin{aligned}\left|\boldsymbol{\epsilon}_{1,i} - \mathbf{sbk}^\top \boldsymbol{\epsilon}_{2,i} - (\mathbf{v}_i^1 + \tilde{\mathbf{R}}_i \mathbf{v}_i^2)^\top \boldsymbol{\epsilon}_{3,i,\mathsf{pre}}\right| &\le O\left((\mu\sigma_1 + \sqrt{m}\,\sigma_2) q\alpha \cdot \omega(\log \mu)\right) \\ &\quad + O\left(\sigma_1 \mu^{3/2} + \sigma_2 \sqrt{m\mu}\right)\end{aligned}$$

except with negligible probability. Therefore, for decryption to succeed, it suffices that the right-hand side is less than $q/5$. This is ensured by choosing

$$\alpha < \left((\mu\sigma_1 + \sqrt{m}\,\sigma_2) \cdot \omega(\log \mu)\right)^{-1} \qquad q = \Omega\left(\sigma_1 \mu^{3/2} + \sigma_2 \sqrt{m\mu}\right),$$

which are satisfied by the parameter choices in Theorem 2 when $\overline{m} \le t$. For the fully general case, the statement of Theorem 2 should be read with every occurrence of t in the decryption-error bound replaced by $\mu = \max(t, \overline{m})$. □

Given the size of the CRS, the public key scales as

$$|\mathsf{pk}| = n(2t + m) \log q = O(d\lambda^2 \log(\lambda) \log(\ell) + d\lambda^2 \log^2(\ell)),$$

the pre-decryption key scales as

$$|\mathbf{sbk}| = O(m \log \sigma_2) = O(\lambda \log^2 \lambda + \lambda \log \lambda \log \ell),$$

and the ciphertexts scale as

$$|\mathsf{ct}| = (1 + m + t + \overline{m}) \log q.$$

In the asymptotic estimates of Table 1, we take $\overline{m} = O(t)$, and therefore

$$|\mathsf{ct}| = O(d\lambda \log(\lambda) \log(\ell) + d\lambda \log^2(\ell)).$$

For general $\overline{m}$, the ciphertext size should be kept in the form $(1 + m + t + \overline{m}) \log q$.

5.2 Selective Security

Theorem 3. *The* BIBE *scheme satisfies selective security, as in Definition 3, whenever q and α are as in Theorem 2,*

$$\begin{gathered} n \ge \lambda \qquad m \ge 3n\lceil \log q \rceil \qquad \overline{m} \ge 3n\lceil \log q \rceil \qquad d = O(\log |\mathcal{I}|) \qquad \ell = \mathsf{poly}(\lambda) \\ t = \tilde{m}(d+1) \qquad \sigma_2 \ge m \cdot \omega(\sqrt{\log(n)}) \qquad q > 2\sqrt{n}/\alpha \\ \sigma_1 \ge \max\left(18(2\ell t + \tilde{m})^{3/2} \log\left((2\ell t + \tilde{m})\lambda\right) \log(\log(q)),\quad 2\overline{m}^{3/2} \cdot \omega(\log(\overline{m} + t))\right), \end{gathered}$$

the $(\mathbb{Z}_q, n, \chi)$-LWE assumption holds, and H_{ro} and H_{sp} are modeled as random oracles.

Proof. Suppose there exists a PPT adversary $\mathcal{A}$ attacking the BIBE system with non-negligible advantage ϵ using a polynomial number of queries q_{ro} and q_{sp} to H_{ro} and H_{sp}. Let r^* denote the identity tag to be attacked. Consider the following sequence of hybrid games between $\mathcal{A}$ and the challenger Ch:

Hybrid 0: This is the game $\mathsf{Game}^{\mathrm{BIBE}}_{\mathcal{A},b}(1^\lambda, 1^\ell)$ in the random oracle model, where Ch simulates the random oracle queries H_{ro} and H_{sp} by sampling random values from their output distributions and answering consistently across queries.

Hybrid 1: In hybrid 1, Ch samples $\mathbf{A}_{r^*} \leftarrow \mathbb{Z}_q^{n\times t}$ and sets the CRS as $\mathsf{crs}_{\mathsf{samp}} \leftarrow \mathsf{GenProg}(1^\lambda, 1^\ell, \tilde{h}^* := H(r^*), \mathbf{A}_{r^*})$ (see Definition 11 for $\mathsf{GenProg}$).

Hybrid 2: Let $\tilde{\mathbf{R}}^* \in \{-1,1\}^{t\times\overline{m}}$ denote the random matrix sampled while creating ct^* at the challenge round. In hybrid 2, Ch chooses $\tilde{\mathbf{R}}^*$ during the setup. It then programs the random oracle H_{ro} so that $H_{\mathsf{ro}}(r^*) := \mathbf{A}_{r^*}\tilde{\mathbf{R}}^*$.

Hybrid 3: Ch follows the same steps as in hybrid 2, except that it returns $\mathbf{sbk} \leftarrow \mathbf{C}_{\sigma_2}^{-1}(\mathbf{c})$ as the pre-decryption key, where $\mathbf{c} = \mathbf{G}\hat{\mathbf{c}}$, and $\hat{\mathbf{c}}$ was obtained by invoking $\mathsf{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \mathsf{rb})$.

Hybrid 4: The matrix $\mathbf{C}$ is sampled uniformly at random: $\mathbf{C} \leftarrow \mathbb{Z}_q^{n\times m}$. When H_{ro} is queried with an identity $r \neq r^*$, Ch programs $H_{\mathsf{ro}}(r)$ using $\mathsf{TrapGen}$: $(H_{\mathsf{ro}}(r), \mathbf{T}_r) \leftarrow \mathsf{TrapGen}(1^n, q, \overline{m})$, where $\mathbf{T}_r$ is a trapdoor for $H_{\mathsf{ro}}(r)$.

Hybrid 5: Upon receiving a new query $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1)$ for the random oracle H_{sp}, Ch constructs the matrix $\hat{\mathbf{A}}$ as in equation (4) from H_{ro} and the identity tags $(r_i)_{i\in[\ell]}$. It then samples a vector $\mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})$, where $\mathbf{F} = [\mathbf{A}|\hat{\mathbf{A}}]$ and $\mathbf{h} := (\mathbf{u}, \ldots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$, and obtains $\mathsf{rb} = \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1)$, with precision parameter $\kappa := 2q_{\mathsf{sp}}/\epsilon$. It stores the tuple $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1, \mathbf{v}; \mathsf{rb})$ and returns rb to $\mathcal{A}$.

Hybrid 6: Upon receiving a new query $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1)$, where $\mathbf{A}$ contains the matrices $\mathbf{A}_{r_i}$, for the random oracle H_{sp}, Ch does the following to respond to the query:

- Ch first checks if $r^* \in (r_i)_{i\in[\ell]}$. Then, there are two cases.
- **Case 1**: $r^* = r_{i^*} \in (r_i)_{i\in[\ell]}$ for some index i^*:
  - Ch samples $\mathbf{v}_{i^*} = \mathrm{col}(\mathbf{v}_{i^*}^1, \mathbf{v}_{i^*}^2) \in \mathcal{D}_{\mathbb{Z}_q,\sigma_1}^{t+\overline{m}}$ and sets $\mathbf{c} := -\mathbf{F}_{r^*}\mathbf{v}_{i^*} + \mathbf{u}$, where $\mathbf{F}_{r^*} = [\mathbf{A}_{r^*}|H_{\mathsf{ro}}(r^*)]$. It then samples $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ using the algorithm in Lemma 21 and sets $\mathbf{sbk} := \perp$.
  - For $i \in [\ell]$, $i \neq i^*$, Ch invokes $\mathsf{SampleLeft}(H_{\mathsf{ro}}(r_i), \mathbf{A}_{r_i}, \mathbf{T}_{r_i}, \mathbf{u}-\mathbf{c}, \sigma_1)$ and permutes the output components to obtain $\mathbf{v}_i = \mathrm{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$; so that $\mathbf{v}_i^1 \in \mathbb{Z}_q^t$ corresponds to $\mathbf{A}_{r_i}$ and $\mathbf{v}_i^2 \in \mathbb{Z}_q^{\overline{m}}$ to $H_{\mathsf{ro}}(r_i)$.
- **Case 2**: $r^* \notin (r_i)_{i\in[\ell]}$:
  - Ch samples $\mathbf{sbk} \leftarrow \mathcal{D}_{\mathbb{Z}_q,\sigma_2}^m$, sets $\mathbf{c} := \mathbf{C}\cdot\mathbf{sbk}$, and samples $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ using the algorithm in Lemma 21.
  - For all $i \in [\ell]$, Ch obtains $\mathbf{v}_i = \mathrm{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$ by invoking and permuting the output of $\mathsf{SampleLeft}(H_{\mathsf{ro}}(r_i), \mathbf{A}_{r_i}, \mathbf{T}_{r_i}, \mathbf{u}-\mathbf{c}, \sigma_1)$.
- Finally, setting $\mathbf{v} := \mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$, Ch obtains

$$\mathsf{rb} := \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1),$$

  stores $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1, \mathbf{v}, \mathbf{sbk}, \mathsf{rb})$, and returns rb to $\mathcal{A}$.

The permutation step is distribution-preserving: $\mathsf{SampleLeft}(H_{\mathsf{ro}}(r_i), \mathbf{A}_{r_i}, \mathbf{T}_{r_i}, \mathbf{u}-\mathbf{c}, \sigma_1)$ outputs a sample from $[H_{\mathsf{ro}}(r_i) \mid \mathbf{A}_{r_i}]_{\sigma_1}^{-1}(\mathbf{u}-\mathbf{c})$, and since the spherical discrete Gaussian $\mathcal{D}_{\mathbb{Z}^{t+\overline{m}},\sigma_1}$ is invariant under coordinate permutation, swapping the two output blocks yields an *identically* distributed sample from $[\mathbf{A}_{r_i} \mid H_{\mathsf{ro}}(r_i)]_{\sigma_1}^{-1}(\mathbf{u}-\mathbf{c})$, matching the column order of $\mathbf{F} = [\mathbf{A} \mid \hat{\mathbf{A}}]$ in hybrid 5. Note that Ch still responds to the pre- and post-challenge queries as in hybrid 5, instead of returning the $\mathbf{sbk}$ above.

Hybrid 7: Given some pre- or post-challenge query $(r_i)_{i\in[\ell]}$, Ch first checks if $r^* \in (r_i)_{i\in[\ell]}$. If so, it aborts the game. Otherwise, Ch constructs the matrix $\mathbf{A}$ as in equation (3). Without loss of generality, suppose $\mathcal{A}$ has already queried H_{sp} with the input $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1)$. In that case, Ch retrieves the tuple $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1, \mathbf{v} \coloneqq \mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2), \mathbf{sbk}, \mathsf{rb})$ and returns $\mathbf{sbk}$.

Completing the proof. To complete the proof, let $\mathsf{Hybrid}_i(\mathcal{A})$ denote the output distribution of hybrid i with adversary $\mathcal{A}$. We argue that the adversary's advantage mostly changes negligibly from hybrid to hybrid, and moreover an adversary with non-negligible advantage in Hybrid 7 can be used to break the LWE assumption.

Lemma 3. *If Π_{samp} is somewhere programmable, then for all adversaries $\mathcal{A}$, $\mathsf{Hybrid}_0(\mathcal{A}) \approx_s \mathsf{Hybrid}_1(\mathcal{A})$.*

Proof. The only difference between the two games is the distribution of $(A_{r^*}, \mathsf{crs}_{\mathsf{samp}})$, which are statistically indistinguishable by somewhere programmability of Π_{samp}. □

Lemma 4. *Suppose $n \geq \lambda$ and $t \geq (n+1)\log(q) + \omega(\log(n))$. Then, for all adversaries $\mathcal{A}$, $\mathsf{Hybrid}_1(\mathcal{A}) \approx_s \mathsf{Hybrid}_2(\mathcal{A})$.*

Proof. Recall that $\tilde{\mathbf{R}}^*$ is used in the calculation of $\boldsymbol{\epsilon}_{3,i}^* \coloneqq \mathrm{col}(\boldsymbol{\epsilon}_{3,i,\mathsf{pre}}^*, (\tilde{\mathbf{R}}^*)^\top \boldsymbol{\epsilon}_{3,i,\mathsf{pre}}^*)$. Since $t \geq (n+1)\log(q) + \omega(\log(n))$, by Lemma 25, $(\mathbf{A}_{r^*}, \mathbf{A}_{r^*}\tilde{\mathbf{R}}^*, (\tilde{\mathbf{R}}^*)^\top \boldsymbol{\epsilon}_{3,i,\mathsf{pre}}^*)$ has a distribution statistically close to that of $(\mathbf{A}_{r^*}, \mathbf{A}'_{r^*}, (\tilde{\mathbf{R}}^*)^\top \boldsymbol{\epsilon}_{3,i,\mathsf{pre}}^*)$ for any $\boldsymbol{\epsilon}_{3,i,\mathsf{pre}}^*$, where $\mathbf{A}'_{r^*} \leftarrow \mathbb{Z}_q^{n\times\overline{m}}$. Thus, $\mathsf{Hybrid}_1(\mathcal{A}) \approx_s \mathsf{Hybrid}_2(\mathcal{A})$. □

Lemma 5. *Suppose $\sigma_2 \geq m\|\mathbf{T}_{\mathbf{C}}\|_\infty \cdot \omega(\sqrt{\log(n)})$. Then, for all PPT adversaries $\mathcal{A}$, $\mathsf{Hybrid}_2(\mathcal{A}) \approx_s \mathsf{Hybrid}_3(\mathcal{A})$.*

Proof. By Lemma 1, the distribution of $\mathbf{sbk} \leftarrow \mathsf{SamplePre}(\mathbf{C}, \mathbf{T}_{\mathbf{C}}, \mathbf{c}, \sigma_2)$ in hybrid 2 is statistically indistinguishable from its distribution in hybrid 3, namely, $\mathbf{sbk} \leftarrow \mathbf{C}_{\sigma_2}^{-1}(\mathbf{c})$. Since $\mathcal{A}$ makes polynomially many pre-decryption and random oracle queries, the lemma follows by a union bound. □

Lemma 6. *Suppose $n \geq \lambda$ and $m, \overline{m} \geq O(n\log(q))$. Then, for all PPT adversaries $\mathcal{A}$, $\mathsf{Hybrid}_3(\mathcal{A}) \approx_s \mathsf{Hybrid}_4(\mathcal{A})$.*

Proof. In hybrid 3, $(\mathbf{C}, \mathbf{T}_{\mathbf{C}}) \leftarrow \mathsf{TrapGen}(1^n, q, m)$ and $H_{\mathsf{ro}}(r) \leftarrow \mathbb{Z}_q^{n\times\overline{m}}$ for $r \neq r^*$, while in hybrid 4, $\mathbf{C} \leftarrow \mathbb{Z}_q^{n\times m}$ and $(H_{\mathsf{ro}}(r), \mathbf{T}_r) \leftarrow \mathsf{TrapGen}(1^n, q, \overline{m})$ for $r \neq r^*$. By Lemma 1, the distributions of $\mathbf{C}$ and of each $H_{\mathsf{ro}}(r)$ are statistically indistinguishable between these two games. Since $\mathcal{A}$ makes polynomially many queries to H_{ro}, the lemma follows by a union bound. □

Lemma 7. *Suppose $q > 2$, $t > n$, $\rho = \mathsf{poly}(\lambda, n, \ell t + \tilde{m}, \ell t, \log(q))$, and for all matrices $\mathbf{A}$ obtained after a pre-decryption query, $\sigma_1 \geq \|\mathbf{T}_{\mathbf{A}}\|_\infty \cdot \sigma_{\mathsf{loss}}$ for $\sigma_{\mathsf{loss}} = 18(2\ell t+\tilde{m})^{3/2}\log((2\ell t+\tilde{m})\lambda)\log(\log(q))$. Then, for all PPT $\mathcal{A}$, the statistical distance between $\mathsf{Hybrid}_4(\mathcal{A})$ and $\mathsf{Hybrid}_5(\mathcal{A})$ is bounded by $\mathsf{negl}(\lambda) + \epsilon/2$.*

Proof. By Theorem 1, $\Pi_{\mathsf{SL}} = (\mathsf{SampleLeft}, \mathsf{ExplainSL})$ is correct and explainable when used with the Gaussian width parameter σ_1. Defining $\mathbf{F} = [\mathbf{A} \mid \hat{\mathbf{A}}]$, we observe that

$$\begin{aligned}&\left\{(\mathbf{v}, \mathsf{rb}') \colon \ \mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h}),\ \mathsf{rb}' \coloneqq \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1)\right\}\\ \approx_s &\{(\mathbf{v}, \mathsf{rb}') \colon \ \mathsf{rb} \leftarrow \{0,1\}^\rho,\ \mathbf{v} \coloneqq \mathsf{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \mathsf{rb}),\\ &\qquad\qquad \mathsf{rb}' \coloneqq \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1)\},\end{aligned}$$

which follows from correctness.

Now, for any given $\kappa \in \mathbb{N}$ that is $\mathsf{poly}(\lambda)$, by the explainability of Π_{SL}, there exists a polynomial time $\mathsf{ExplainSL}$ algorithm such that the statistical distance between the latter distribution and

$$\left\{(\mathbf{v}, \mathsf{rb}') \colon \ \mathsf{rb}' \leftarrow \{0,1\}^\rho,\ \mathbf{v} \coloneqq \mathsf{SampleLeft}(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \mathsf{rb}')\right\}$$

is bounded by $\mathsf{negl}(\lambda) + 1/\kappa$ for all adversaries. As $\kappa = 2q_{\mathsf{sp}}/\epsilon$ and $q_{\mathsf{sp}} = \mathsf{poly}(\lambda)$, $\mathsf{ExplainSL}$ invoked by Ch remains polynomial time, and the statistical distance between hybrids 4 and 5 is bounded by $q_{\mathsf{sp}}(\mathsf{negl}(\lambda) + 1/\kappa) = \mathsf{negl}(\lambda) + \epsilon/2$. □

Lemma 8. *Suppose*

1. $q > 2$, $t, m, \overline{m} \geq 2n \log(q)$, *and* $\sigma_1, \sigma_2 \geq \omega(\sqrt{\log(n)})$.
2. *For all identity tags r in the pre-decryption queries,* $\sigma_1 \geq 2\overline{m}^{3/2} \|\mathbf{T}_r\|_\infty \cdot \omega(\log(\overline{m} + t))$, *where $\mathbf{T}_r$ is the gadget trapdoor for $H_{\mathsf{ro}}(r)$ generated by* $\mathsf{TrapGen}$.
3. *For all identity tags r_i in the pre-decryption queries, the matrices $\mathbf{A}_{r_i}$ and $H_{\mathsf{ro}}(r_i)$ are full-rank (rank n), and $\lambda_1^\infty(\mathbf{A}_{r_i}), \lambda_1^\infty(H_{\mathsf{ro}}(r_i)) \geq w$ for a fixed value w. Moreover, $\sigma_1 \geq (q/w)\log(2\ell t)$.*

Then, for all PPT adversaries $\mathcal{A}$, $\mathsf{Hybrid}_5(\mathcal{A}) \approx_s \mathsf{Hybrid}_6(\mathcal{A})$.

Proof. Consider the following two cases in hybrid 6:

Case 1: $r^* = r_{i^*}$ for some i^*. Since $H_{\mathsf{ro}}(r^*) = \mathbf{A}_{r^*}\tilde{\mathbf{R}}^*$, it holds that

$$\mathbf{F}_{r^*} \coloneqq [\mathbf{A}_{r^*} \mid H_{\mathsf{ro}}(r^*)] = [\mathbf{A}_{r^*} \mid \mathbf{A}_{r^*}\tilde{\mathbf{R}}^*]$$

Here, as $\mathbf{A}_{r^*}$ was drawn from a uniform distribution, $\mathbf{A}_{r^*}\tilde{\mathbf{R}}^*$ is uniform in $\mathbb{Z}_q^{n \times \overline{m}}$ and independent of the other matrices by the standard leftover hash lemma. Therefore, by assumption (1) and Lemma 17, $\mathbf{c} \coloneqq -\mathbf{F}_{r^*} \cdot \mathbf{v}_{i^*} + \mathbf{u}$ in hybrid 6 is statistically close to a uniform sample $\mathbf{c} \leftarrow \mathbb{Z}_q^n$. Moreover, by Lemma 22, for uniformly-random $\mathbf{c} \leftarrow \mathbb{Z}_q^n$, $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ is statistically close to a sample $\hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^m$.

Case 2: $r^* \notin (r_i)_{i \in [\ell]}$**.** Again by assumption (1) and Lemma 17, $\mathbf{c} \coloneqq \mathbf{C} \cdot \mathbf{sbk}$ in hybrid 6 is statistically close to a uniform sample $\mathbf{c} \leftarrow \mathbb{Z}_q^n$. Again by Lemma 22, $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ is statistically close to a sample $\hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z}_q, \sigma_1}^m$.

In both cases, for $i \in [\ell]$, $r_i \neq r^*$, Ch invokes $\mathsf{SampleLeft}(H_{\mathsf{ro}}(r_i), \mathbf{A}_{r_i}, \mathbf{T}_{r_i}, \mathbf{u} - \mathbf{c}, \sigma_1)$ using the trapdoor $\mathbf{T}_{r_i}$ for $H_{\mathsf{ro}}(r_i)$ obtained from $\mathsf{TrapGen}$. By assumption (2) and Lemma 24, the output (after permuting coordinates) is statistically close to a sample from $([\mathbf{A}_{r_i} | H_{\mathsf{ro}}(r_i)])_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{c}) = (\mathbf{F}_{r_i})_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{c})$. Here, coordinate permutation preserves the discrete Gaussian distribution since permutation matrices are orthogonal. In case (1), by the definition of $\mathbf{c}$, $\mathbf{v}_{i^*}$ is statistically close to a sample from $(\mathbf{F}_{r^*})_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{c})$.

Finally, by assumption (3) and Lemma 18, the statistical distance between the following distributions is negligible for $\mathbf{F} = [\mathbf{A} \mid \hat{\mathbf{A}}]$ and $\mathbf{h} = (\mathbf{u}, \ldots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$:

$$\{\mathbf{v} : \mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h})\} \quad \text{and}$$
$$\left\{ \mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2) : \substack{\hat{\mathbf{c}} \leftarrow \mathcal{D}^m_{\mathbb{Z}_q, \sigma_1} \\ \mathrm{col}(\mathbf{v}_{r_i}^1, \mathbf{v}_{r_i}^2) \leftarrow ([\mathbf{A}_{r_i} | H_{\mathsf{ro}}(r_i)]_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{G}\hat{\mathbf{c}})} \right\}$$

Hence, the distribution of rb in hybrid 5 is statistically close to its distribution in hybrid 6:

$$\left\{ (\mathbf{v}, \mathsf{rb}) \colon \mathbf{v} \leftarrow \mathbf{F}_{\sigma_1}^{-1}(\mathbf{h}), \mathsf{rb} \coloneqq \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1) \right\} \quad \text{and}$$
$$\approx_s \left\{ (\mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2), \mathsf{rb}) \colon \substack{\hat{\mathbf{c}} \leftarrow \mathcal{D}^m_{\mathbb{Z}_q, \sigma_1} \\ \mathrm{col}(\mathbf{v}_{r_i}^1, \mathbf{v}_{r_i}^2) \leftarrow ([\mathbf{A}_{r_i} | H_{\mathsf{ro}}(r_i)]_{\sigma_1}^{-1}(\mathbf{u} - \mathbf{G}\hat{\mathbf{c}}) \\ \mathsf{rb} \coloneqq \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1)} \right\}$$

Since $\mathcal{A}$ makes polynomially many pre-decryption and random oracle queries, the lemma follows by a union bound. □

Lemma 9. *For all adversaries $\mathcal{A}$, $\mathsf{Hybrid}_6(\mathcal{A}) \equiv \mathsf{Hybrid}_7(\mathcal{A})$.*

Proof. Since $\mathbf{sbk} \leftarrow \mathcal{D}^m_{\mathbb{Z}_q, \sigma_2}$ and $\mathbf{c} \coloneqq \mathbf{C} \cdot \mathbf{sbk}$ in hybrid 6, the distribution of the pre-decryption key in hybrid 7 is indistinguishable from its distribution in hybrid 6. □

Lemma 10. *Suppose $n \geq \lambda$, $q > 2$, $m \geq 3n\lceil \log q \rceil$, $t = \tilde{m}(d+1)$, $\overline{m} \geq 3n\lceil \log q \rceil$,*

$$\sigma_1 \geq \max\left(18(2\ell t + \tilde{m})^{3/2} \log\left((2\ell t + \tilde{m})\lambda\right) \log\left(\log\left(q\right)\right), \quad 2\overline{m}^{3/2} \cdot \omega(\log(\overline{m} + t)) \right)$$

and $\sigma_2 \geq m \cdot \omega(\sqrt{\log(n)})$. Then, except with negligible probability,

1. *The construction in Appendix B is an (n, t, q, σ_1)-shifted multi-preimage trapdoor sampler with d-bit labels, perfect somewhere programmability and local expansion.*
2. *$\rho = \mathsf{poly}(\lambda, n, \ell t + \tilde{m}, \ell\overline{m}, \log(q))$, and for all matrices $\mathbf{A}$ obtained after a pre-decryption query, $\sigma_1 \geq \|\mathbf{T}_{\mathbf{A}}\|_\infty \cdot \sigma_{\mathsf{loss}}$ for*

$$\sigma_{\mathsf{loss}} = 18(2\ell t + \tilde{m})^{3/2} \log\left((2\ell t + \tilde{m})\lambda\right) \log\left(\log\left(q\right)\right).$$

3. *For all identity tags r in the pre-decryption queries, $\sigma_1 \geq 2\overline{m}^{3/2}\|\mathbf{T}_r\|_\infty \cdot \omega(\log(\overline{m} + t)) \geq \omega(\sqrt{\log(n)})$, where $\mathbf{T}_r$ is the gadget trapdoor for $H_{\mathsf{ro}}(r)$ generated by $\mathsf{TrapGen}$.*
4. *$\sigma_2 \geq m\|\mathbf{T}_{\mathbf{C}}\|_\infty \cdot \omega(\sqrt{\log(n)}) \geq \omega(\sqrt{\log(n)})$.*
5. *$t, m, \overline{m}, \tilde{m} \geq O(n \log(q))$ and $t \geq (n+1)\log(q) + \omega(\log(n))$.*
6. *For all identity tags r_i in the pre-decryption queries, the matrices $\mathbf{A}_{r_i}$ and $H_{\mathsf{ro}}(r_i)$ are full-rank (rank n), and $\lambda_1^\infty(\mathbf{A}_{r_i}), \lambda_1^\infty(H_{\mathsf{ro}}(r_i)) \geq w$ for a fixed value w. Moreover, $\sigma_1 \geq (q/w)\log(2\ell t)$.*

Here, $s_{\mathbf{R}} \coloneqq \sup_{\|\mathbf{x}\|=1} \|\mathbf{R}\mathbf{x}\|$.

Proof. Since $n \geq \lambda$, $m = 3n\lceil \log q \rceil$, $t = \tilde{m}(d+1)$, and

$$\sigma_1 \geq 18(2\ell t + \tilde{m})^{3/2} \log\left((2\ell t + \tilde{m})\lambda\right) \log\left(\log\left(q\right)\right) \geq (\ell t + \tilde{m}) \log(\ell n),$$

by Lemma 2, the construction in Appendix B is an (n, t, q, σ_1)-shifted multi-preimage trapdoor sampler with d-bit labels, perfect somewhere programmability and local expansion (item 1). Moreover, for all matrices $\mathbf{A}$ obtained after a pre-decryption query, $\|\mathbf{T}_\mathbf{A}\|_\infty \leq 1$ and the lower bound on σ_1 together imply $\sigma_1 \geq \|\mathbf{T}_\mathbf{A}\|_\infty \cdot \sigma_{\mathsf{loss}}$ (item 2).

Since $\|\mathbf{T}_r\|_\infty \leq 1$ for any identity tag r by Lemma 1, $2\overline{m}^{3/2}\|\mathbf{T}_r\|_\infty \cdot \omega(\log(\overline{m}+t)) \leq 2\overline{m}^{3/2} \cdot \omega(\log(\overline{m}+t)) \leq \sigma_1$ for all identity tags r in the pre-decryption queries (item 3).

Since $\|\mathbf{T}_\mathbf{C}\|_\infty \leq 1$ by Lemma 1, $\omega(\sqrt{\log(n)}) \leq m\|\mathbf{T}_\mathbf{C}\|_\infty \cdot \omega(\sqrt{\log(n)}) \leq m \cdot \omega(\sqrt{\log(n)}) \leq \sigma_2$ (item 4).

Item 5 follows from the assumption on the sizes of m, t, $\overline{m}$, and $\tilde{m}$.

By Lemmas 1 and 20, for any given identity tag r_i, $\lambda_1^\infty(\mathbf{A}_{r_i}), \lambda_1^\infty(H_{\mathsf{ro}}(r)) \geq w = q/4$, which implies that $(q/w)\log(2\ell t) \leq 4\log(2\ell t) \leq \sigma_1$, except with negligible probability. Furthermore, by Lemmas 1 and 2, $\mathbf{A}_{r_i}$ is full-rank; and by Lemma 1, $H_{\mathsf{ro}}(r_i)$ generated by $\mathsf{TrapGen}$ is full-rank except with negligible probability. Then, item (6) follows by a union bound over the identity tags of the pre-decryption queries. □

Conditioned on the events in Lemma 10, which imply that the assumptions for the previous lemmas hold, the statistical distance between $\mathsf{Hybrid}_1(\mathcal{A})$ and $\mathsf{Hybrid}_7(\mathcal{A})$ is $\mathsf{negl}(\lambda) + (\epsilon/4)$. By Lemma 10, since these events happen except with negligible probability, the statistical distance between $\mathsf{Hybrid}_1(\mathcal{A})$ and $\mathsf{Hybrid}_7(\mathcal{A})$ is indeed $\mathsf{negl}(\lambda) + (\epsilon/2)$. Hence, to conclude the proof of Theorem 3, we construct an adversary $\mathcal{B}$ that solves the $(\mathbb{Z}_q, n, \chi)$-LWE problem by acting as a challenger of hybrid 7 for $\mathcal{A}$. Upon receiving the LWE challenges $((\mathbf{y}_i, z_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q)_{i\in[t+m+1]}$, at the setup, $\mathcal{B}$ sets

$$\mathbf{u} \coloneqq \mathbf{y}_0 \qquad \mathbf{C} \coloneqq [\mathbf{y}_1, \ldots, \mathbf{y}_m] \qquad \mathbf{A}_{r^*} \coloneqq [\mathbf{y}_{m+1}, \ldots, \mathbf{y}_{m+t}]$$

It constructs the rest of the public key as in hybrid 7, and answers the pre-decryption queries as in hybrid 7.

For the challenge ciphertext, $\mathcal{B}$ sets

$$\mathsf{ct}^*[1] \coloneqq z_0 + \mathsf{m}_b\lfloor q/2\rfloor \in \mathbb{Z}_q \qquad \mathsf{ct}^*[2] \coloneqq \mathrm{col}(z_1, \ldots z_m) \in \mathbb{Z}_q^m$$
$$\mathsf{ct}^*[3] \coloneqq \begin{bmatrix} \mathrm{col}(z_{m+1}, \ldots, z_{m+t}) \\ (\tilde{\mathbf{R}}^*)^\top \mathrm{col}(z_{m+1}, \ldots, z_{m+t}) \end{bmatrix} \in \mathbb{Z}_q^{t+\overline{m}}$$

We note that when the LWE oracle is pseudorandom (*i.e.*, $\mathcal{O} = \mathcal{O}_s$), then ct^* has the same distribution as in hybrid 7. This is because $\mathbf{F}_{r^*} = [\mathbf{A}_{r^*}|H_{\mathsf{ro}}(r^*)] = [\mathbf{A}_{r^*}|\mathbf{A}_{r^*}\tilde{\mathbf{R}}^*]$ in hybrid 7, and by the definition of $\mathcal{O}_s$,

$$\begin{aligned}
\mathsf{ct}^*[1] &= z_0 + \mathsf{m}_b\lfloor q/2\rfloor = (\mathbf{u})^\top\mathbf{s} + \boldsymbol{\epsilon}_1^* \\
\mathsf{ct}^*[2] &= \mathrm{col}(z_1, \ldots, z_m) = [\mathbf{y}_1, \ldots, \mathbf{y}_m]^\top \mathbf{s} + \boldsymbol{\epsilon}_2^* = \mathbf{C}^\top\mathbf{s} + \boldsymbol{\epsilon}_2^* \\
\mathsf{ct}^*[3] &= \begin{bmatrix} \mathrm{col}(z_{m+1}, \ldots, z_{m+t}) \\ (\tilde{\mathbf{R}}^*)^\top \mathrm{col}(z_{m+1}, \ldots, z_{m+t}) \end{bmatrix} = \begin{bmatrix} [\mathbf{y}_{m+1}, \ldots, \mathbf{y}_{m+t}]^\top \mathbf{s} + \boldsymbol{\epsilon}_{3,\mathsf{pre}}^* \\ (\tilde{\mathbf{R}}^*)^\top [\mathbf{y}_{m+1}, \ldots, \mathbf{y}_{m+t}]^\top \mathbf{s} + (\tilde{\mathbf{R}}^*)^\top \boldsymbol{\epsilon}_{3,\mathsf{pre}}^* \end{bmatrix} \\
&= \begin{bmatrix} \mathbf{A}_{r^*}^\top\mathbf{s} + \boldsymbol{\epsilon}_{3,\mathsf{pre}}^* \\ (\tilde{\mathbf{R}}^*)^\top\mathbf{A}_{r^*}^\top\mathbf{s} + (\tilde{\mathbf{R}}^*)^\top\boldsymbol{\epsilon}_{3,\mathsf{pre}}^* \end{bmatrix} = \mathbf{F}_{r^*}^\top\mathbf{s} + \begin{bmatrix} \boldsymbol{\epsilon}_{3,\mathsf{pre}}^* \\ (\tilde{\mathbf{R}}^*)^\top\boldsymbol{\epsilon}_{3,\mathsf{pre}}^* \end{bmatrix} = \mathbf{F}_{r^*}^\top\mathbf{s} + \boldsymbol{\epsilon}_3^*
\end{aligned}$$

for some short noise vectors $\boldsymbol{\epsilon}_1^* \leftarrow \chi, \boldsymbol{\epsilon}_2^* \leftarrow \chi^m$ and $\boldsymbol{\epsilon}_{3,\mathsf{pre}}^* \leftarrow \chi^t$.

Now, when $\mathcal{O} = \mathcal{O}_{\$}$, $\mathsf{ct}^*[1]$ and $\mathsf{ct}^*[2]$ are uniformly-random samples from $\mathbb{Z}_q$ and $\mathbb{Z}_q^m$ respectively. Similarly, $\mathrm{col}(z_{m+1}, \ldots, z_{m+t})$ is a random sample from $\mathbb{Z}_q^t$, and $(\tilde{\mathbf{R}}^*)^\top \mathrm{col}(z_{m+1}, \ldots, z_{m+t})$ is uniform in $\mathbb{Z}_q^{\overline{m}}$ and independent of the other vectors by the standard leftover hash lemma. Here, the hash function is defined by the matrix $[\mathbf{A}_{r^*}^\top | \mathrm{col}(z_{m+1}, \ldots, z_{m+t})]$ and ensures that $\mathbf{A}_{r^*}\tilde{\mathbf{R}}^*$ and $(\tilde{\mathbf{R}}^*)^\top \mathrm{col}(z_{m+1}, \ldots, z_{m+t})$ are uniformly distributed and independent quantities. Hence, when $\mathcal{O} = \mathcal{O}_{\$}$, the challenge ciphertext is always uniform in $\mathbb{Z}_q \times \mathbb{Z}_q^m \times \mathbb{Z}_q^{t+\overline{m}}$.

Finally, we observe that if $\mathcal{A}$ has advantage ϵ, then $\mathcal{B}$ has advantage at least $\epsilon - ((\epsilon/2) + \mathsf{negl}(\lambda)) = \epsilon/2 - \mathsf{negl}(\lambda)$ in deciding the $(\mathbb{Z}_q, n, \chi)$-LWE problem. This concludes the proof. □

5.3 Multi-bit Encryption and Adaptive Security

To encrypt messages with N bits, the public parameters can be made to include N vectors $\mathbf{u}_1, \ldots, \mathbf{u}_N$, each used to encrypt one of the message bits with the same ephemeral $\mathbf{s} \in \mathbb{Z}_q^n$. This increases the size of $\mathsf{ct}_r[1]$ by a factor of N, while keeping $\mathsf{ct}_r[2]$ and $\mathsf{ct}_r[3]$ the same. The security proof remains largely unchanged, except that the challenger must now query the LWE oracle $N + m + t$ times instead of $1 + m + t$ times. An alternative method would be representing the messages as integers in the range $[0, 2^\ell]$, and calculating $\mathsf{ct}_r[1]$ as $\mathsf{ct}_r[1] \coloneqq \mathbf{u}^\top \mathbf{s}_r + \boldsymbol{\epsilon}_{1,r} + \mathsf{m} \cdot \lfloor q/2^N \rfloor$. This in turn requires the error term to be less than $q/(1 + 2^{N+1})$, which increases q by a factor of 2^N. Whereas the first solution increases the ciphertext size by $O(N(\log(\ell) + \log(\lambda)))$ bits, the latter results in a factor of N increase.

To move from static to adaptive security, we can design our BIBE system starting from the adaptively-secure version of the IBE scheme by Agrawal, Boneh, Boyen [5], which in turn is based on a similar technique by Waters [61].

Remark 2. Among the matrices $\mathsf{crs}_{\mathsf{samp}} = [\mathbf{A} \mid \mathbf{B}]$, $\mathbf{C}$ and $\mathbf{u}$ that constitute the public keys of our BIBE system (see Section 3.5 for the structure of $\mathsf{crs}_{\mathsf{samp}}$), $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{u}$ are randomly-generated. Thus, they can instead be generated through a random oracle evaluation. With this modification, the public key only includes the matrix $\mathbf{C}$ of size

$$|\mathsf{pk}| = O(nm \log(q)) = O(n^2 \log^2(q)) = O(\lambda^2 \log^2(\ell) + \lambda^2 \log^2(\lambda))$$

The only change to the proof of security in Theorem 3 is that whenever the simulator chooses the matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{u}$, it will instead program the random oracle used to generate those matrices.

6 Threshold Batch IBE System

We next thresholdize our BIBE scheme in Section 5. Recall that when the number of parties N is small, the batch IBE scheme can be thresholdized non-interactively at the expense of an N factor increase in ciphertext and key sizes. Alternatively, one can use the non-interactive protocol in [12, §3]; however, this requires either trusted setup or a multi-step MPC protocol. To mitigate these drawbacks, in this section, we describe an efficient two-round protocol for sampling from a discrete Gaussian distribution over a desired coset of a hard lattice Λ given Shamir secret shares of a trapdoor. We then apply this protocol to our BIBE scheme to obtain a threshold BIBE system.

6.1 Threshold Gaussian Preimage Sampling

Let $\tilde{m} \coloneqq n(\lfloor \log q \rfloor + 1)$ denote the number of columns of the gadget matrix $\mathbf{G}_n$. Let $\Lambda^{\mathbf{c}}(\mathbf{C}) \coloneqq \{\mathbf{v} \in \mathbb{Z}^m : \mathbf{C}\mathbf{v} = \mathbf{c} \bmod q\}$ denote the desired lattice coset with trapdoor $\mathbf{T}_{\mathbf{C}} \in \mathbb{Z}_q^{m \times \tilde{m}}$. Let σ_{flood} denote a *flooding* parameter, and define $B_{\mathsf{td}} \coloneqq m^{3/2}\lceil \log q \rceil$. There are N parties in total, and the threshold is denoted by τ. Each protocol execution is assigned a unique public session identifier $\mathsf{sid} \in \mathbb{Z}$ and is associated with an active set of parties $\mathsf{act} \subseteq [N]$, $|\mathsf{act}| = \tau$. Let $\mathsf{PRF} : \{0,1\}^\kappa \times \mathbb{Z} \to \mathbb{Z}_q^m$ be a pseudorandom function.

TGS.Setup$(1^\lambda, 1^N, 1^\tau, \mathbf{T}_{\mathbf{C}})$**.** Setup consists of two sub-algorithms:

1. *Shamir secret sharing.* The algorithm assigns share $\mathbf{T}_{\mathbf{C},i} \in \mathbb{Z}_q^{m \times \tilde{m}}$ of the trapdoor $\mathbf{T}_{\mathbf{C}}$ to party $i \in [N]$, where each element of $\mathbf{T}_{\mathbf{C},i}$ is a τ-of-N Shamir secret share of the corresponding element in $\mathbf{T}_{\mathbf{C}}$. Hence, for any set $\mathsf{act} \subseteq [N]$, $|\mathsf{act}| = \tau$, it holds that $\sum_{i \in \mathsf{act}} \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} = \mathbf{T}_{\mathbf{C}} \bmod q$, where $\lambda_{i,\mathsf{act}} \coloneqq \prod_{j \in \mathsf{act}, j \neq i} j/(j-i) \in \mathbb{Z}_q$. Since $\|\mathbf{T}_{\mathbf{C}}\|_\infty \leq 1$ and $q > 2$, the centered representative of each entry mod q is the integer entry itself, so we identify $\mathbf{T}_{\mathbf{C}}$ with its image in $\mathbb{Z}_q^{m \times \tilde{m}}$ throughout.
2. *Pairwise seeds.* For every ordered pair $(i,j) \in [N]^2$ with $i \neq j$, the algorithm samples an independent seed $\mathsf{seed}_{i,j} \leftarrow \{0,1\}^\kappa$. Party i receives $(\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N],\, j \neq i}$.

For each party i, setup outputs $\mathsf{sk}_i \coloneqq (\mathbf{T}_{\mathbf{C},i}, (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N],\, j \neq i})$.

TGS.RoundOne$(\mathsf{sk}_i, \mathsf{sid}, \mathsf{act}, \mathbf{C})$**.** Each party $i \in \mathsf{act}$ samples $\mathbf{p}_i \leftarrow \mathcal{D}_{\mathbb{Z}^m, \sigma_{\mathsf{flood}}}$, computes $\mathbf{e}_i \coloneqq \mathbf{C}\mathbf{p}_i \bmod q$ and $\mathbf{m}_i \coloneqq \sum_{j \in \mathsf{act},\, j \neq i} \mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid}) \in \mathbb{Z}_q^m$, and publishes $(\mathbf{e}_i, \mathbf{m}_i)$. The value $\mathbf{p}_i$ is kept as local state for the second round.

TGS.RoundTwo$(\mathsf{sk}_i, \mathbf{p}_i, \mathsf{sid}, \mathsf{act}, \mathbf{c}, (\mathbf{e}_j, \mathbf{m}_j)_{j \in \mathsf{act}})$**.** After collecting the accepted first-round transcript, every party computes $\mathbf{c}' \coloneqq \mathbf{c} - \sum_{j \in \mathsf{act}} \mathbf{e}_j \in \mathbb{Z}_q^n$ and $\mathbf{y}' \coloneqq \mathbf{G}^{-1}(\mathbf{c}')$. Then, each party $i \in \mathsf{act}$ derives $\mathbf{m}_i^\star \coloneqq \sum_{j \in \mathsf{act},\, j \neq i} \mathsf{PRF}(\mathsf{seed}_{j,i}, \mathsf{sid}) \in \mathbb{Z}_q^m$ and publishes

$$\mathbf{sbk}_i \coloneqq \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} \mathbf{y}' + \mathbf{p}_i + \mathbf{m}_i^\star \in \mathbb{Z}_q^m.$$

TGS.Comb$(\mathsf{act}, \mathbf{C}, \mathbf{c}, (\mathbf{m}_i, \mathbf{sbk}_i)_{i \in \mathsf{act}})$**.** The combiner algorithm computes $\mathbf{sbk} \coloneqq \sum_{i \in \mathsf{act}} (\mathbf{sbk}_i - \mathbf{m}_i) \in \mathbb{Z}_q^m$. If the check $\mathbf{C} \cdot \mathbf{sbk} = \mathbf{c} \bmod q$ succeeds, it outputs the combined Gaussian key $\mathbf{sbk}$. Otherwise, it outputs $\perp$.

6.2 TBIBE Protocol

In the TBIBE protocol, we domain-separate the hash function H_{sp} used to obtain the random coins for $\mathsf{SampleLeft}$ by the first-round syndrome transcript. The pre-decryption algorithm first runs TGS.RoundOne$(\mathsf{sk}_i, \mathsf{sid}, \mathsf{act}, \mathbf{C})$ to obtain the syndromes $(\mathbf{e}_i)_{i \in \mathsf{act}}$. After these values are fixed, the parties compute $\mathsf{sid}_{\mathsf{sp}}$ and query $H_{\mathsf{sp}}^{\mathsf{th}}$ on $\mathsf{sid}_{\mathsf{sp}}$.

Definition 16 (The hash function H_{tr}). *The hash function*

$$H_{\mathsf{tr}} : \mathbb{Z} \times 2^{[N]} \times (\mathbb{Z}_q^n)^\tau \to \{0,1\}^{\lambda_{\mathsf{tr}}}$$

takes a session id $\mathsf{sid} \in \mathbb{Z}$*, an active set* $\mathsf{act} \subseteq [N]$*, and the syndromes* $(\mathbf{e}_i)_{i \in \mathsf{act}} \in (\mathbb{Z}_q^n)^\tau$ *as input, and outputs a transcript tag* $\mathsf{sid}_{\mathsf{sp}} \in \{0,1\}^{\lambda_{\mathsf{tr}}}$*.*

Definition 17 (The hash function $H^{\mathsf{th}}_{\mathsf{sp}}$). *Let $\rho_{\mathsf{SL}} := \rho(\lambda, \ell n, \ell t + \tilde{m}, \ell\overline{m}, q)$ denote the randomness length of the explainable* SampleLeft *sampler from Theorem 1 when applied to matrices $\mathbf{A} \in \mathbb{Z}_q^{\ell n \times (\ell t + \tilde{m})}$ and $\hat{\mathbf{A}} \in \mathbb{Z}_q^{\ell n \times \ell\overline{m}}$. The hash function*

$$H^{\mathsf{th}}_{\mathsf{sp}} : \mathbb{Z}_q^{\ell n \times (\ell t + \tilde{m})} \times \mathbb{Z}_q^{(\ell t + \tilde{m}) \times \ell \tilde{m}} \times \mathcal{I}^{\ell} \times \mathbb{Z}_q^{n} \times \mathbb{Z}^{+} \times \{0,1\}^{\lambda_{\mathsf{tr}}} \to \{0,1\}^{\rho_{\mathsf{SL}}}$$

has the same domain and range as H_{sp} with the additional input of a transcript tag $\mathsf{sid}_{\mathsf{sp}} \in \{0,1\}^{\lambda_{\mathsf{tr}}}$, where $\lambda_{\mathsf{tr}} = \lambda$.

TBIBE.Setup$(1^{\lambda}, 1^{\ell}, 1^{N}, 1^{\tau})$**.** Setup algorithm runs BIBE.$\textsc{Setup}(1^{\lambda}, 1^{\ell})$ to obtain $(\mathsf{crs}_{\mathsf{samp}}, \mathbf{C}, \mathbf{u}, \mathbf{T}_{\mathbf{C}})$. It then invokes TGS.$\textsc{Setup}(1^{\lambda}, 1^{N}, 1^{\tau}, \mathbf{T}_{\mathbf{C}})$ to obtain sk_i, $i \in [N]$. It returns $\mathsf{pk} := (\mathsf{crs}_{\mathsf{samp}}, \mathbf{C}, \mathbf{u})$ as the public parameters and sk_i as party i's secret key.

TBIBE.PredecOne$(\mathsf{pk}, \mathsf{sk}_i, \mathsf{sid}, \mathsf{act}, (r_j)_{j \in [\ell]})$**.** Parsing $\mathsf{pk} = (\mathsf{crs}_{\mathsf{samp}}, \mathbf{C}, \mathbf{u})$, the algorithm invokes TGS.$\textsc{RoundOne}(\mathsf{sk}_i, \mathsf{sid}, \mathsf{act}, \mathbf{C})$ to obtain $\mathbf{p}_i$, keeps it as internal state, and returns $(\mathbf{e}_i, \mathbf{m}_i)$.

TBIBE.PredecTwo$(\mathsf{pk}, \mathsf{sk}_i, \mathsf{sid}, \mathsf{act}, (r_j)_{j \in [\ell]}, (\mathbf{e}_j, \mathbf{m}_j)_{j \in \mathsf{act}})$**.** The algorithm first computes $\mathsf{sid}_{\mathsf{sp}} := H_{\mathsf{tr}}(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i \in \mathsf{act}})$ and $\tilde{h}_i := H(r_i)$ for every $i \in [\ell]$, aborting if $\tilde{h}_1, \ldots, \tilde{h}_\ell$ are not distinct. Using $(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td} = (\mathbf{A}, \mathbf{T}_{\mathbf{A}}))$ from Expand, it constructs $\hat{\mathbf{A}}$ as in (4). It sets $\mathbf{h} := (\mathbf{u}, \ldots, \mathbf{u}) \in \mathbb{Z}_q^{\ell n}$, computes $\mathsf{rb} := H^{\mathsf{th}}_{\mathsf{sp}}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i \in [\ell]}, \mathbf{u}, \sigma_1, \mathsf{sid}_{\mathsf{sp}})$, and runs SampleLeft on $(\mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \sigma_1; \mathsf{rb})$ to get $\mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$. Finally, it sets $\mathbf{c} := \mathbf{G}\hat{\mathbf{c}}$, invokes

$$\mathbf{sbk}_i := \text{TGS.}\textsc{RoundTwo}(\mathsf{sk}_i, \mathbf{p}_i, \mathsf{sid}, \mathsf{act}, \mathbf{c}, (\mathbf{e}_j, \mathbf{m}_j)_{j \in \mathsf{act}}),$$

and outputs $(\mathbf{m}_i, \mathbf{sbk}_i, \mathsf{sid}_{\mathsf{sp}}, \mathbf{c})$.

TBIBE.Comb$(\mathsf{pk}, \mathsf{sid}, \mathsf{act}, (\mathbf{m}_i, \mathbf{sbk}_i, \mathsf{sid}_{\mathsf{sp}}, \mathbf{c})_{i \in \mathsf{act}})$**.** The algorithm checks that all submitted values $\mathsf{sid}_{\mathsf{sp}}$ and $\mathbf{c}$ are equal. If either check fails, it outputs $\perp$. Parsing pk, it then computes $\mathbf{sbk} := \text{TGS.}\textsc{Comb}(\mathsf{act}, \mathbf{C}, \mathbf{c}, (\mathbf{m}_i, \mathbf{sbk}_i)_{i \in \mathsf{act}})$. If $\mathbf{sbk} = \perp$, it outputs $\perp$. Otherwise, it outputs the combined threshold pre-decryption key $\mathsf{pdk} := (\mathsf{act}, \mathsf{sid}_{\mathsf{sp}}, \mathbf{sbk})$.

TBIBE.Dec$(\mathsf{pk}, \mathsf{pdk}, (\mathsf{ct}_i)_{i \in [\ell]})$**.** The algorithm parses $\mathsf{pdk} = (\mathsf{act}, \mathsf{sid}_{\mathsf{sp}}, \mathbf{sbk})$ and proceeds with the usual decryption computation, except that it outputs $\perp$ if $\mathbf{C} \cdot \mathbf{sbk} \neq \mathbf{c} \bmod q$ and queries $H^{\mathsf{th}}_{\mathsf{sp}}$ with $\mathsf{sid}_{\mathsf{sp}}$.

6.3 Correctness, Parameter Sizes, and Selective Security

Correctness and security theorems for our TBIBE system are presented below.

Theorem 4 (Threshold Correctness). *The* TBIBE *scheme is correct whenever the conditions of Theorem 2 hold, and for $\mu = \max(t, \overline{m})$,*

$$\alpha < ((\mu\sigma_1 + B_{\mathsf{th}}) \cdot \omega(\log \mu))^{-1} \qquad q = \Omega\left(\sigma_1 \mu^{3/2} + B_{\mathsf{th}}\sqrt{\mu}\right),$$

where

$$B_{\mathsf{th}} := \sqrt{m}\tilde{m}\log(q) + \sqrt{\tau m}\,\sigma_{\mathsf{flood}} \cdot \omega(\sqrt{\log \lambda}).$$

Proof. By construction, all honest parties and the combiner algorithm compute the same transcript tag $\mathsf{sid}_{\mathsf{sp}}$ from the syndromes $\mathbf{e}$. Thus, they use the same $H^{\mathsf{th}}_{\mathsf{sp}}$ value, the same $\mathsf{SampleLeft}$ output, and the same target $\mathbf{c} = \mathbf{G}\hat{\mathbf{c}}$. By mask cancellation, $\sum_{i\in\mathsf{act}} \mathbf{m}_i = \sum_{i\in\mathsf{act}} \mathbf{m}^\star_i$, and by Shamir reconstruction, $\sum_{i\in\mathsf{act}} \lambda_{i,\mathsf{act}}\mathbf{T}_{\mathbf{C},i} = \mathbf{T}_{\mathbf{C}} \bmod q$. Therefore, $\mathbf{sbk} = \sum_{i\in\mathsf{act}}(\mathbf{sbk}_i - \mathbf{m}_i) = \mathbf{T}_{\mathbf{C}}\mathbf{y}' + \sum_{i\in\mathsf{act}} \mathbf{p}_i$. Moreover, $\mathbf{C}\cdot\mathbf{sbk} = \mathbf{C}\mathbf{T}_{\mathbf{C}}\mathbf{y}' + \sum_{i\in\mathsf{act}} \mathbf{C}\mathbf{p}_i = \mathbf{G}\mathbf{y}' + \sum_{i\in\mathsf{act}} \mathbf{e}_i = (\mathbf{c} - \sum_{i\in\mathsf{act}} \mathbf{e}_i) + \sum_{i\in\mathsf{act}} \mathbf{e}_i = \mathbf{c} \bmod q$. Hence, TBIBE.Comb outputs $\mathsf{pdk} = (\mathsf{act}, \mathsf{sid}_{\mathsf{sp}}, \mathbf{sbk})$, and the validity check in TBIBE.Dec succeeds.

Next, for each $i \in [\ell]$, the public $\mathsf{SampleLeft}$ computation gives $\mathbf{F}_{r_i}\mathbf{v}_i = \mathbf{u} - \mathbf{c}$, where $\mathbf{F}_{r_i} \coloneqq [\mathbf{A}_{r_i} \mid H_{\mathsf{ro}}(r_i)]$. Then, $\mathbf{C}\cdot\mathbf{sbk} + \mathbf{F}_{r_i}\mathbf{v}_i = \mathbf{u}$, which is the same algebraic relation used in the non-threshold correctness proof. Hence, to conclude the proof, it remains to bound the norm of $\mathbf{sbk}$. Since $\|\mathbf{T}_{\mathbf{C}}\|_\infty \le 1$ and $\|\mathbf{y}'\|_\infty \le \lceil \log q \rceil$, we have $\|\mathbf{T}_{\mathbf{C}}\mathbf{y}'\|_2 \le \sqrt{m}\tilde{m}\log(q)$. Moreover, by the standard tail bound for sums of independent discrete Gaussians, $\|\sum_{i\in\mathsf{act}} \mathbf{p}_i\|_2 \le \sqrt{\tau m}\,\sigma_{\mathsf{flood}}\cdot\omega(\sqrt{\log\lambda})$ except with negligible probability. Thus $\|\mathbf{sbk}\|_2 \le B_{\mathsf{th}}$ except with negligible probability. The decryption error analysis is now identical to the proof of Theorem 2, except that the term $\sqrt{m}\,\sigma_2$ is replaced by B_{th}. Therefore the error is bounded by

$$O\left((\mu\sigma_1 + B_{\mathsf{th}})q\alpha\cdot\omega(\log\mu)\right) + O\left(\sigma_1\mu^{3/2} + B_{\mathsf{th}}\sqrt{\mu}\right),$$

which is less than $q/5$ under the stated conditions on α and q. Hence decryption succeeds except with negligible probability. □

Conceptually, in Section 5, $\|\mathbf{sbk}\| \le \sqrt{m}\,\sigma_2$, whereas in this section, this condition becomes $\|\mathbf{sbk}\| \le B_{\mathsf{th}}$ due to noise smudging, which in turn requires the new values for α and q. Thus, the public key and ciphertext sizes are unchanged from the non-threshold scheme, except through the possibly larger modulus q required by Theorem 4. Thus,

$$|\mathsf{pk}| = n(2t+m)\log q \qquad \text{and} \qquad |\mathsf{ct}| = (1 + m + t + \overline{m})\log q.$$

The combined threshold pre-decryption key is $\mathsf{pdk} = (\mathsf{act}, \mathsf{sid}_{\mathsf{sp}}, \mathbf{sbk})$, where $\|\mathbf{sbk}\|_2 \le B_{\mathsf{th}}$. Hence,

$$|\mathsf{pdk}| = O(m\log B_{\mathsf{th}} + \tau\log N + \lambda_{\mathsf{tr}}).$$

Each threshold pre-decryption execution also communicates first-round values $(\mathbf{e}_i, \mathbf{m}_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q^m$ and second-round values $\mathbf{sbk}_i \in \mathbb{Z}_q^m$ for every $i \in \mathsf{act}$, for a total communication of $O(\tau(n + 2m)\log q)$ bits, up to the lower-order cost of party indices and transcript tags. Note that due to the exponential dependence of B_{th} on λ (Remark 3), all parameter sizes increase by a factor of λ.

Theorem 5 (Threshold Selective Security). *Suppose the assumptions of Theorem 3 hold, $\sigma_{\mathsf{flood}} > \eta_\varepsilon(\Lambda^\perp(\mathbf{C}))$, for negligible ε satisfying $q_{\mathsf{pd}}\varepsilon = \mathsf{negl}(\lambda)$, PRF is a secure PRF, and*

$$\alpha < \left((\mu\sigma_1 + B_{\mathsf{th}})\cdot\omega(\log\mu)\right)^{-1} \qquad q = \Omega\left(\sigma_1\mu^{3/2} + B_{\mathsf{th}}\sqrt{\mu}\right),$$

Let $\mathcal{A}$ be a PPT adversary for the TBIBE *security game (Definition 6), and $\mathcal{B}_1$ and $\mathcal{B}_2$ be PPT adversaries for the PRF and LWE security games (Definitions 10 and 21). Then, for every fixed $a \in (1,\infty)$,*

$$\mathsf{Adv}^{\mathrm{TBIBE}}_{\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^\tau) \le O\left(\frac{B_{\mathsf{td}}\sqrt{q_{\mathsf{pd}}}}{\sigma_{\mathsf{flood}}}\right) + 2\mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(1^\lambda) + 2\mathsf{Adv}^{\mathsf{LWE}}_{\mathcal{B}_2}(1^\lambda) + \mathsf{negl}(\lambda)$$

whenever H_{tr}, H_{ro}, and $H^{\mathsf{th}}_{\mathsf{sp}}$ are modeled as random oracles.

Remark 3. Setting σ_{flood} to be $2^{\Omega(\lambda)}$ implies a negligible advantage for $\mathcal{A}$ whenever PRF is a secure PRF and the LWE assumption holds. The probability-preservation property of Rényi divergence (Lemma 27) would allow one to preserve a single success event under the smudging hybrid with only polynomial flooding. This is sufficient for unforgeability-style games as in the case of GPV signatures (Section 7), but not for the indistinguishability game considered for TBIBE. Accordingly, the proof uses the additive Pinsker inequality (Lemma 27), which yields the loss $O(B_{\mathsf{td}}\sqrt{q_{\mathsf{pd}}}/\sigma_{\mathsf{flood}})$, and requires $\sigma_{\mathsf{flood}} = 2^{\Omega(\lambda)}$. This results in a factor of λ increase in the parameter sizes of the threshold scheme compared to the non-threshold scheme.

The proof relies on the following lemmas.

Lemma 11. *Let $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$ and $a \in (1,\infty)$. Let $\varepsilon \in (0,1)$, and consider $s > \eta_\varepsilon(\Lambda^\perp(\mathbf{A}))$. Let W be auxiliary information such that, conditioned on W, the random variable $\mathbf{p}$ is distributed as $\mathcal{D}_{\Lambda^{\mathbf{e}}(\mathbf{A}),s}$ for some value $\mathbf{e} \in \mathbb{Z}_q^n$. For any given value of W, let $\mathbf{x} = f(\mathsf{W}) \in \mathbb{Z}^m$ be a vector satisfying $\|\mathbf{x}\| \le B$. Define the conditional distributions $P_{\mathsf{real}} \mid \mathsf{W} := \mathbf{x} + \mathbf{p}$ and $P_{\mathsf{sim}} \mid \mathsf{W} := \mathcal{D}_{\Lambda^{\mathbf{e}+\mathbf{A}\mathbf{x}}(\mathbf{A}),s,\mathbf{0}}$. Then, for every value of W,*

$$R_a(P_{\mathsf{real}} \mid \mathsf{W} \| P_{\mathsf{sim}} \mid \mathsf{W}) \le \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{2/(a-1)} \exp\left(a\pi B^2/s^2\right).$$

The same bound remains true after applying any deterministic or randomized post-processing to both distributions.

Proof. Conditioned on any value of W, $\mathbf{e}$ and $\mathbf{x}$ are fixed, and $\mathbf{p}$ has distribution $\mathcal{D}_{\Lambda^{\mathbf{e}}(\mathbf{A}),s}$. Therefore $\mathbf{x}+\mathbf{p}$ has distribution $\mathcal{D}_{\Lambda^{\mathbf{e}+\mathbf{A}\mathbf{x}}(\mathbf{A}),s,\mathbf{x}}$. The simulated variable has distribution $\mathcal{D}_{\Lambda^{\mathbf{e}+\mathbf{A}\mathbf{x}}(\mathbf{A}),s,\mathbf{0}}$. The Rényi bound follows from Lemma 28. The final statement follows from the data processing inequality in Lemma 27. □

Lemma 12. *Let $\mathcal{H} \subseteq [N]$ be a non-empty set of indices. For every ordered pair $(i,j) \in \mathcal{H}^2$ with $i \ne j$, let $\mathbf{r}_{i,j} \leftarrow \mathbb{Z}_q^m$ be independent and uniform. Let $\mathbf{a}_i, \mathbf{b}_i \in \mathbb{Z}_q^m$ be arbitrary fixed vectors. Define $\mathbf{m}_i := \mathbf{b}_i + \sum_{j\in\mathcal{H}, j\ne i} \mathbf{r}_{i,j}$ and $\mathbf{sbk}_i := \mathbf{a}_i + \sum_{j\in\mathcal{H}, j\ne i} \mathbf{r}_{j,i}$. Fix any index $h \in \mathcal{H}$. Then, the joint distribution of $(\mathbf{m}_i, \mathbf{sbk}_i)_{i\in\mathcal{H}}$ is identical to the following distribution: sample $\mathbf{m}_i \leftarrow \mathbb{Z}_q^m$ independently for all $i \in \mathcal{H}$, sample $\mathbf{sbk}_i \leftarrow \mathbb{Z}_q^m$ independently for all $i \in \mathcal{H}\setminus\{h\}$, and set*

$$\mathbf{sbk}_h := \sum_{i\in\mathcal{H}} (\mathbf{m}_i + \mathbf{a}_i - \mathbf{b}_i) - \sum_{i\in\mathcal{H}\setminus\{h\}} \mathbf{sbk}_i.$$

Proof. It suffices to argue coordinate-wise over $\mathbb{Z}_q$. For a fixed coordinate, consider the linear map from $(r_{i,j})_{i,j\in\mathcal{H}, i\ne j}$ to the row and column sums $M_i := \sum_{j\in\mathcal{H}, j\ne i} r_{i,j}$ and $M_i^\star := \sum_{j\in\mathcal{H}, j\ne i} r_{j,i}$. Image of this map is exactly $\{(M_i, M_i^\star)_{i\in\mathcal{H}} : \sum_i M_i = \sum_i M_i^\star\}$. Indeed, the displayed equality is always satisfied, and it is the only linear relation among the row and column sums. Since the inputs are uniform, the image is uniform over this subspace. After conditioning on the row sums, the column sums are uniform subject only to having the same total sum. Adding the fixed offsets gives that $(\mathbf{sbk}_i)_{i\in\mathcal{H}}$ is uniform conditioned on $(\mathbf{m}_i)_{i\in\mathcal{H}}$, over the affine set determined by $\sum_{i\in\mathcal{H}}(\mathbf{sbk}_i - \mathbf{m}_i) = \sum_{i\in\mathcal{H}}(\mathbf{a}_i - \mathbf{b}_i)$. Sampling all $\mathbf{sbk}_i$ for $i \ne h$ uniformly and then setting $\mathbf{sbk}_h$ by the displayed equation is exactly the uniform distribution over this affine set. □

Proof of Theorem 5. Suppose there exists a PPT adversary $\mathcal{A}$ attacking the TBIBE system with non-negligible advantage ϵ using a polynomial number q_{pd} of threshold pre-decryption queries (including the aborted ones), q_{tr} queries to H_{tr}, and q_{sp} queries to $H^{\mathsf{th}}_{\mathsf{sp}}$. Let $\mathsf{corrupt} \subseteq [N]$ be the static corrupted set, with $|\mathsf{corrupt}| \leq \tau - 1$. Let $\mathsf{Hybrid}_i(\mathcal{A})$ denote the output distribution of hybrid i's transcript with adversary $\mathcal{A}$, and $\mathsf{Adv}_{\mathcal{A},i}$ denote $\mathcal{A}$'s advantage in hybrid i. We argue that the adversary's advantage mostly changes negligibly from hybrid to hybrid, and moreover an adversary with non-negligible advantage in Hybrid 11 can be used to break the LWE assumption.

Hybrid 0. This is the real game $\mathsf{Game}^{\mathrm{TBIBE}}_{\mathcal{A},b}(1^\lambda, 1^\ell, 1^N, 1^\tau)$ in the random oracle model. Thus, $\mathsf{Hybrid}_0(\mathcal{A}) = \mathsf{Adv}^{\mathrm{TBIBE}}_{\mathcal{A}}(1^\lambda, 1^\ell, 1^N, 1^\tau)$.

Hybrid 1. For every threshold pre-decryption query with active set act and session identifier sid, and for every ordered honest-honest pair $(i,j) \in (\mathsf{act} \setminus \mathsf{corrupt})^2$ with $i \neq j$, the challenger Ch replaces $\mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid})$ by an independent uniform vector in $\mathbb{Z}_q^m$, with consistent answers on repeated inputs. Values for pairs with at least one corrupted party are left unchanged. Now, consider the PPT PRF adversary $\mathcal{B}_1$ that acts as the challenger of Hybrid 0 by sampling $b \leftarrow \{0,1\}$, except that it answers the honest-honest PRF values using its own PRF oracle. It outputs 1 if $\mathcal{A}$ correctly guesses the bit b. Then, $|\mathsf{Adv}_{\mathcal{A},0} - \mathsf{Adv}_{\mathcal{A},1}| \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(1^\lambda) = \mathsf{negl}(\lambda)$.

Hybrid 2. Let T_{tr} and T_{sp} denote the tables of H_{tr} and $H^{\mathsf{th}}_{\mathsf{sp}}$ queries, respectively. Whenever H_{tr} is queried on an input $x = (\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i \in \mathsf{act}})$, Ch returns the previously stored value if x is already in T_{tr}. Otherwise, Ch samples $y \leftarrow \{0,1\}^{\lambda_{\mathsf{tr}}}$, stores $\mathsf{T}_{\mathsf{tr}}[x] = y$, and returns y. If the sampled value y is already equal to $\mathsf{T}_{\mathsf{tr}}[x']$ for some $x' \neq x$, or was previously used in an $H^{\mathsf{th}}_{\mathsf{sp}}$ query before appearing in T_{tr}, then Ch aborts. We denote this event by $\mathsf{Bad}_{\mathsf{th}}$. Otherwise, Hybrid 2 is identical to Hybrid 1. Conditioned on $\neg\mathsf{Bad}_{\mathsf{th}}$, every transcript tag $\mathsf{sid}_{\mathsf{sp}}$ in the range of H_{tr} has a unique preimage in T_{tr}, and no $H^{\mathsf{th}}_{\mathsf{sp}}$ value was returned before the corresponding transcript tag was known.

Since H_{tr} is a random oracle with at most $q_{\mathsf{tr}} + q_{\mathsf{pd}}$ queries by either $\mathcal{A}$ or Ch, and at most q_{sp} $\mathsf{sid}_{\mathsf{sp}}$ tags can be guessed before $H^{\mathsf{th}}_{\mathsf{sp}}$ queries,

$$|\mathsf{Adv}_{\mathcal{A},1} - \mathsf{Adv}_{\mathcal{A},2}| \leq \Pr[\mathsf{Bad}_{\mathsf{th}}] = \frac{(q_{\mathsf{tr}} + q_{\mathsf{pd}})^2 + q_{\mathsf{sp}}(q_{\mathsf{tr}} + q_{\mathsf{pd}})}{2^{\lambda_{\mathsf{tr}}}} = \mathsf{negl}(\lambda)$$

Hybrid 3. Fix a threshold pre-decryption query with active set act and session identifier sid. Let $\mathcal{C} := \mathsf{act} \cap \mathsf{corrupt}$ and $\mathcal{H} := \mathsf{act} \setminus \mathcal{C}$. Since $|\mathsf{act}| = \tau$ and $|\mathsf{corrupt}| \leq \tau - 1$, we have $|\mathcal{H}| \geq 1$. Fix a canonical last honest party $h \in \mathcal{H}$, and write $\mathcal{H}^- := \mathcal{H} \setminus \{h\}$. The first-round values $(\mathbf{e}_j, \mathbf{m}_j)_{j \in \mathcal{C}}$ are given by $\mathcal{A}$ and treated as fixed side information. For every $i \in \mathcal{H}$, Ch samples $\mathbf{p}_i \leftarrow \mathcal{D}_{\mathbb{Z}^m, \sigma_{\mathsf{flood}}}$ and sets $\mathbf{e}_i := \mathbf{C}\mathbf{p}_i \bmod q$. After the first-round transcript is fixed, Ch computes $\mathsf{sid}_{\mathsf{sp}} := H_{\mathsf{tr}}(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i \in \mathsf{act}})$, obtains $\mathbf{c}$ from the corresponding $H^{\mathsf{th}}_{\mathsf{sp}}$ value, sets $\mathbf{c}' := \mathbf{c} - \sum_{i \in \mathsf{act}} \mathbf{e}_i$ and $\mathbf{y}' := \mathbf{G}^{-1}(\mathbf{c}')$. For indices $i \in \mathcal{H}$, define $\mathbf{a}_i := \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} \mathbf{y}' + \mathbf{p}_i + \sum_{j \in \mathcal{C}} \mathsf{PRF}(\mathsf{seed}_{j,i}, \mathsf{sid})$, and $\mathbf{b}_i := \sum_{j \in \mathcal{C}} \mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid})$. Hybrid 3 samples $\mathbf{m}_i \leftarrow \mathbb{Z}_q^m$ independently for all $i \in \mathcal{H}$, samples $\mathbf{sbk}_i \leftarrow \mathbb{Z}_q^m$ independently for all $i \in \mathcal{H}^-$, and sets

$$\mathbf{sbk}_h := \sum_{i \in \mathcal{H}} (\mathbf{m}_i + \mathbf{a}_i - \mathbf{b}_i) - \sum_{i \in \mathcal{H} \setminus \{h\}} \mathbf{sbk}_i \tag{5}$$

$$= \mathbf{T}_{\mathbf{C}} \mathbf{y}' + \sum_{i \in \mathcal{H}} \mathbf{p}_i - \sum_{i \in \mathcal{C}} \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} \mathbf{y}' - \sum_{i \in \mathcal{H} \setminus \{h\}} \mathbf{sbk}_i + \sum_{i \in \mathcal{H}} \mathbf{m}_i + \Delta_{\mathcal{C}} \tag{6}$$

where $\Delta_{\mathcal{C}} := \sum_{i\in\mathcal{H}}(\sum_{j\in\mathcal{C}} \mathsf{PRF}(\mathsf{seed}_{j,i}, \mathsf{sid}) - \sum_{j\in\mathcal{C}} \mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid}))$, and the equality follows from $\sum_{i\in\mathsf{act}} \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i} = \mathbf{T}_{\mathbf{C}}$. Conditioned on the fixed side information, before revealing the row masks, the honest-honest masks are independent and uniform. Therefore, by Lemma 12, $\mathsf{Adv}_{\mathcal{A},2} = \mathsf{Adv}_{\mathcal{A},3}$.

Hybrid 4. Hybrid 4 is identical to Hybrid 3 except that, in each threshold pre-decryption query, the term $\mathbf{T}_{\mathbf{C}}\mathbf{y}' + \sum_{i\in\mathcal{H}} \mathbf{p}_i$ in equation (6) is replaced by a centered Gaussian over the same coset. Fix one query, and a canonical honest party $h \in \mathcal{H}$, and write $\mathcal{H}^- := \mathcal{H} \setminus \{h\}$. Define $\mathbf{d}_h := \mathbf{c} - \sum_{i\in\mathsf{act}\setminus\{h\}} \mathbf{e}_i$. In Hybrid 3, let $\mathbf{z}_h^{\mathsf{real}} := \mathbf{T}_{\mathbf{C}}\mathbf{y}' + \mathbf{p}_h$. Since $\mathbf{C}\mathbf{T}_{\mathbf{C}}\mathbf{y}' = \mathbf{G}\mathbf{y}' = \mathbf{c}' = \mathbf{c} - \sum_{i\in\mathsf{act}} \mathbf{e}_i$, we have $\mathbf{C}\mathbf{z}_h^{\mathsf{real}} = \mathbf{d}_h \bmod q$, *i.e.*, $\mathbf{z}_h^{\mathsf{real}} \in \Lambda^{\mathbf{d}_h}(\mathbf{C})$. In Hybrid 4, Ch samples $\mathbf{z}_h^{\mathsf{sim}} := \mathbf{sbk}' \leftarrow \mathcal{D}_{\Lambda^{\mathbf{d}_h}(\mathbf{C}),\sigma_{\mathsf{flood}},\mathbf{0}}$ and sets

$$\mathbf{sbk}_h := \mathbf{sbk}' - \sum_{i\in\mathcal{C}} \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i}\mathbf{y}' - \sum_{i\in\mathcal{H}\setminus\{h\}} \mathbf{sbk}_i + \sum_{i\in\mathcal{H}} \mathbf{m}_i + \Delta_{\mathcal{C}}$$

The first-round value $\mathbf{e}_h$ is still sampled honestly as $\mathbf{e}_h = \mathbf{C}\mathbf{p}_h \bmod q$. Note that the virtual value of $\mathbf{p}$, *i.e.*, $\mathbf{p}_h^{\mathsf{virt}} := \mathbf{sbk}' - \mathbf{T}_{\mathbf{C}}\mathbf{y}'$ still satisfies $\mathbf{C}\mathbf{p}_h^{\mathsf{virt}} = \mathbf{d}_h - (\mathbf{c} - \sum_{i\in\mathsf{act}} \mathbf{e}_i) = \mathbf{e}_h \bmod q$.

We next bound $R_a(\mathsf{Hybrid}_3(\mathcal{A})\|\mathsf{Hybrid}_4(\mathcal{A}))$. Consider the k-th threshold pre-decryption query, and condition on the secret keys and seeds of the corrupt parties, trapdoor $\mathbf{T}_{\mathbf{C}}$, transcripts of the past queries, and within the k-th query, the values $(\mathbf{e}_i, \mathbf{m}_i)_{i\in\mathsf{act}}$, $(\mathbf{p}_i)_{i\in\mathcal{H}^-}$, and $(\mathbf{sbk}_i)_{i\in\mathcal{H}^-}$. Under this conditioning, $\mathbf{y}'$ and $\mathbf{x} := \mathbf{T}_{\mathbf{C}}\mathbf{y}'$ are fixed, $\|\mathbf{x}\|_2 \le B_{\mathsf{td}}$, and $\mathbf{p}_h$ is distributed as $\mathcal{D}_{\Lambda^{\mathbf{e}_h}(\mathbf{C}),\sigma_{\mathsf{flood}}}$. Therefore, $\mathbf{z}_h^{\mathsf{real}}$ has distribution $\mathcal{D}_{\Lambda^{\mathbf{d}_h}(\mathbf{C}),\sigma_{\mathsf{flood}},\mathbf{T}_{\mathbf{C}}\mathbf{y}'}$, while $\mathbf{z}_h^{\mathsf{sim}}$ has $\mathcal{D}_{\Lambda^{\mathbf{d}_h}(\mathbf{C}),\sigma_{\mathsf{flood}},\mathbf{0}}$. By Lemma 11, for every $a \in (1,\infty)$, the conditional Rényi divergence for the k-th query between $\mathbf{z}_h^{\mathsf{real}}$ and $\mathbf{z}_h^{\mathsf{sim}}$ is at most

$$\left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{2/(a-1)} \exp\left(a\pi B_{\mathsf{td}}^2/\sigma_{\mathsf{flood}}^2\right).$$

Moreover, the rest of the transcript for the k-th query is obtained by post-processing $\mathbf{z}_h^{\mathsf{real}}$ or $\mathbf{z}_h^{\mathsf{sim}}$, so the data processing inequality in Lemma 27 preserves the bound. Applying the adaptive product rule in Lemma 27 over the q_{pd} threshold pre-decryption queries gives

$$R_a(\mathsf{Hybrid}_3(\mathcal{A})\|\mathsf{Hybrid}_4(\mathcal{A})) \le \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{2q_{\mathsf{pd}}/(a-1)} \exp\left(a\pi q_{\mathsf{pd}} B_{\mathsf{td}}^2/\sigma_{\mathsf{flood}}^2\right).$$

By Pinsker's inequality in Lemma 27, the statistical distance between $\mathsf{Hybrid}_3(\mathcal{A})$ and $\mathsf{Hybrid}_4(\mathcal{A})$ is bounded by

$$\sqrt{\frac{q_{\mathsf{pd}}}{a-1}\ln\left(\frac{1+\varepsilon}{1-\varepsilon}\right) + \frac{a\pi q_{\mathsf{pd}} B_{\mathsf{td}}^2}{2\sigma_{\mathsf{flood}}^2}}.$$

For fixed $a > 1$ and negligible ε with $q_{\mathsf{pd}}\varepsilon = \mathsf{negl}(\lambda)$, this implies

$$|\mathsf{Adv}_{\mathcal{A},3} - \mathsf{Adv}_{\mathcal{A},4}| \le O\left(\frac{B_{\mathsf{td}}\sqrt{q_{\mathsf{pd}}}}{\sigma_{\mathsf{flood}}}\right) + \mathsf{negl}(\lambda).$$

Remaining hybrids. After Hybrid 4, simulation no longer uses the honest Shamir secret shares of $\mathbf{T}_{\mathbf{C}}$. The remaining hybrids follow the proof of Theorem 3, with the following changes. In all hybrids, $H_{\mathsf{sp}}^{\mathsf{th}}$ now has the additional input $\mathsf{sid}_{\mathsf{sp}}$.

Hybrids 5, 6, 7. The changes are the same as in Hybrids 1, 2, and 3 in Section 5.2, and $|\mathsf{Adv}_{\mathcal{A},5} - \mathsf{Adv}_{\mathcal{A},7}| = \mathsf{negl}(\lambda)$.

Hybrid 8. The changes are the same as in Hybrid 4, except that after $\mathbf{C}$ is replaced by a uniform matrix in Hybrid 4 , the Shamir secret shares of the corrupt parties are sampled uniformly in $\mathbb{Z}_q^{m\times\tilde{m}}$. Then, $|\mathsf{Adv}_{\mathcal{A},7} - \mathsf{Adv}_{\mathcal{A},8}| = \mathsf{negl}(\lambda)$ by Lemma 6 and the perfect privacy of Shamir secret sharing, since $|\mathsf{corrupt}| \leq \tau - 1$.

Hybrid 9. The changes are the same as in Hybrid 5 in Section 5.2. Then, the statistical distance between $\mathsf{Hybrid}_{\mathcal{A},8}$ and $\mathsf{Hybrid}_{\mathcal{A},9}$ is bounded by $\epsilon/2 + \mathsf{negl}(\lambda)$.

Hybrid 10. Hybrid 6 in Section 5.2 is modified as follows. Upon a new query $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1, \mathsf{sid}_{\mathsf{sp}})$ for the random oracle $H^{\mathsf{th}}_{\mathsf{sp}}$, Ch checks if $r^* \in (r_i)_{i\in[\ell]}$ and if there is an entry x such that $T_{\mathsf{tr}}[x] = \mathsf{sid}_{\mathsf{sp}}$. It then does the following:

1. $r^* \in (r_i)_{i\in[\ell]}$: The simulation is identical to Case 1 of Hybrid 6.
2. $r^* \notin (r_i)_{i\in[\ell]}$ and there is no entry x such that $T_{\mathsf{tr}}[x] = \mathsf{sid}_{\mathsf{sp}}$: Ch proceeds as in the previous hybrids.
3. $r^* \notin (r_i)_{i\in[\ell]}$ and there is an entry such that $T_{\mathsf{tr}}[\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i\in\mathsf{act}}] = \mathsf{sid}_{\mathsf{sp}}$:
   - Ch samples $\mathbf{sbk}' \leftarrow \mathcal{D}_{\mathbb{Z}^m,\sigma_{\mathsf{flood}}}$, sets $\mathbf{c} \coloneqq \mathbf{C}\cdot\mathbf{sbk}' + \sum_{i\in\mathsf{act}\setminus\{h\}} \mathbf{e}_i$, and samples $\hat{\mathbf{c}} \leftarrow \mathbf{G}_{\sigma_1}^{-1}(\mathbf{c})$ using the algorithm in Lemma 21.
   - For all $i \in [\ell]$, Ch obtains $\mathbf{v}_i = \mathrm{col}(\mathbf{v}_i^1, \mathbf{v}_i^2)$ by invoking and permuting the output of $\mathsf{SampleLeft}(H_{\mathsf{ro}}(r_i), \mathbf{A}_{r_i}, \mathbf{T}_{r_i}, \mathbf{u} - \mathbf{c}, \sigma_1)$.
4. Finally, setting $\mathbf{v} \coloneqq \mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2)$, Ch finds

$$\mathsf{rb} \coloneqq \mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \hat{\mathbf{A}}, \mathbf{T}_{\mathbf{A}}, \mathbf{h}, \mathbf{v}, \sigma_1),$$

stores $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1, \mathbf{v}, \mathbf{sbk}', \mathsf{sid}_{\mathsf{sp}}, \mathbf{c}; \mathsf{rb})$, and returns rb to $\mathcal{A}$.

Note that the pre- and post-challenge queries are still answered by sampling $\mathbf{sbk}' \leftarrow \mathcal{D}_{\Lambda^{\mathbf{d}_h}(\mathbf{C}),\sigma_{\mathsf{flood}},\mathbf{0}}$.

Conditioned on $\neg\mathsf{Bad}_{\mathsf{th}}$, $\mathsf{sid}_{\mathsf{sp}}$ was not used in a threshold pre-decryption response before. Since $\mathbf{C}$ is uniform, and $\sigma_{\mathsf{flood}} \geq \omega(\sqrt{\log m})$, the value $\mathbf{C}\mathbf{sbk}'$ is statistically close to a uniform sample from $\mathbb{Z}_q^n$ by Lemma 17. Then, $|\mathsf{Adv}_{\mathcal{A},9} - \mathsf{Adv}_{\mathcal{A},10}| = \mathsf{negl}(\lambda)$ by Lemma 8.

Hybrid 11. Given some pre- or post-challenge query $(r_i)_{i\in[\ell]}$, act and sid, Ch first checks if $r^* \in (r_i)_{i\in[\ell]}$. If so, it aborts the game. Otherwise, Ch constructs the matrix $\mathbf{A}$ as in equation (3). Without loss of generality, suppose $\mathcal{A}$ has already queried H_{tr} with the input $(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i\in\mathsf{act}})$ to obtain $\mathsf{sid}_{\mathsf{sp}}$, and $H^{\mathsf{th}}_{\mathsf{sp}}$ with the input $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1, \mathsf{sid}_{\mathsf{sp}})$. Then, Ch retrieves the tuple $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, (r_i)_{i\in[\ell]}, \mathbf{u}, \sigma_1, \mathbf{v}, \mathbf{sbk}', \mathsf{sid}_{\mathsf{sp}}, \mathbf{c}; \mathsf{rb})$, computes $\mathbf{c}' \coloneqq \mathbf{c} - \sum_{i\in\mathsf{act}} \mathbf{e}_i = \mathbf{C}\cdot\mathbf{sbk}' - \mathbf{e}_h$ and $\mathbf{y}' \coloneqq \mathbf{G}^{-1}(\mathbf{c}')$, samples $\mathbf{sbk}_i \leftarrow \mathbb{Z}_q^m$ for $i \in \mathcal{H}^-$, and sets

$$\mathbf{sbk}_h \coloneqq \mathbf{sbk}' + \sum_{i\in\mathcal{H}^-} \mathbf{p}_i - \sum_{i\in\mathcal{C}} \lambda_{i,\mathsf{act}} \mathbf{T}_{\mathbf{C},i}\mathbf{y}' - \sum_{i\in\mathcal{H}\setminus\{h\}} \mathbf{sbk}_i + \sum_{i\in\mathcal{H}} \mathbf{m}_i + \Delta_{\mathcal{C}}$$

Note that conditioned on $\mathbf{c} = \mathbf{C}\cdot\mathbf{sbk}' + \sum_{i\in\mathsf{act}\setminus\{h\}} \mathbf{e}_i$, the value $\mathbf{sbk}'$ is distributed as $\mathcal{D}_{\Lambda^{\mathbf{d}_h}(\mathbf{C}),\sigma_{\mathsf{flood}},\mathbf{0}}$. Hence, $\mathsf{Adv}_{\mathcal{A},10} = \mathsf{Adv}_{\mathcal{A},11}$.

Finally, the LWE reduction is identical to the final reduction in Theorem 3. Then, if $\mathcal{A}$ has advantage ϵ in the TBIBE security game (Definition 6), the LWE adversary $\mathcal{B}_2$ would have advantage at least

$$\mathsf{Adv}^{\mathsf{LWE}}_{\mathcal{B}_2} \geq \frac{\epsilon}{2} - O\left(\frac{B_{\mathsf{td}}\sqrt{q_{\mathsf{pd}}}}{\sigma_{\mathsf{flood}}}\right) - \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(1^\lambda) - \mathsf{negl}(\lambda),$$

in deciding the $(\mathbb{Z}_q, n, \chi)$-LWE problem, concluding the proof. □

7 Threshold GPV Signatures

In our threshold GPV signatures, the first-round transcript is used to evaluate the hash target. Let $H_{\mathsf{GPV}} : \mathcal{M} \times \{0,1\}^{\lambda_{\mathsf{tr}}} \to \mathbb{Z}_q^n$ denote the random oracle used by the signature scheme, and $\mathbf{C}$ denote the public matrix. After the first-round transcript $(\mathbf{e}_i, \mathbf{m}_i)_{i\in\mathsf{act}}$ is fixed, where $(\mathbf{e}_i, \mathbf{m}_i) \leftarrow \mathrm{TGS.RoundOne}(\mathsf{sk}_i, \mathsf{sid}, \mathsf{act}, \mathbf{C})$, the parties compute $\mathsf{sid}_{\mathsf{sp}} := H_{\mathsf{tr}}(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i\in\mathsf{act}})$, set $\mathbf{c} := H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}})$, and output $(\mathsf{sid}_{\mathsf{sp}}, \tilde{\sigma}_i) \leftarrow \mathrm{TGS.RoundTwo}(\mathsf{sk}_i, \mathbf{p}_i, \mathsf{sid}, \mathsf{act}, \mathbf{c})$. The final signature is $\sigma := (\mathsf{sid}_{\mathsf{sp}}, \tilde{\sigma})$, where $\tilde{\sigma} \in \mathbb{Z}^m \leftarrow \mathrm{TGS.Comb}(\mathsf{act}, \mathbf{C}, \mathbf{c}, (\mathbf{m}_i, \tilde{\sigma}_i)_{i\in\mathsf{act}})$. The verification algorithm accepts $\sigma = (\mathsf{sid}_{\mathsf{sp}}, \tilde{\sigma})$ on message m if and only if $\mathbf{C}\cdot\tilde{\sigma} = H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}}) \bmod q$ and $\|\tilde{\sigma}\|_2 \le B_{\mathsf{th}}$, where $B_{\mathsf{th}} = \sqrt{m}\tilde{m}\log(q) + \sqrt{\tau m}\,\sigma_{\mathsf{flood}}\cdot\omega(\sqrt{\log\lambda})$.

Theorem 6. *The threshold GPV signature scheme satisfies correctness.*

Proof. Let $\mathsf{pk} = \mathbf{C}$, and fix an honest signing execution on message m with active set act and session identifier sid. In the first round, every party $i \in \mathsf{act}$ samples $\mathbf{p}_i \leftarrow \mathcal{D}_{\mathbb{Z}^m,\sigma_{\mathsf{flood}}}$, publishes $\mathbf{e}_i := \mathbf{C}\mathbf{p}_i \bmod q$ and $\mathbf{m}_i := \sum_{j\in\mathsf{act},\, j\neq i} \mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid})$, and all parties compute the same transcript tag $\mathsf{sid}_{\mathsf{sp}} := H_{\mathsf{tr}}(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i\in\mathsf{act}})$ from the first-round transcript. The target syndrome for the signature is $\mathbf{c} := H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}})$. In the second round, every party computes $\mathbf{c}' := \mathbf{c} - \sum_{i\in\mathsf{act}} \mathbf{e}_i$ and $\mathbf{y}' := \mathbf{G}^{-1}(\mathbf{c}')$, and publishes $\tilde{\sigma}_i := \lambda_{i,\mathsf{act}}\mathbf{T}_{\mathbf{C},i}\mathbf{y}' + \mathbf{p}_i + \mathbf{m}_i^\star$, where $\mathbf{m}_i^\star := \sum_{j\in\mathsf{act},\, j\neq i} \mathsf{PRF}(\mathsf{seed}_{j,i}, \mathsf{sid})$. The combiner computes $\tilde{\sigma} := \sum_{i\in\mathsf{act}}(\tilde{\sigma}_i - \mathbf{m}_i)$. By mask cancellation, $\sum_{i\in\mathsf{act}} \mathbf{m}_i = \sum_{i\in\mathsf{act}} \mathbf{m}_i^\star$, and by Shamir secret sharing, it holds that $\sum_{i\in\mathsf{act}} \lambda_{i,\mathsf{act}}\mathbf{T}_{\mathbf{C},i} = \mathbf{T}_{\mathbf{C}} \bmod q$. Therefore, $\tilde{\sigma} = \mathbf{T}_{\mathbf{C}}\mathbf{y}' + \sum_{i\in\mathsf{act}} \mathbf{p}_i$, and

$$\mathbf{C}\tilde{\sigma} = \mathbf{C}\mathbf{T}_{\mathbf{C}}\mathbf{y}' + \sum_{i\in\mathsf{act}} \mathbf{C}\mathbf{p}_i = \mathbf{G}\mathbf{y}' + \sum_{i\in\mathsf{act}} \mathbf{e}_i = \left(\mathbf{c} - \sum_{i\in\mathsf{act}} \mathbf{e}_i\right) + \sum_{i\in\mathsf{act}} \mathbf{e}_i = \mathbf{c} \bmod q$$

Thus, the algebraic verification equation $\mathbf{C}\tilde{\sigma} = H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}}) \bmod q$ holds.

It remains to check the norm bound. Since $\|\mathbf{T}_{\mathbf{C}}\|_\infty \le 1$ and $\|\mathbf{y}'\|_\infty \le \lceil\log q\rceil$, we have $\|\mathbf{T}_{\mathbf{C}}\mathbf{y}'\|_2 \le \sqrt{m}\tilde{m}\log(q)$. Moreover, by the standard tail bound for sums of independent discrete Gaussians, $\|\sum_{i\in\mathsf{act}} \mathbf{p}_i\|_2 \le \sqrt{\tau m}\,\sigma_{\mathsf{flood}}\cdot\omega(\sqrt{\log\lambda})$ except with negligible probability. Thus, $\|\tilde{\sigma}\|_2 \le \sqrt{m}\tilde{m}\log(q) + \sqrt{\tau m}\,\sigma_{\mathsf{flood}}\cdot\omega(\sqrt{\log\lambda}) = B_{\mathsf{th}}$ except with negligible probability, concluding the proof. □

The threshold GPV public key consists of $\mathbf{C} \in \mathbb{Z}_q^{n\times m}$, so $|\mathsf{pk}| = nm\log q$. The final signature is $\sigma = (\mathsf{sid}_{\mathsf{sp}}, \tilde{\sigma})$ with $\|\tilde{\sigma}\|_2 \le B_{\mathsf{th}}$, and hence $|\sigma| = O(m\log B_{\mathsf{th}} + \lambda_{\mathsf{tr}})$. A signing execution communicates $(\mathbf{e}_i, \mathbf{m}_i) \in \mathbb{Z}_q^n \times \mathbb{Z}_q^m$ in the first round and $\tilde{\sigma}_i \in \mathbb{Z}_q^m$ in the second round for each active party, for a total communication of $O(\tau(n+2m)\log q)$ bits.

We next show TS-UF-1 security [10,34] of our threshold GPV signatures.

Theorem 7. *Let $\mathcal{A}$ be a PPT adversary making at most q_{sig} threshold signing queries, q_{tr} queries to H_{tr}, and q_{gpv} queries to H_{GPV}. Suppose PRF is a secure PRF, H_{tr} and H_{GPV} are modeled as random oracles, and $\sigma_{\mathsf{flood}} > \eta_\varepsilon(\Lambda^\perp(\mathbf{C}))$ for negligible ε satisfying $q_{\mathsf{sig}}\varepsilon = \mathsf{negl}(\lambda)$. Moreover, suppose $\sigma_{\mathsf{flood}} = O(B_{\mathsf{td}}\sqrt{q_{\mathsf{sig}}}\lambda)$. Then, there exist PPT adversaries $\mathcal{B}_1, \mathcal{B}_2$ such that*

$$\mathsf{Adv}^{\mathrm{TSIG}}_{\mathcal{A}}(1^\lambda, 1^N, 1^\tau) \le \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(1^\lambda) + \sqrt{2\left((q_{\mathsf{gpv}}+1)\mathsf{Adv}^{\mathsf{ISIS}}_{\mathcal{B}_2}(1^\lambda)\right)} + \mathsf{negl}(\lambda),$$

where the ISIS game uses the bound B_{th}. In particular, if PRF is a secure PRF, and $\mathsf{ISIS}_{q,n,m,B_{\mathsf{th}}}$ is hard, then the threshold GPV scheme satisfies TS-UF-1 security.

Proof. Let $\mathsf{corrupt} \subseteq [N]$ be the static corrupted set, with $|\mathsf{corrupt}| \le \tau - 1$. Let $\mathsf{Hybrid}_i(\mathcal{A})$ denote the output distribution of Hybrid i, and $\mathsf{Adv}_{\mathcal{A},i}$ denote $\mathcal{A}$'s advantage in Hybrid i.

Hybrid 0. This is the real TS-UF-1 game in Figure 3.

Hybrid 1. For every threshold signing query with active set act and session identifier sid, and for every ordered honest-honest pair $(i,j) \in (\mathsf{act} \setminus \mathsf{corrupt})^2$ with $i \neq j$, Ch replaces $\mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid})$ by an independent uniform vector in $\mathbb{Z}_q^m$, with consistent answers on repeated inputs. Values for pairs with at least one corrupted party are left unchanged. Then, as in Hybrid 1 of Theorem 5, there exists a PRF adversary $\mathcal{B}_1$ such that

$$|\mathsf{Adv}_{\mathcal{A},0} - \mathsf{Adv}_{\mathcal{A},1}| \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1}(1^\lambda).$$

Hybrid 2. Whenever H_{tr} is queried on $x = (\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i\in\mathsf{act}})$, Ch returns the stored value if one exists, and otherwise samples $y \leftarrow \{0,1\}^{\lambda_{\mathsf{tr}}}$ and stores $\mathsf{T}_{\mathsf{tr}}[x] = y$. If the sampled y collides with a previous H_{tr} output on a different input, or if y was previously used in an H_{GPV} query before appearing in the H_{tr} table, Ch aborts and sets $\mathsf{Bad}_{\mathsf{tag}}$. Since H_{tr} is a random oracle, as argued in Hybrid 2 of the proof of Theorem 5,

$$|\mathsf{Adv}_{\mathcal{A},1} - \mathsf{Adv}_{\mathcal{A},2}| \leq \Pr[\mathsf{Bad}_{\mathsf{tag}}] = \mathsf{negl}.$$

Hybrid 3. Fix a threshold signing query with active set act and session identifier sid. Let $\mathcal{C} := \mathsf{act} \cap \mathsf{corrupt}$, $\mathcal{H} := \mathsf{act} \setminus \mathcal{C}$, and $h \in \mathcal{H}$ be the canonical last honest party, and define $\mathcal{H}^- := \mathcal{H} \setminus \{h\}$. For every $i \in \mathsf{act}$, Ch samples $\mathbf{p}_i \leftarrow \mathcal{D}_{\mathbb{Z}^m,\sigma_{\mathsf{flood}}}$ and sets $\mathbf{e}_i := \mathbf{C}\mathbf{p}_i \bmod q$ and $\mathbf{m}_i := \sum_{j\in\mathsf{act},\, j\neq i} \mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid})$. It sets $\mathsf{sid}_{\mathsf{sp}} := H_{\mathsf{tr}}(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i\in\mathsf{act}})$, $\mathbf{c} := H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}})$, $\mathbf{c}' := \mathbf{c} - \sum_{i\in\mathsf{act}} \mathbf{e}_i$, and $\mathbf{y}' := \mathbf{G}^{-1}(\mathbf{c}')$. For the parties $j \in \mathcal{C}$, Ch computes $\mathbf{m}_j^\star := \sum_{k\in\mathsf{act},\, k\neq j} \mathsf{PRF}(\mathsf{seed}_{k,j}, \mathsf{sid})$ and $\tilde{\sigma}_j := \lambda_{j,\mathsf{act}}\mathbf{T}_{\mathbf{C},j}\mathbf{y}' + \mathbf{p}_j + \mathbf{m}_j^\star$ honestly. It then samples $\mathbf{m}_i \leftarrow \mathbb{Z}_q^m$ independently for all $i \in \mathcal{H}$, samples $\tilde{\sigma}_i \leftarrow \mathbb{Z}_q^m$ independently for all $i \in \mathcal{H}^-$, and sets

$$\tilde{\sigma}_h := \mathbf{T}_{\mathbf{C}}\mathbf{y}' + \sum_{i\in\mathcal{H}} \mathbf{p}_i - \sum_{i\in\mathcal{C}} \lambda_{i,\mathsf{act}}\mathbf{T}_{\mathbf{C},i}\mathbf{y}' - \sum_{i\in\mathcal{H}\setminus\{h\}} \tilde{\sigma}_i + \sum_{i\in\mathcal{H}} \mathbf{m}_i + \Delta_{\mathcal{C}}$$

where $\Delta_{\mathcal{C}} := \sum_{i\in\mathcal{H}}(\sum_{j\in\mathcal{C}} \mathsf{PRF}(\mathsf{seed}_{j,i}, \mathsf{sid}) - \sum_{j\in\mathcal{C}} \mathsf{PRF}(\mathsf{seed}_{i,j}, \mathsf{sid}))$. As argued in Hybrid 3 of the proof of Theorem 5, by Lemma 12, $\mathsf{Adv}_{\mathcal{A},2} = \mathsf{Adv}_{\mathcal{A},3}$.

Hybrid 4. Hybrid 4 is identical to Hybrid 3 except that, in each threshold signing query, the value $\mathbf{z}_h^{\mathsf{real}} := \mathbf{T}_{\mathbf{C}}\mathbf{y}' + \mathbf{p}_h$ is replaced by a centered Gaussian over the same exposed-syndrome coset for the canonical honest party h. Define $\mathbf{d}_h := \mathbf{c} - \sum_{i\in\mathsf{act}\setminus\{h\}} \mathbf{e}_i$. Then, Ch samples $\mathbf{z}_h^{\mathsf{sim}} := \tilde{\sigma}' \leftarrow \mathcal{D}_{\Lambda^{\mathbf{d}_h}(\mathbf{C}),\sigma_{\mathsf{flood}},\mathbf{0}}$ and sets

$$\tilde{\sigma}_h := \tilde{\sigma}' - \sum_{i\in\mathcal{C}} \lambda_{i,\mathsf{act}}\mathbf{T}_{\mathbf{C},i}\mathbf{y}' - \sum_{i\in\mathcal{H}\setminus\{h\}} \tilde{\sigma}_i + \sum_{i\in\mathcal{H}} \mathbf{m}_i + \Delta_{\mathcal{C}}$$

The already published value $\mathbf{e}_h$ remains consistent, since $\mathbf{p}_h^{\mathsf{virt}} := \mathbf{z}_h^{\mathsf{sim}} - \mathbf{T}_{\mathbf{C}}\mathbf{y}'$ satisfies $\mathbf{C}\cdot\mathbf{p}_h^{\mathsf{virt}} = \mathbf{e}_h \bmod q$.

Now, as argued in Hybrid 4 of the proof of Theorem 5, by Lemma 11, the data processing inequality, and the adaptive product rule in Lemma 27, when $\sigma_{\mathsf{flood}} = O(B_{\mathsf{td}}\sqrt{q_{\mathsf{sig}}\lambda})$, for a sufficiently large constant in the asymptotic term,

$$R_2(\mathsf{Hybrid}_3(\mathcal{A}) \| \mathsf{Hybrid}_4(\mathcal{A})) \leq \left(\frac{1+\varepsilon}{1-\varepsilon}\right)^{2q_{\mathsf{sig}}} \exp\left(2\pi q_{\mathsf{sig}} B_{\mathsf{td}}^2/\sigma_{\mathsf{flood}}^2\right) \leq 2.$$

Now, since $\mathbf{z}_h^{\mathsf{real}}$ and $\mathbf{z}_h^{\mathsf{sim}}$ are supported on the same coset $\Lambda^{\mathbf{d}_h}(\mathbf{C})$, and all other transcript values are sampled or computed over the same support in the two hybrids, we have $\mathrm{Supp}(\mathsf{Hybrid}_3) \subseteq \mathrm{Supp}(\mathsf{Hybrid}_4)$. Therefore, by the probability preservation property in Lemma 27, for any measurable event E,

$$\Pr_{\mathsf{Hybrid}_4}[\mathsf{E}] \geq \frac{\Pr_{\mathsf{Hybrid}_3}[\mathsf{E}]^2}{R_2(\mathsf{Hybrid}_3(\mathcal{A}) \| \mathsf{Hybrid}_4(\mathcal{A}))}.$$

This implies that

$$\mathsf{Adv}_{\mathcal{A},4} \geq \frac{\mathsf{Adv}_{\mathcal{A},3}^2}{2}.$$

Hybrid 5. We next replace $\mathbf{C}$ by a uniformly random matrix and sample the Shamir secret shares of the corrupt parties uniformly from $\mathbb{Z}_q^{m \times \tilde{m}}$. By Lemma 1 and the perfect privacy of Shamir secret sharing, $\mathsf{Hybrid}_4(\mathcal{A}) \approx_s \mathsf{Hybrid}_5(\mathcal{A})$, which implies $|\mathsf{Adv}_{\mathcal{A},4} - \mathsf{Adv}_{\mathcal{A},5}| = \mathsf{negl}(\lambda)$.

Hybrid 6. The random oracle H_{GPV} is now answered as follows. On a query $(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}})$, if $\mathsf{sid}_{\mathsf{sp}}$ is not in the range of the H_{tr} table, the challenger answers uniformly at random. Conditioned on $\neg\mathsf{Bad}_{\mathsf{tag}}$, such an unregistered answer is never later used in a signing query. If $\mathsf{sid}_{\mathsf{sp}}$ has a unique preimage $(\mathsf{sid}, \mathsf{act}, (\mathbf{e}_i)_{i \in \mathsf{act}})$ in the H_{tr} table and the query is not yet answered, Ch chooses the canonical $h \in \mathcal{H}$, samples $\tilde{\sigma}' \leftarrow \mathcal{D}_{\mathbb{Z}^m, \sigma_{\mathsf{flood}}}$, and sets

$$H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}}) := \mathbf{C} \cdot \tilde{\sigma}' + \sum_{i \in \mathsf{act} \setminus \{h\}} \mathbf{e}_i,$$

and stores $(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}}, \tilde{\sigma}'; H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}}))$. Since $\mathbf{C}$ is uniform and $\sigma_{\mathsf{flood}} \geq \omega(\sqrt{\log m})$, the value $\mathbf{C}\tilde{\sigma}'$ is statistically close to uniform over $\mathbb{Z}_q^n$ by Lemma 17, even conditioned on the fixed offset $\sum_{i \in \mathsf{act} \setminus \{h\}} \mathbf{e}_i$. Moreover, conditioned on the programmed value, $\tilde{\sigma}'$ has distribution $\mathcal{D}_{\Lambda^{\mathbf{d}_h}(\mathbf{C}), \sigma_{\mathsf{flood}}, \mathbf{0}}$. Thus, $\mathsf{Adv}_{\mathcal{A},5} = \mathsf{Adv}_{\mathcal{A},6}$.

ISIS reductions. Let Forge be the event that $\mathcal{A}$ wins, *i.e.*, for $(\mathsf{m}, \mathsf{sid})$ output by the adversary, $|T(\mathsf{m})| < \tau - |\mathcal{C}|$. For Forge, we construct an adversary $\mathcal{B}_2$ for $\mathsf{ISIS}_{q,n,m,B_{\mathsf{th}}}$. It receives an ISIS challenge $(\mathbf{C}, \mathbf{y})$, guesses an index $\iota \leftarrow \{1, \ldots, q_{\mathsf{gpv}}\}$ for the H_{GPV} query corresponding to the final fresh forgery, and simulates Hybrid 6 using the challenge matrix $\mathbf{C}$. On the guessed H_{GPV} query $(\mathsf{m}^\star, \mathsf{sid}_{\mathsf{sp}}^\star)$, it sets $H_{\mathsf{GPV}}(\mathsf{m}^\star, \mathsf{sid}_{\mathsf{sp}}^\star) := \mathbf{y}$ and marks $(\mathsf{m}^\star, \mathsf{sid}_{\mathsf{sp}}^\star)$ as the target pair. For every other transcript tag, it answers H_{GPV} as in Hybrid 6 by sampling $\tilde{\sigma}'$ and programming $H_{\mathsf{GPV}}(\mathsf{m}, \mathsf{sid}_{\mathsf{sp}}) := \mathbf{C}\tilde{\sigma}' + \sum_{i \in \mathsf{act} \setminus \{h\}} \mathbf{e}_i$. If a signing query would cause $|S_1(\mathsf{m})| \geq \tau - |\mathsf{corrupt}|$ for the target pair, the reduction may abort, since a later forgery on that target pair would not be in the event Forge. Note that $\mathcal{A}$ can simulate the signing queries for the parties $j \in \mathcal{H}^-$ and the target message m as in Hybrid 3. If $\mathcal{A}$ outputs a fresh forgery $\sigma^\star = (\mathsf{sid}_{\mathsf{sp}}^\star, \tilde{\sigma}^\star)$ on the target pair, then $\mathbf{C}\tilde{\sigma}^\star = \mathbf{y} \bmod q$ and $\|\tilde{\sigma}^\star\|_2 \leq B_{\mathsf{th}}$, so $\tilde{\sigma}^\star$ is a solution to $\mathsf{ISIS}_{q,n,m,B_{\mathsf{th}}}$. Except with negligible probability, $\mathcal{A}$ must have queried $H_{\mathsf{GPV}}(\mathsf{m}^\star, \mathsf{sid}_{\mathsf{sp}}^\star)$, or else guessed an independently uniform value in $\mathbb{Z}_q^n$. Thus,

$$\Pr[\mathsf{Forge}] \leq (q_{\mathsf{gpv}} + 1)\mathsf{Adv}_{\mathcal{B}_2}^{\mathsf{ISIS}}(1^\lambda) + \mathsf{negl}(\lambda).$$

Therefore, we observe that

$$\Pr[\mathsf{Forge}] \leq (q_{\mathsf{gpv}} + 1)\mathsf{Adv}_{\mathcal{B}_2}^{\mathsf{ISIS}}(1^\lambda) + \mathsf{negl}(\lambda).$$

Combining all hybrids, we conclude that

$$\mathsf{Adv}^{\mathrm{TSIG}}_{\mathcal{A}} \leq \mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{B}_1} + \delta_{\mathsf{tag}} + \sqrt{2(q_{\mathsf{gpv}}+1)\mathsf{Adv}^{\mathsf{ISIS}}_{\mathcal{B}_2}} + \mathsf{negl}(\lambda).$$

□

8 Batch IBE from Attribute Based Encryption

In this section, we present a batch IBE system that uses, as a black-box, a KP-ABE scheme with attribute space $\mathcal{I} = \{0,1\}^k$ and message space $\mathcal{M}$, instantiated with the following class of functions:

$$\mathcal{F}_{k,\ell} = \left\{ f_S : \{0,1\}^k \to \{0,1\} \ : \ S \subseteq \{0,1\}^k,\ |S| \leq \ell,\ f_S(x) = 0 \text{ if and only if } x \in S \right\}.$$

8.1 Definitions

Definition 18. *A* **key-policy attribute-based encryption (KP-ABE)** *scheme for some class of functions $\mathcal{F}$ and message space $\mathcal{M}$ consists of four algorithms:*

- $\mathrm{ABE.}\textsc{Setup}(1^\lambda, \mathcal{F}) \to (\mathsf{mpk}, \mathsf{msk})$: *The setup algorithm takes as input the security parameter λ and the description of the class of functions $\mathcal{F}$, and outputs the master public key mpk and the master secret key msk.*
- $\mathrm{ABE.}\textsc{Enc}(\mathsf{mpk}, x, \mathsf{m}) \to \mathsf{ct}$: *The encryption algorithm takes as input mpk, an attribute $x \in \{0,1\}^*$ and a message $\mathsf{m} \in \mathcal{M}$, and outputs a ciphertext ct.*
- $\mathrm{ABE.}\textsc{KeyGen}(\mathsf{msk}, f) \to \mathsf{sk}_f$: *The key generation algorithm takes as input msk and a function $f \in \mathcal{F}$, and outputs a secret key sk_f.*
- $\mathrm{ABE.}\textsc{Dec}(\mathsf{sk}_f, (x, \mathsf{ct})) \to m$: *The decryption algorithm takes as input a secret key sk_f and an attribute-ciphertext tuple (x, ct), and if $f(x) = 0$, outputs the corresponding message m.*

Definition 19 (Correctness of KP-ABE, taken from [5]). *For every function $f \in \mathcal{F}$, attribute strings $x \in \{0,1\}^*$, where $f(x) = 0$, and messages $\mathsf{m} \in \mathcal{M}$, it holds that*

$$\Pr\left[\mathrm{ABE.}\textsc{Dec}(\mathsf{sk}_f, (x, \mathsf{ct})) = \mathsf{m} \middle| \begin{smallmatrix} (\mathsf{mpk},\mathsf{msk}) \leftarrow \mathrm{ABE.}\textsc{Setup}(1^\lambda, \mathcal{F}) \\ \mathsf{ct} \leftarrow \mathrm{ABE.}\textsc{Enc}(\mathsf{mpk}, x, \mathsf{m}) \\ \mathsf{sk}_f \leftarrow \mathrm{ABE.}\textsc{KeyGen}(\mathsf{msk}, f) \end{smallmatrix}\right] > 1 - \mathsf{negl}(\lambda)$$

Definition 20 (Selective Security of KP-ABE, taken from [5]). *A KP-ABE scheme for a class of functions $\mathcal{F}_\lambda = \{f : \mathcal{X}_\lambda \to \mathcal{Y}_\lambda\}$ is selectively secure if for all PPT adversaries $\mathcal{A}$, where $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$, it holds that*

$$\left|\Pr[\mathsf{Game}^{\mathrm{ABE}}_{\mathcal{A},0}(\lambda) = 1] - \Pr[\mathsf{Game}^{\mathrm{ABE}}_{\mathcal{A},1}(\lambda) = 1]\right| \leq \mathsf{negl}(\lambda),$$

The Security Game $\mathsf{Game}^{\mathrm{ABE}}_{\mathcal{A},b}(\lambda)$ for Functions $\mathcal{F}_\lambda$

Setup. The adversary first commits to the challenge attribute $x^\star$ and sends it to the challenger. Ch generates $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathrm{ABE.}\textsc{Setup}(1^\lambda, \mathcal{F}_\lambda)$ and sends mpk to $\mathcal{A}$.

Key-generation oracle. The adversary has access to the oracle $\mathsf{KG}(\cdot)$ defined as follows. On input $f \in \mathcal{F}_\lambda$, the oracle returns

$$\mathsf{KG}(f) \coloneqq \begin{cases} \mathrm{ABE.}\textsc{KeyGen}(\mathsf{msk}, f), & \text{if } f(x^\star) \neq 0, \\ \bot, & \text{otherwise.} \end{cases}$$

Pre-challenge phase. $\mathcal{A}$ outputs two equal-length messages m_0 and m_1.

Challenge ciphertext. Ch computes $\mathsf{ct}^\star \leftarrow \mathrm{ABE.}\textsc{Enc}(\mathsf{mpk}, x^\star, \mathsf{m}_b)$ and sends $\mathsf{ct}^\star$ to $\mathcal{A}$.

Post-challenge phase. $\mathcal{A}$ continues to access $\mathsf{KG}(\cdot)$ and outputs some b'.

Output. The game outputs b'.

Figure 4: The ABE security game

A fully secure ABE scheme has a similar definition, except that the adversary determines the challenge attribute x^* after receiving mpk and making a polynomial number of secret key queries. Any selectively secure ABE scheme can be converted into a fully secure one at the cost of an exponential loss in the parameter size [14].

8.2 The Construction

BIBE.Setup(1^λ)**.** The setup algorithm invokes

$$\mathrm{ABE.}\textsc{Setup}(1^\lambda, \mathcal{F}_{k,\ell}) \to (\mathrm{ABE.}\mathsf{mpk}, \mathrm{ABE.}\mathsf{msk})$$

and outputs $\mathrm{BIBE.}\mathsf{pk} \coloneqq \mathrm{ABE.}\mathsf{mpk}$ and $\mathrm{BIBE.}\mathsf{sk} \coloneqq \mathrm{ABE.}\mathsf{msk}$.

BIBE.Enc$(\mathbf{BIBE}.\mathsf{pk}, r \in \{0,1\}^k, m \in \mathcal{M})$**.** Parsing $\mathrm{ABE.}\mathsf{mpk} \coloneqq \mathrm{BIBE.}\mathsf{pk}$, the encryption algorithm calls

$$\mathrm{ABE.}\textsc{Enc}(\mathrm{ABE.}\mathsf{mpk}, r, m) \to \mathrm{ABE.}\mathsf{ct}.$$

and outputs $\mathsf{ct} \coloneqq (r, \mathrm{ABE.}\mathsf{ct})$.

BIBE.PreDec$(\mathbf{BIBE}.\mathsf{sk}, (r_i)_{i\in[\ell]})$**.** Parsing $\mathrm{ABE.}\mathsf{msk} \coloneqq \mathrm{BIBE.}\mathsf{sk}$, the pre-decryption algorithm invokes

$$\mathrm{ABE.}\textsc{KeyGen}(\mathrm{ABE.}\mathsf{msk}, f) \to \mathrm{ABE.}\mathsf{sk}_f,$$

with the function $f\colon \{0,1\}^k \to \{0,1\}$ such that $f(x) = 0$ if and only if $x \in (r_i)_{i\in[\ell]}$. It outputs $\mathrm{BIBE.}\mathsf{sbk} \coloneqq \mathrm{ABE.}\mathsf{sk}_f$.

BIBE.Dec$(\mathbf{BIBE}.\mathsf{pk}, \mathbf{BIBE}.\mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]})$**.** The decryption algorithm parses $\mathrm{ABE.}\mathsf{pk} \coloneqq \mathrm{BIBE.}\mathsf{pk}$, $\mathrm{ABE.}\mathsf{sk}_f \coloneqq \mathrm{BIBE.}\mathsf{sbk}$, and $(r_i, \mathrm{ABE.}\mathsf{ct}_i) \coloneqq \mathsf{ct}_i$. Then, for each $i \in [\ell]$, it outputs

$$\mathrm{ABE.}\textsc{Dec}(\mathsf{sk}_f, (r_i, \mathrm{ABE.}\mathsf{ct}_i)) \to \mathsf{m}_i.$$

8.3 Correctness

For $(\text{BIBE.}\mathsf{pk}, \text{BIBE.}\mathsf{sk}) \coloneqq (\text{ABE.}\mathsf{mpk}, \text{ABE.}\mathsf{msk}) \leftarrow \text{ABE.}\textsc{Setup}(1^\lambda, \mathcal{F})$, correct ciphertexts $\mathsf{ct}_i \coloneqq (r_i \in \{0,1\}^k, \text{ABE.}\mathsf{ct}_i)$, where

$$\text{ABE.}\mathsf{ct}_i \leftarrow \text{ABE.}\textsc{Enc}(\text{ABE.}\mathsf{mpk}, r_i, m_i)$$

for $i \in [\ell]$, and a correct pre-decryption key

$$\text{BIBE.}\mathsf{sbk} \coloneqq \mathsf{sk}_f \leftarrow \text{ABE.}\textsc{KeyGen}(\text{ABE.}\mathsf{msk}, f)$$

for the function $f \in \mathcal{F}_{k,l}$ such that $f(x) = 0$ if and only if $x \in (r_i)_{i\in[\ell]}$, by the correctness of the KP-ABE scheme,

$$\text{BIBE.}\textsc{Dec}(\text{BIBE.}\mathsf{pk}, \text{BIBE.}\mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}) = (\text{ABE.}\textsc{Dec}(\mathsf{sk}_f, (r_i, \text{ABE.}\mathsf{ct}_i)))_{i\in[\ell]} = (\mathsf{m}_i)_{i\in[\ell]}$$

except with negligible probability.

8.4 Security

Theorem 8. *The* BIBE *scheme instantiated with a selectively secure* ABE *protocol is selectively secure (as in Definition 3).*

Proof. Suppose that a PPT adversary $\mathcal{A}$ has a non-negligible advantage ϵ against the BIBE game of Definition 3. Then, we construct an adversary $\mathcal{B}$ with advantage at least ϵ in the ABE security game. Here, $\mathcal{B}$ simulates the view of $\mathcal{A}$ as follows:

- $\mathcal{B}$ receives a challenge identity r^* from $\mathcal{A}$ and sends the challenge attribute $x^* \coloneqq r^*$ to Ch.
- $\mathcal{B}$ receives $\text{ABE.}\mathsf{mpk}$ for the class of functions $\mathcal{F}_{k,l}$ from the challenger Ch and sends $\text{BIBE.}\mathsf{pk} \coloneqq \text{ABE.}\mathsf{mpk}$ to $\mathcal{A}$.
- Whenever $\mathcal{A}$ sends a pre-challenge query for the identities $(r_1, \ldots, r_\ell)$ that does not contain r^*, $\mathcal{B}$ queries the oracle $\mathsf{KG}(\text{ABE.}\mathsf{msk}, x^*, \cdot)$ with the function $f\colon \{0,1\}^k \to \{0,1\}$ such that $f(x) = 0$ if and only if $x \in (r_i)_{i\in[\ell]}$. Upon receiving the secret key sk_f, it forwards $\text{BIBE.}\mathsf{sbk} \coloneqq \mathsf{sk}_f$ to $\mathcal{A}$. (If $r^* \in (r_1, \ldots, r_\ell)$, then $\mathcal{B}$ instead aborts and returns a random bit.)
- When $\mathcal{A}$ sends the two messages $(\mathsf{m}_0, \mathsf{m}_1)$ it would like to be challenged on, $\mathcal{B}$ forwards $\mathsf{m}_0, \mathsf{m}_1$ to Ch. Upon receiving a ciphertext ct^* from Ch, it forwards (r^*, ct^*) to $\mathcal{A}$.
- For the post-challenge queries, $\mathcal{B}$ repeats the same approach as the pre-challenge queries.
- Once $\mathcal{A}$ outputs a bit b', $\mathcal{B}$ outputs the same bit.

We now analyze the reduction. Conditioned on $\mathcal{B}$ not aborting, $\mathsf{KG}(\text{ABE.}\mathsf{msk}, r^*, \cdot)$ responds with the correct secret key $\mathsf{sk}_f \leftarrow \text{ABE.}\textsc{KeyGen}(\text{ABE.}\mathsf{msk}, f)$, which is the same as the pre-decryption key $\text{BIBE.}\mathsf{sbk}$ corresponding to the sequence of identities defining the function f. Hence, the view $\mathcal{B}$ provides to $\mathcal{A}$ is statistically indistinguishable from the view of a real interaction with the challenger of the BIBE scheme. Hence, $\mathcal{A}$ has advantage ϵ against the BIBE game of Definition 3. Now, when $\mathcal{A}$ wins the BIBE game in Definition 3, $\mathcal{B}$ wins the ABE game in Definition 20. Therefore, $\mathcal{B}$ has advantage at least ϵ against the ABE game of Definition 20. □

8.5 Parameter Sizes

We next estimate the public key, ciphertext and pre-decryption key sizes achieved by the BIBE systems instantiated with LWE-based KP-ABE schemes with succinct secret keys. Let k denote the identity-tag length, let ℓ denote the maximum batch size, and let $D_{k,\ell} := O(\log k + \log \ell)$ denote the depth of the membership circuit for $\mathcal{F}_{k,\ell}$. As in the main text, $O_\lambda(\cdot)$ suppresses factors polynomial in the security parameter. When $k \le \mathsf{poly}(\lambda)$, we also absorb $\mathsf{poly}(\log k)$ factors into $O_\lambda(\cdot)$. For the KP-ABE rows in Table 1, we count the cryptographic overhead of the ABE public key, ciphertext, and secret key, and not the explicit description of the identity list, which is already present in the batch of ciphertexts given to BIBE.Dec.

The KP-ABE scheme by BGG+14 [19]. The KP-ABE scheme of Boneh et al. [19] supports boolean and arithmetic circuits and has succinct secret keys. Instantiating it with the membership circuit above gives depth $D_{k,\ell} = O(\log k + \log \ell)$. Following the parameterization in [19], the public key and ciphertext retain a linear dependence on the attribute length k, and the lattice parameters depend polynomially on the circuit depth. Under the same convention as in Table 1, the resulting batch IBE has

$$|\mathsf{pk}| = O_\lambda(k \log^2 \ell), \qquad |\mathsf{ct}| = O_\lambda(k \log^2 \ell), \qquad |\mathsf{sbk}| = O_\lambda(\log^2 \ell).$$

The KP-ABE scheme by CW23 [31]. The KP-ABE scheme by Cini and Wee [31] supports boolean circuits and achieves keys of size $\mathsf{poly}(\lambda)$, independent of the circuit depth. Instantiating it with the membership circuit above gives

$$|\mathsf{pk}| = O_\lambda(k \log^2 \ell), \qquad |\mathsf{sbk}| = \mathsf{poly}(\lambda), \qquad |\mathsf{ct}| = O_\lambda(k \log^2 \ell).$$

Here the public key and ciphertext inherit the bounded-attribute dependence on the input length k.

The unbounded KP-ABE scheme by Cini–Wee24 [32]. The unbounded ABE scheme of Cini and Wee [32] is based on plain LWE and removes the dependence on the attribute length from the public key. For depth-d circuits over k-bit inputs, where only d is fixed at setup, its public key has size $\mathsf{poly}(d, \lambda)$, while ciphertexts have size $k \cdot \mathsf{poly}(d, \lambda)$. The ciphertext size is only a constant factor larger than the corresponding bounded BGG+14 ciphertext. For our membership circuit, this gives

$$|\mathsf{pk}| = \mathsf{poly}(\lambda, D_{k,\ell}), \qquad |\mathsf{ct}| = k \cdot \mathsf{poly}(\lambda, D_{k,\ell}).$$

The secret key consists of a private component of size $\mathsf{poly}(\lambda, D_{k,\ell})$ and a public component of size $k \cdot \mathsf{poly}(\lambda, D_{k,\ell})$. The public component can be reused across different keys for circuits over k-bit inputs. Thus, if this public component is amortized, the per-batch pre-decryption key has size

$$|\mathsf{sbk}| = \mathsf{poly}(\lambda, D_{k,\ell}),$$

whereas counting the reusable public component inside each key gives

$$|\mathsf{sbk}| = \mathsf{poly}(\lambda, D_{k,\ell}) + k \cdot \mathsf{poly}(\lambda, D_{k,\ell})_{\mathsf{pub}}.$$

The KP-ABE scheme by Wee25 [63]. The KP-ABE scheme of Wee [63] gives almost optimal KP-ABE and CP-ABE for depth-d circuits from succinct LWE. For depth-d circuits over k-bit inputs, its public key, ciphertext, and secret key sizes are independent of the attribute length k, where the hidden factors are $\mathsf{poly}(d, \lambda)$. Instantiating it with the membership circuit gives

$$|\mathsf{pk}| = \mathsf{poly}(\lambda, D_{k,\ell}), \qquad |\mathsf{sbk}| = \mathsf{poly}(\lambda, D_{k,\ell}), \qquad |\mathsf{ct}| = \mathsf{poly}(\lambda, D_{k,\ell}).$$

In particular, when $k \leq \mathsf{poly}(\lambda)$, all three quantities are $\mathsf{poly}(\lambda, \log \ell)$. This removes the identity-length dependence from all three parameters, at the price of relying on succinct LWE.

Remark 4 (On decomposed LWE). Abram, Malavolta, and Roy [1] introduce decomposed LWE and show that it is implied by succinct LWE. Their work also gives ABE-style applications for RAM programs from decomposed LWE and standard lattice assumptions. For the comparison in Table 1, we treat decomposed LWE as a possible future weakening of the succinct-LWE assumption, rather than as a separate KP-ABE instantiation for our circuit-membership function class.

Comparison with our LWE-based batch IBE. The newer KP-ABE schemes improve the generic ABE baseline substantially. Cini–Wee24 removes the linear dependence on the identity length from the public key under plain LWE, while Wee25 removes it from the public key, ciphertext, and pre-decryption key under succinct LWE. Our direct LWE-based BIBE scheme remains incomparable: it relies on standard LWE in the random oracle model, supports a tunable index length d, and has a threshold version.

9 Batch IBE from Trilinear Maps

In this section, we present a batch IBE system based on trilinear maps.

9.1 Background on Trilinear Maps and Generic Group Model

Let $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_3$, $\mathbb{G}_\mathrm{t}$ be multiplicative groups of prime order p with generators G_1, G_2, G_3, G_T, respectively, equipped with a non-degenerate trilinear map $(\cdot)\colon \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_3 \to \mathbb{G}_\mathrm{t}$ satisfying

$$(aG_1) \cdot (bG_2) \cdot (cG_3) = abc \cdot G_T \quad \text{for all } a, b, c \in \mathbb{Z}_p.$$

Let $H\colon \mathbb{G}_\mathrm{t} \to \{0,1\}^{|\mathcal{M}|}$ denote a one-way function.

The following description of Shoup's GGM [58] is adapted from [66,4]. Let $S \subseteq \{0,1\}^*$ be a set of strings with size at least $p \in \mathbb{Z}^+$ and with a known upper bound. A group $\mathbb{G}$ of prime order p is generically represented by sampling a random injection $L\colon \mathbb{Z}_p \to S$, known as the labeling function. This function encodes the string representation of xG, where $G \in \mathbb{G}$ is a fixed generator of $\mathbb{G}$. In the following sections, L will be generated dynamically as new queries arrive, and we will treat it as a dictionary indexed by values from its domain. The model provides the following query functionalities:

- **Labeling queries:** Given $x \in \mathbb{Z}_p$, returns $L(x)$.
- **Group operations:** Given $(\ell_1, \ell_2, a_1, a_2) \in S^2 \times \mathbb{Z}_p^2$, if there are $x_1, x_2 \in \mathbb{Z}_p$ such that $L(x_1) = \ell_1$ and $L(x_2) = \ell_2$, returns $L(a_1x_1 + a_2x_2)$. Otherwise, returns $\perp$.

In the trilinear group setting, the model supports an additional query type:

- **Mapping query:** Given $(\ell_1, \ell_2, \ell_3) \in S_1 \times S_2 \times S_3$, if there are $x_1, x_2, x_3 \in \mathbb{Z}_p$ such that $L_1(x_1) = \ell_1$, $L_2(x_2) = \ell_2$, and $L_3(x_3) = \ell_3$, returns $L_T(x_1 \cdot x_2 \cdot x_3)$. Otherwise, returns $\perp$.

9.2 The Construction

BIBE.Setup(1^λ)**.** The setup algorithm samples $\alpha \leftarrow \mathbb{Z}_p$ and outputs the secret key BIBE.$\mathsf{sk} \coloneqq \alpha$. It calculates $A_1 \coloneqq \alpha G_1$, and $B_i \coloneqq \alpha^i G_2$ for $i \in [\ell - 1]$, and outputs the public key BIBE.$\mathsf{pk} \coloneqq (A_1, B_1, \ldots, B_{\ell-1})$.

BIBE.Enc(**BIBE**.$\mathsf{pk}, r \in \mathbb{Z}_p, \mathsf{m} \in \mathcal{M}$)**.** Parsing $(A_1, B_1, \ldots, B_{\ell-1}) \coloneqq$ BIBE.pk, the encryption algorithm samples $t \leftarrow \mathbb{Z}_p$ and outputs $\mathsf{ct} \coloneqq (r, \mathsf{ct}[1], \mathsf{ct}[2])$:

$$\mathsf{ct}[1] \coloneqq tA_1 + rtG_1 \qquad \mathsf{ct}[2] \coloneqq m \oplus H(tG_T)$$

BIBE.PreDec(**BIBE**.$\mathsf{sk}, (r_i)_{i\in[\ell]}$)**.** Parsing $\alpha \coloneqq$ BIBE.sk, the pre-decryption algorithm outputs

$$\text{BIBE.}\mathsf{sbk} \coloneqq \frac{1}{\prod_{i\in[\ell]}(\alpha + r_i)} G_3.$$

BIBE.Dec(**BIBE**.pk, **BIBE**.$\mathsf{sbk}, (\mathsf{ct}_i)_{i\in[\ell]}$)**.** Parsing the public key and ciphertexts $(r_i, \mathsf{ct}_i[1], \mathsf{ct}_i[2]) \coloneqq \mathsf{ct}_i$, for each $i \in [\ell]$, the decryption algorithm finds the coefficients $(f_i^j)_{j=0,\ldots,\ell-2}$ of the polynomials

$$f_i(X) = X^{\ell-1} + \sum_{j=0}^{\ell-2} f_i^j X^j = \prod_{j\in[\ell], j\neq i} (X + r_j).$$

Then, for each $i \in [\ell]$, it calculates

$$f_i(\alpha)G_2 = \prod_{j\in[\ell], j\neq i} (\alpha + r_j)G_2 = B_{\ell-1} + \sum_{j=0}^{\ell-2} f_i^j B_j,$$

where $B_0 = G_2$. Finally, for $i \in [\ell]$, it outputs

$$\mathsf{m}_i' \coloneqq \mathsf{ct}_i[2] \oplus H(\mathsf{ct}_i[1] \cdot f_i(\alpha)G_2 \cdot \text{BIBE.}\mathsf{sbk}).$$

9.3 Correctness

For a correct ciphertext $\mathsf{ct}_i \coloneqq (r_i, \mathsf{ct}_i[1], \mathsf{ct}_i[2])$, where $\mathsf{ct}_i[1] \coloneqq tA_1 + r_i tG_1$ and $\mathsf{ct}_i[2] \coloneqq \mathsf{m}_i \oplus H(tG_T)$, and correct pre-decryption key $\mathsf{sbk} \coloneqq \frac{1}{\prod_{i\in[\ell]}(\alpha+r_i)} G_3$, it holds that

$$\begin{aligned}
\mathsf{m}_i' =& \mathsf{ct}_i[2] \oplus H(\mathsf{ct}_i[1] \cdot f_i(\alpha)G_2 \cdot \text{BIBE.}\mathsf{sbk}) \\
=& \mathsf{m}_i \oplus H(tG_T) \oplus H\left((\alpha + r_i)tG_1 \cdot \prod_{j\in[\ell], j\neq i} (\alpha + r_j)G_2 \cdot \frac{1}{\prod_{i\in[\ell]}(\alpha + r_i)} G_3\right) \\
=& \mathsf{m}_i \oplus H(tG_T) \oplus H(tG_T) = \mathsf{m}_i
\end{aligned}$$

9.4 Thresholdizing the Secret Key

To thresholdize the pre-decryption key $\frac{1}{\prod_{i\in[\ell]}(\alpha+r_i)}G_3$, where $r_1,\ldots,r_\ell$ are all distinct, we can first express it as a sum of partial fractions:

$$\frac{1}{\prod_{i\in[\ell]}(\alpha+r_i)}G_3 = \sum_{i\in[\ell]}\frac{\beta_i}{\alpha+r_i}G_3$$

for some $\beta_1,\ldots,\beta_\ell \in \mathbb{Z}_p$. Now, suppose α is t-out-of-n secret shared among n parties. Then for $i\in[\ell]$, it suffices to describe a secure protocol for computing $(1/(\alpha+r_i))G_3$ among a quorum of t parties. This problem is the same as computing a threshold signature in the Boneh-Boyen signature scheme [13], and Wang et al. [60] give an efficient two-round protocol for this problem. Briefly, the parties first jointly compute and publish $\beta = s(\alpha+r_i) \in \mathbb{Z}_p$ in the clear, where $s \leftarrow \mathbb{Z}_p$ is additively secret shared among them, say $s = s_1 + \ldots + s_t$. Each party then publishes $P_i = (s_i/\beta)G_3$. Then, $\sum_{i\in[t]} P_i = (1/(\alpha+r_i))G_3$, as required.

9.5 Security

Theorem 9. *For all $\ell \in \mathbb{N}$ and unbounded adversaries $\mathcal{A}$ making at most q queries to the group oracle, hash oracle, and for key computation, the* BIBE *scheme is selectively secure (Definition 3) in the generic group model.*

For the proof of Theorem 9, we follow the GGM proof blueprint and notation by Agarwal, Fernando, Pinkas [4], applied to our protocol. The challenger simulates the GGM oracle by selecting secret integers $x_1,\ldots,x_n \in \mathbb{Z}_p$ and responding to the adversary's queries with $L(f(x_1,\ldots,x_n))$ for some functions f. In the security proof, $x_i \in \mathbb{Z}_p$ are replaced with indeterminates X_i as in the symbolic model of Shoup [58], and the challenger reveals $L(f(X_1,\ldots,X_n))$ to the adversary. Before we present the proof, in Figure 1 and Figure 2, we describe the security game in the generic group and symbolic models respectively.

Proof of Theorem 9 relies on the following lemmas:

Lemma 13. *For all $\ell \in \mathbb{N}$ and all unbounded adversaries $\mathcal{A}$,*

$$\left|\Pr[\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},0}(1^\lambda,1^\ell)=1] - \Pr[\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},1}(1^\lambda,1^\ell)=1]\right| = 0.$$

Proof. We describe an intermediate hybrid game $\mathsf{Game}^{\mathsf{Hyb},\mathsf{SM}}_{\mathcal{A}}(1^\lambda,1^\ell)$ such that $\mathsf{Game}^{\mathsf{Hyb},\mathsf{SM}}_{\mathcal{A}}(1^\lambda,1^\ell) \equiv \mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},b}(1^\lambda,1^\ell)$ for $b\in\{0,1\}$, which implies the lemma. Here, $\mathsf{Game}^{\mathsf{Hyb},\mathsf{SM}}_{\mathcal{A}}(1^\lambda,1^\ell)$ is the same as $\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},1}(1^\lambda,1^\ell)$ except that $\mathsf{ct}^*[2]$ is computed as follows:

- The challenger defines an indeterminate X_u, sets $L_T(X_u) \leftarrow S_T$ and updates $S_T := S_T \setminus L_T(X_u)$.
- It does a hash query for $L_T(X_u)$ with output mask. It sets $\mathsf{ct}^*[2] = \mathsf{m}_b \oplus \mathsf{mask}$.

Security Game in the Generic Group Model: $\mathsf{Game}^{\mathbf{BIBE},\mathsf{GGM}}_{\mathcal{A},b}(1^\lambda, 1^\ell)$

Setup: Let $S_1 = S_2 = S_3 = S_T = \{0,1\}^{p'}$ where $2^{p'} \geq p$, and p is the group order. The challenger first initializes the labeling functions $L_i : \mathbb{Z}_p \to S_i$, $i \in \{1,2,3,T\}$, and the hash function $H : S_T \to \{0,1\}^m$. It then receives the challenge identity r^*, and runs $(\text{BIBE.}\mathsf{pk}, \text{BIBE.}\mathsf{sk}) \leftarrow \text{BIBE.}\textsc{Setup}(1^\lambda, 1^\ell)$. During the call to BIBE.Setup, it does the following:

- Sample $\alpha \leftarrow \mathbb{Z}_p$.
- Sample $e^1_\alpha \leftarrow S_1$, and set $L_1(\alpha) \coloneqq e^1_\alpha$ and $S_1 \coloneqq S_1 \setminus \{e^1_\alpha\}$.
- For $i \in [\ell - 1]$, sample $e^2_{\alpha^i} \leftarrow S_2$, and set $L_2(\alpha^i) \coloneqq e^2_{\alpha^i}$ and $S_2 \coloneqq S_2 \setminus e^2_{\alpha^i}$.

It sets $\text{BIBE.}\mathsf{pk} \coloneqq (L_1(\alpha), L_2(\alpha), \ldots, L_2(\alpha^{\ell-1}))$ and sends $\text{BIBE.}\mathsf{pk}$ to $\mathcal{A}$.

Group and hash oracle queries:

- **Labeling query:** For a group $\mathbb{G}_i$, upon receiving $x \in \mathbb{Z}_p$, if $x \notin L_i$, Ch samples $e \leftarrow S_i$, sets $L_i(x) \coloneqq e$ and updates $S_i \coloneqq S_i \setminus \{e\}$. It then sends $L_i(x)$ to $\mathcal{A}$.
- **Group operation:** Upon receiving $(e_1, e_2, a_1, a_2) \in (\{0,1\}^{p'})^2 \times \mathbb{Z}_p^2$, if there are $x_1, x_2 \in \mathbb{Z}_p$ such that $L_i(x_1) = e_1$ and $L_i(x_2) = e_2$, Ch computes $x_3 \coloneqq a_1 x_1 + a_2 x_2$, does a labeling query for x_3, and sends $L_i(x_3)$ to $\mathcal{A}$. Otherwise, it sends $\perp$ to $\mathcal{A}$.
- **Map operation:** Upon receiving (e_1, e_2, e_3), if there are $x_1, x_2, x_3 \in \mathbb{Z}_p$ such that $L_1(x_1) = e_1$, $L_2(x_2) = e_2$ and $L_3(x_3) \coloneqq e_3$, Ch computes $x_T = x_1 \cdot x_2 \cdot x_3$, does a labeling query for x_T, and sends $L_T(x_T)$ to $\mathcal{A}$.
- **Hash query:** Upon receiving e, if there is $x \in \mathbb{Z}_p$ such that $L_T(x) = e$, and $e \notin H$, Ch samples $h \leftarrow \mathbb{Z}_{p'}$, sets $H(e) \coloneqq h$, and sends h to $\mathcal{A}$.

Pre and post-challenge queries: Upon receving a list of ℓ identities $(r_1, \ldots, r_l) \subseteq \mathcal{I}^\ell$ from $\mathcal{A}$, Ch does the following:

- If the challenge identity $r^* \in (r_1, \ldots, r_l)$, it halts the game.
- If $x \coloneqq \frac{1}{\prod_{i \in [\ell]} (\alpha + r_i)} \notin L_3$, it does a labeling query for x.
- It sends the pre-decryption key $\text{BIBE.}\mathsf{sbk} \coloneqq L_3(x)$ to $\mathcal{A}$.

Challenge round: Upon receiving the messages $\mathsf{m}_0, \mathsf{m}_1 \in \mathbb{Z}_{p'}$ from $\mathcal{A}$, Ch does the following:

- It samples $t^* \leftarrow \mathbb{Z}_q$ and does a group operation $(L_1(\alpha), G_1, t^*, r^* t^*)$ for L_1 with output set to $\mathsf{ct}^*[1]$.
- It does a group operation $(1, G_T, 1, t^*)$ for L_T with output e. It then does a hash query for e with output mask. It sets $\mathsf{ct}^*[2]$ to be $\mathsf{m}_b \oplus \mathsf{mask}$.
- It sends the ciphertext $\mathsf{ct}^* = (r^*, \mathsf{ct}^*[1], \mathsf{ct}^*[2])$ to $\mathcal{A}$.

Output: Once $\mathcal{A}$ halts and outputs a bit b', the game outputs b'.

Fig. 1: The security game for BIBE in the generic group model.

Security Game in the Symbolic Model: $\mathsf{Game}^{\mathbf{BIBE},\mathsf{SM}}_{\mathcal{A},b}(1^\lambda, 1^\ell)$

Setup: Let $S_1 = S_2 = S_3 = S_T = \{0,1\}^{p'}$ where $2^{p'} \geq p$, and p is the group order. The challenger first initializes the labeling functions $L_i : \mathbb{Z}_p \to S_i$, $i \in \{1,2,3,T\}$, and the hash function $H : S_T \to \{0,1\}^m$. It then receives the challenge identity r^* and runs BIBE.$\textsc{Setup}(1^\lambda, 1^\ell) \to (\text{BIBE.}\mathsf{pk}, \text{BIBE.}\mathsf{sk})$. During the call to BIBE.$\textsc{Setup}$, it does the following:

- Sample $e^1_\alpha \leftarrow S_1$, and set $L_1(X_\alpha) \coloneqq e^1_\alpha$ and $S_1 \coloneqq S_1 \setminus \{e^1_\alpha\}$, where X_α is an indeterminate corresponding to BIBE.sk.
- For $i \in [\ell - 1]$, sample $e^2_{\alpha^i} \leftarrow S_2$, and set $L_2(X^i_\alpha) \coloneqq e^2_{\alpha^i}$ and $S_2 \coloneqq S_2 \setminus e^2_{\alpha^i}$.

It sets BIBE.$\mathsf{pk} \coloneqq (L_1(X_\alpha), L_2(X_\alpha), \ldots, L_2(X^{\ell-1}_\alpha))$ and sends BIBE.pk to $\mathcal{A}$.

Group and hash oracle queries:

- **Labeling query:** For a group $\mathbb{G}_i$, upon receiving $x \in \mathbb{Z}_p$, if $x \notin L_i$, Ch samples $e \leftarrow S_i$, sets $L_i(x) \coloneqq e$ and updates $S_i \coloneqq S_i \setminus \{e\}$. It then sends $L_i(x)$ to $\mathcal{A}$.
- **Group operation:** Upon receiving $(e_1, e_2, a_1, a_2) \in (\{0,1\}^{p'})^2 \times \mathbb{Z}_p^2$, if there are polynomials $X_1, X_2 \in \mathbb{Z}_p[*]$ such that $L_i(X_1) = e_1$ and $L_i(X_2) = e_2$, Ch computes the polynomial $X_3 \coloneqq a_1 X_1 + a_2 X_2$, does a labeling query for X_3, and sends $L_i(X_3)$ to $\mathcal{A}$. Otherwise, it sends $\perp$ to $\mathcal{A}$.
- **Map operation:** Upon receiving (e_1, e_2, e_3), if there are polynomials $X_1, X_2, X_3 \in \mathbb{Z}_p[*]$ such that $L_1(X_1) = e_1$, $L_2(X_2) = e_2$ and $L_3(X_3) = e_3$, Ch computes the polynomial $X_T \coloneqq X_1 \cdot X_2 \cdot X_3$, does a labeling query for X_T, and sends $L_T(X_T)$ to $\mathcal{A}$. Otherwise, it sends $\perp$ to $\mathcal{A}$.
- **Hash query:** Upon receiving e, if there is a polynomial $X \in \mathbb{Z}_p[*]$ such that $L_T(X) = e$, and $e \notin H$, Ch samples $h \leftarrow \mathbb{Z}_{p'}$, sets $H(e) \coloneqq h$, and sends h to $\mathcal{A}$.

Pre and post-challenge queries: Upon receving a list of ℓ identities $(r_1, \ldots, r_l) \subseteq \mathcal{I}^\ell$ from $\mathcal{A}$, Ch does the following:

- If the challenge identity $r^* \in (r_1, \ldots, r_l)$, it halts the game.
- Let $X_{\mathsf{sbk}} \coloneqq \frac{1}{\prod_{i \in [\ell]}(X_\alpha + r_i)}$ be a Laurent polynomial with the indeterminate X_α. If $X_{\mathsf{sbk}} \notin L_3$, Ch does a labeling query for X_{sbk}.
- It sends the pre-decryption key BIBE.$\mathsf{sbk} \coloneqq L_3(X_{\mathsf{sbk}})$ to $\mathcal{A}$.

Challenge round: Upon receiving the messages $\mathsf{m}_0, \mathsf{m}_1 \in \mathbb{Z}_{p'}$ from $\mathcal{A}$, Ch does the following:

- Let X_{t^*} be an indeterminate. Ch sets $L_1(X_{t^*} X_\alpha + r^* X_{t^*}) \leftarrow \{0,1\}^{p'}$ and $\mathsf{ct}^*[1] \coloneqq L_1(X_{t^*} X_\alpha + r^* X_{t^*})$.
- It then sets $L_T(X_{t^*}) \leftarrow \{0,1\}^{p'}$ and does a hash query for $L_T(X_{t^*})$ with output mask. It sets $\mathsf{ct}^*[2] = \mathsf{m}_b \oplus \mathsf{mask}$.
- It sends the ciphertext $\mathsf{ct}^* = (r^*, \mathsf{ct}^*[1], \mathsf{ct}^*[2])$ to $\mathcal{A}$.

Output: Once $\mathcal{A}$ halts and outputs a bit b', the game outputs b'.

Fig. 2: The security game for BIBE in the symbolic model.

Suppose the adversary makes n queries for the pre-decryption key with the identity tags r_i, $i \in [n\ell]$. In this case, it holds the following multivariate Laurent polynomials:

$$L_1 \coloneqq \{1, X_\alpha, X_{t^*}(X_\alpha + r^*)\}$$
$$L_2 \coloneqq \{X_\alpha^i\}_{i=0,\ldots,\ell-1}$$
$$L_3 \coloneqq \left\{1, \frac{1}{\prod_{i\in[j\ell+1,(j+1)\ell]}(X_\alpha + r_i)}\right\}_{j=0,\ldots,n-1}$$

The polynomial used to calculate $\mathsf{ct}^*[2]$ in $\mathsf{Game}_{\mathcal{A},b}^{\mathrm{BIBE},\mathsf{SM}}$ is X_{t^*} (for both $b = 0, 1$), and in order to prove the equivalence of the adversary's views in $\mathsf{Game}_{\mathcal{A}}^{\mathsf{Hyb},\mathsf{SM}}$ and $\mathsf{Game}_{\mathcal{A},b}^{\mathrm{BIBE},\mathsf{SM}}$, we argue that X_{t^*} is linearly independent of the polynomials in $L_1 \otimes L_2 \otimes L_3$. Towards contradiction, suppose this is not the case. Since X_{t^*} appears only as part of the monomials $X_{t^*}(X_\alpha + r^*)$, it must be that

$$X_{t^*} = X_{t^*}(X_\alpha + r^*)\sum_{j=0}^{n-1} f_j(X_\alpha)\frac{1}{\prod_{i\in[j\ell+1,(j+1)\ell]}(X_\alpha + r_i)},$$

where $f_j(.)$ are degree $\leq (\ell - 1)$ polynomials in X_α. In other words,

$$\frac{1}{(X_\alpha + r^*)} = \sum_{j=0}^{n-1} f_j(X_\alpha)\frac{1}{\prod_{i\in[j\ell+1,(j+1)\ell]}(X_\alpha + r_i)}.$$

Now, for this equality to be satisfied, r^* must be among the values $(r_i)_{i\in[n\ell]}$, as otherwise $f_j(X_\alpha)$ would have to be a rational polynomial. However, this is a contradiction as $r^* \notin (r_{j\ell+1}, \ldots, r_{(j+1)\ell})$ for any $j = 0, \ldots, n-1$. □

Lemma 14. *For all $\ell \in \mathbb{N}$ and unbounded $\mathcal{A}$ making at most q queries to the group oracle, hash oracle, and for key computation, for $b \in \{0, 1\}$;*

$$\left|\Pr[\mathsf{Game}_{\mathcal{A},b}^{\mathrm{BIBE},\mathsf{GGM}}(1^\lambda, 1^\ell) = 1] - \Pr[\mathsf{Game}_{\mathcal{A},b}^{\mathrm{BIBE},\mathsf{SM}}(1^\lambda, 1^\ell) = 1]\right| \leq \binom{q+\ell+3}{2}\frac{\ell}{p}.$$

Proof. Note that the challenger can create a perfect simulation of the game $\mathsf{Game}_{\mathcal{A},b}^{\mathrm{BIBE},\mathsf{GGM}}(1^\lambda, 1^\ell)$ by sampling values from $\mathbb{Z}_p$ uniformly at random at the end of the game $\mathsf{Game}_{\mathcal{A},b}^{\mathrm{BIBE},\mathsf{SM}}(1^\lambda, 1^\ell)$ for the involved indeterminates, unless the sampled values cause different polynomials to assume the same value. By the Schwartz-Zippel lemma [67,57], the probability of this event for two polynomials of degree (d_1, d_2) is at most $\max(d_1, d_2)/p$. Now, when the adversary makes q pre-decryption queries, there will be q Laurent polynomials from the pre-decryption queries, $\ell + 1$ polynomials from setup (including the '1' polynomials) and 2 polynomials from encryption, implying a total of $q + \ell + 3$ polynomials across all four groups. Since the largest degree across the polynomials is ℓ, by the union bound, the probability that different polynomials assume the same value can be upper-bounded by

$$\binom{q+\ell+3}{2}\frac{\ell}{p}.$$

□

Proof of Theorem 9. Since by Lemma 13,

$$\left|\Pr[\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},0}(1^\lambda,1^\ell)=1]-\Pr[\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},1}(1^\lambda,1^\ell)=1]\right|=0,$$

by Lemma 14,

$$\left|\Pr[\mathsf{Game}^{\mathrm{BIBE},\mathsf{GGM}}_{\mathcal{A},b}(1^\lambda,1^\ell)=1]-\Pr[\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},b}(1^\lambda,1^\ell)=1]\right|\leq\binom{q+\ell+3}{2}\frac{\ell}{p},$$

and

$$\binom{q+\ell+3}{2}\frac{\ell}{p}=\mathsf{negl}(\lambda),$$

it holds that

$$\begin{aligned}&\mathsf{Game}^{\mathrm{BIBE},\mathsf{GGM}}_{\mathcal{A},0}(1^\lambda,1^\ell)\approx_c\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},0}(1^\lambda,1^\ell)\\&\equiv\mathsf{Game}^{\mathrm{BIBE},\mathsf{SM}}_{\mathcal{A},1}(1^\lambda,1^\ell)\approx_c\mathsf{Game}^{\mathrm{BIBE},\mathsf{GGM}}_{\mathcal{A},1}(1^\lambda,1^\ell)\end{aligned}$$

□

10 Conclusions and open problems

We have presented three new batch IBE and threshold batch IBE systems without epochs, based respectively on key-policy ABE, lattices, and trilinear maps. Our novel lattice-based batch decryption scheme is plausibly post-quantum secure, and uses techniques from lattice-based identity-based encryption in the random oracle model [5,27] and the WWW24 preimage sampler [62]. It can be thresholdized via a two-round interactive protocol that combines Shamir secret sharing of the trapdoor with noise flooding and pairwise masks [36]. Although this enables keeping the parameter, key and ciphertext sizes of our scheme succinct in the number of parties, designing more efficient non-interactive thresholding techniques for lattice-based batch IBE remains an important problem.

Acknowledgments. We thank Gregory Neven for helpful discussions about this work. This work was partially supported by the Simons Foundation, the Stanford Center for Blockchain Research, and UBRI. The third author was further supported by a Research Hub Collaboration agreement between Stanford University and Input Output Global Inc.

References

1. Damiano Abram, Giulio Malavolta, and Lawrence Roy. Key-homomorphic computations for RAM: fully succinct randomised encodings and more. In *CRYPTO (3)*, Lecture Notes in Computer Science, pages 236–268. Springer, 2025.
2. Amit Agarwal, Kushal Babel, Sourav Das, Babak Poorebrahim Gilkalaye, Arup Mondal, Benny Pinkas, Peter Rindal, and Aayush Yadav. Weighted batched threshold encryption with applications to mempool privacy. 2025.
3. Amit Agarwal, Sourav Das, Babak Poorebrahim Gilkalaye, Peter Rindal, and Victor Shoup. BTX: Simple and efficient batch threshold encryption. 2026.
4. Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. 2024. To appear in CRYPTO 2025.
5. Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In *EUROCRYPT*, volume 6110 of *Lecture Notes in Computer Science*, pages 553–572. Springer, 2010.
6. Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In *ICALP*, LIPIcs, pages 8:1–8:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.
7. Martin R. Albrecht, Russell W. F. Lai, Oleksandra Lapiha, and Ivy K. Y. Woo. Partial lattice trapdoors: How to split lattice trapdoors, literally. *IACR Cryptol. ePrint Arch.*, page 367, 2025.
8. Shi Bai, Tancrède Lepoint, Adeline Roux-Langlois, Amin Sakzad, Damien Stehlé, and Ron Steinfeld. Improved security proofs in lattice-based cryptography: Using the rényi divergence rather than the statistical distance. *J. Cryptol.*, 31(2):610–640, 2018.
9. Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. 2022.
10. Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. 2022. In CRYPTO 2022.
11. Rikke Bendlin and Ivan Damgård. Threshold decryption and zero-knowledge proofs for lattice-based cryptosystems. In *TCC*, volume 5978 of *Lecture Notes in Computer Science*, pages 201–218. Springer, 2010.
12. Rikke Bendlin, Sara Krehbiel, and Chris Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In *ACNS*, volume 7954 of *Lecture Notes in Computer Science*, pages 218–236. Springer, 2013.
13. Dan Boneh and Xavier Boyen. Short signatures without random oracles. In *EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 56–73. Springer, 2004.
14. Dan Boneh and Xavier Boyen. Efficient selective identity-based encryption without random oracles. *J. Cryptol.*, 24(4):659–693, 2011.
15. Dan Boneh, Xavier Boyen, and Shai Halevi. Chosen ciphertext secure public key threshold encryption without random oracles. In *CT-RSA*, volume 3860 of *Lecture Notes in Computer Science*, pages 226–243. Springer, 2006.
16. Dan Boneh, Benedikt Bünz, Kartik Nayak, Lior Rotem, and Victor Shoup. Context-dependent threshold decryption and its applications. 2025.
17. Dan Boneh, Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. *SIAM J. Comput.*, 36(5):1301–1328, 2007.
18. Dan Boneh and Matthew K. Franklin. Identity-based encryption from the weil pairing. In *CRYPTO*, volume 2139 of *Lecture Notes in Computer Science*, pages 213–229. Springer, 2001.
19. Dan Boneh, Craig Gentry, Sergey Gorbunov, Shai Halevi, Valeria Nikolaenko, Gil Segev, Vinod Vaikuntanathan, and Dhinakaran Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit ABE and compact garbled circuits. In *EUROCRYPT*, volume 8441 of *Lecture Notes in Computer Science*, pages 533–556. Springer, 2014.
20. Dan Boneh, Craig Gentry, and Michael Hamburg. Space-efficient identity based encryption without pairings. In *FOCS*, pages 647–657. IEEE Computer Society, 2007.
21. Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic prfs and their applications. In *CRYPTO (1)*, volume 8042 of *Lecture Notes in Computer Science*, pages 410–428. Springer, 2013.
22. Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the weil pairing. *J. Cryptol.*, 17(4):297–319, 2004.
23. Dan Boneh, Rohit Nema, Arnab Roy, and Ertem Nusret Tas. Efficient batch threshold encryption using partial fraction techniques. 2026. In CRYPTO 2026.
24. Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2008.
25. Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. BEAST-MEV: batched threshold encryption with silent setup for MEV prevention. *IACR Cryptol. ePrint Arch.*, page 1419, 2025.

26. Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. BEAT-MEV: epochless approach to batched threshold encryption for MEV prevention. *IACR Cryptol. ePrint Arch.*, page 1533, 2024. To appear in Usenix Security 2025.
27. David Cash, Dennis Hofheinz, Eike Kiltz, and Chris Peikert. Bonsai trees, or how to delegate a lattice basis. *J. Cryptol.*, 25(4):601–639, 2012.
28. Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. *IACR Cryptol. ePrint Arch.*, page 44, 2025.
29. Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In *USENIX Security Symposium*. USENIX Association, 2024.
30. Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. *IACR Cryptol. ePrint Arch.*, page 1516, 2024. To appear in Usenix Security 2025.
31. Valerio Cini and Hoeteck Wee. ABE for circuits with poly (λ) -sized keys from LWE. In *FOCS*, pages 435–446. IEEE, 2023.
32. Valerio Cini and Hoeteck Wee. Unbounded ABE for circuits from lwe, revisited. In *ASIACRYPT (4)*, Lecture Notes in Computer Science, pages 238–267. Springer, 2024.
33. Clifford C. Cocks. An identity based encryption scheme based on quadratic residues. In *IMACC*, volume 2260 of *Lecture Notes in Computer Science*, pages 360–363. Springer, 2001.
34. Elizabeth Crites, Chelsea Komlo, and Mary Maller. How to prove schnorr assuming schnorr: Security of multi- and threshold signatures. 2021. In CRYPTO 2022.
35. Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *SP*, pages 910–927. IEEE, 2020.
36. Rafael del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In *CRYPTO*, 2024.
37. Yevgeniy Dodis, Leonid Reyzin, and Adam D. Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In *EUROCRYPT*, volume 3027 of *Lecture Notes in Computer Science*, pages 523–540. Springer, 2004.
38. Fangqi Dong, Zihan Hao, Ethan Mook, Hoeteck Wee, and Daniel Wichs. Laconic function evaluation and ABE for rams from (ring-)lwe. In *CRYPTO (3)*, Lecture Notes in Computer Science, pages 107–142. Springer, 2024.
39. Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. Mcfly: Verifiable encryption to the future made practical. In *FC (1)*, volume 13950 of *Lecture Notes in Computer Science*, pages 252–269. Springer, 2023.
40. Jesko Dujmovic, Rachit Garg, and Giulio Malavolta. Time-lock puzzles with efficient batch solving. In *EUROCRYPT (2)*, volume 14652 of *Lecture Notes in Computer Science*, pages 311–341. Springer, 2024.
41. Stefan Dziembowski, Sebastian Faust, and Jannik Luhn. Shutter network: Private transactions from threshold cryptography. Cryptology ePrint Archive, Paper 2024/1981, 2024.
42. Rex Fernando, Guru-Vamsi Policharla, Andrei Tonkikh, and Zhuolun Xiang. TrX: Encrypted mempools in high performance BFT protocols. 2025.
43. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *STOC*, pages 197–206. ACM, 2008.
44. Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. 2025.
45. Rishab Goyal and Vinod Vaikuntanathan. Locally verifiable signature and key aggregation. In *CRYPTO (2)*, volume 13508 of *Lecture Notes in Computer Science*, pages 761–791. Springer, 2022.
46. Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *CCS*, pages 89–98. ACM, 2006.
47. Kaijie Jiang, Hoeteck Wee, and Chenzhi Zhu. Oriole: Adaptively secure partially non-interactive threshold signatures from lattices. Cryptology ePrint Archive, Paper 2026/793, 2026.
48. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *ASIACRYPT*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer, 2010.
49. George Lu and Brent Waters. How to sample a discrete gaussian (and more) from a random oracle. In *TCC (2)*, volume 13748 of *Lecture Notes in Computer Science*, pages 653–682. Springer, 2022.
50. Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *EUROCRYPT*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718. Springer, 2012.

51. Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. Fairblock: Preventing blockchain front-running with minimal overheads. In *SecureComm*, volume 462 of *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pages 250–271. Springer, 2022.
52. Chris Peikert. Bonsai trees (or, arboriculture in lattice-based cryptography). *IACR Cryptol. ePrint Arch.*, page 359, 2009.
53. Guru-Vamsi Policharla. A simple batched threshold encryption scheme. 2026.
54. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, pages 84–93. ACM, 2005.
55. Michael Reiter and Ken Birman. How to securely replicate services. *ACM TOPLAS*, 16(3):986–1009, 1994.
56. Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *EUROCRYPT*, volume 3494 of *Lecture Notes in Computer Science*, pages 457–473. Springer, 2005.
57. Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, 1980.
58. Victor Shoup. Lower bounds for discrete logarithms and related problems. In *EUROCRYPT*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266. Springer, 1997.
59. Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. Constant-cost batched partial decryption in threshold encryption. *IACR Cryptol. ePrint Arch.*, page 762, 2024.
60. Hong Wang, Yuqing Zhang, and Dengguo Feng. Short threshold signature schemes without random oracles. In *INDOCRYPT 2005*, volume 3797 of *Lecture Notes in Computer Science*, pages 297–310. Springer, 2005.
61. Brent Waters. Efficient identity-based encryption without random oracles. In *EUROCRYPT*, volume 3494 of *Lecture Notes in Computer Science*, pages 114–127. Springer, 2005.
62. Brent Waters, Hoeteck Wee, and David J. Wu. New techniques for preimage sampling: Improved nizks and more from LWE. *IACR Cryptol. ePrint Arch.*, page 1401, 2024. Eurocrypt 2025.
63. Hoeteck Wee. Almost optimal KP and CP-ABE for circuits from succinct LWE. In *EUROCRYPT (3)*, Lecture Notes in Computer Science, pages 34–62. Springer, 2025.
64. Hoeteck Wee and David J. Wu. Succinct vector, polynomial, and functional commitments from lattices. In *EUROCRYPT (3)*, volume 14006 of *Lecture Notes in Computer Science*, pages 385–416. Springer, 2023.
65. Saisi Xiong, Yijian Zhang, and Jie Chen. Efficient batched IBE from lattices in the standard model. 2025.
66. Mark Zhandry. To label, or not to label (in generic groups). In *CRYPTO (3)*, volume 13509 of *Lecture Notes in Computer Science*, pages 66–96. Springer, 2022.
67. Richard Zippel. Probabilistic algorithms for sparse polynomials. In *EUROSAM*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.

A Extended Preliminaries

A.1 Pseudorandom Functions

Definition 21. *Let $\mathcal{X}$ and $\mathcal{Y}$ be finite sets. A keyed function $\mathsf{PRF} : \{0,1\}^\kappa \times \mathcal{X} \to \mathcal{Y}$ is a secure pseudorandom function (PRF) if for all PPT adversaries $\mathcal{A}$,*

$$\mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{A}}(1^\lambda) := \left|\Pr\left[\mathsf{Game}^{\mathsf{PRF}}_{\mathcal{A},0}(1^\lambda) = 1\right] - \Pr\left[\mathsf{Game}^{\mathsf{PRF}}_{\mathcal{A},1}(1^\lambda) = 1\right]\right| = \mathsf{negl}(\lambda)$$

The game $\mathsf{Game}^{\mathsf{PRF}}_{\mathcal{A},b}$, parameterized by a bit $b \in \{0,1\}$, is defined as follows: The adversary $\mathcal{A}$ may adaptively query pairs (ν, x), where ν is an arbitrary instance label and $x \in \mathcal{X}$. The challenger maintains two initially empty tables $\mathsf{K}[\cdot]$ and $\mathsf{R}[\cdot,\cdot]$.

- $b = 0$**:** On query (ν, x), if $\mathsf{K}[\nu]$ is undefined, the challenger samples $\mathsf{K}[\nu] \leftarrow \{0,1\}^\kappa$ and returns $\mathsf{PRF}(\mathsf{K}[\nu], x) \in \mathcal{Y}$.
- $b = 1$**:** On query (ν, x), if $\mathsf{R}[\nu, x]$ is undefined, the challenger samples $\mathsf{R}[\nu, x] \leftarrow \mathcal{Y}$ and returns $\mathsf{R}[\nu, x]$. Thus repeated queries to the same pair (ν, x) are answered consistently.

At the end of the game, $\mathcal{A}$ outputs a bit b'.

A.2 Extended Lattice Background

For a full-rank matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the n-dimensional lattice generated by $\mathbf{A}$ is given by

$$\Lambda(\mathbf{A}) \coloneqq \{\mathbf{A}\mathbf{v} \colon \mathbf{v} \in \mathbb{Z}^n\}.$$

When $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ for some integers n, m, q, we define the following q-ary lattices:

$$\begin{aligned}
\Lambda(\mathbf{A}) &\coloneqq \{\mathbf{y} \in \mathbb{Z}_q^m \colon \mathbf{y} = \mathbf{A}^\top \mathbf{v} \bmod q, \mathbf{v} \in \mathbb{Z}_q^n\} \\
\Lambda^\perp(\mathbf{A}) &\coloneqq \{\mathbf{v} \in \mathbb{Z}_q^m \colon \mathbf{A}\mathbf{v} = \mathbf{0} \bmod q\} \\
\Lambda^{\mathbf{u}}(\mathbf{A}) &\coloneqq \{\mathbf{v} \in \mathbb{Z}_q^m \colon \mathbf{A}\mathbf{v} = \mathbf{u} \bmod q\}
\end{aligned}$$

We denote by $\lambda_1^\infty(\mathbf{A}) \coloneqq \min_{\mathbf{v} \neq 0} \|\mathbf{A}^\top \mathbf{v}\|_\infty$, the ℓ_∞-norm of the shortest non-zero vector in the lattice defined by the matrix $\mathbf{A}$.

Lemma 15 (Lemma 15 [5]). *Let $\mathbf{R} \leftarrow \{-1, 1\}^{k \times m}$. Then, there exists a universal constant C such that*

$$\Pr\left[\|\mathbf{R}\| > C\sqrt{k+m}\right] < e^{-(k+m)}.$$

A.2.1 Discrete Gaussians Over Lattices. For a Gaussian width parameter $\sigma \in \mathbb{R}^+$ and dimension $n \in \mathbb{N}$, let $\rho_\sigma : \mathbb{R}^n \to \mathbb{R}$ be the Gaussian function:

$$\rho_\sigma(\mathbf{x}) \coloneqq \exp\left(-\pi \|\mathbf{x}\|_2^2 / \sigma^2\right).$$

For a coset $\mathbf{c} + \Lambda$, the Gaussian probability mass is defined as

$$\rho_\sigma(\mathbf{c} + \Lambda) \coloneqq \sum_{\mathbf{x} \in \Lambda} \rho_\sigma(\mathbf{c} + \mathbf{x}).$$

We denote the discrete Gaussian distribution over $\mathbb{Z}^n$ with parameter σ by

$$\mathcal{D}_{\mathbb{Z}^n, \sigma}(\mathbf{x}) \coloneqq \frac{\rho_\sigma(\mathbf{x})}{\rho_\sigma(\mathbb{Z}^n)}.$$

For any discrete set $\Lambda \subseteq \mathbb{Z}^m$ and center $\mathbf{v} \in \mathbb{Z}^m$, define $\mathcal{D}_{\Lambda, \sigma, \mathbf{v}}(\mathbf{x}) \coloneqq \rho_\sigma(\mathbf{x} - \mathbf{v}) / \rho_\sigma(\Lambda - \mathbf{v})$ for $\mathbf{x} \in \Lambda$, and 0 otherwise, where $\rho_\sigma(\Lambda - \mathbf{v}) \coloneqq \sum_{\mathbf{y} \in \Lambda} \rho_\sigma(\mathbf{y} - \mathbf{v})$.

For a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a vector $\mathbf{u} \in \mathbb{Z}_q^n$, we define $\mathbf{A}_\sigma^{-1}(\mathbf{u})$ as the random variable $\mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}^m, \sigma}$ conditioned on $\mathbf{A}\mathbf{x} = \mathbf{u} \bmod q$. This function extends naturally to matrices by applying $\mathbf{A}_\sigma^{-1}$ to each column individually.

Lemma 16 (From [52], Adapted from Lemma 8 [5]). *Let $q \geq 2$ and $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ be a matrix with $m > n$. Then for $\mathbf{u} \in \mathbb{Z}_q^n$, it holds that*

$$\Pr\left[\|\mathbf{x}\| > \sqrt{m}\sigma \ : \ \mathbf{x} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{u})\right] \leq \mathsf{negl}(n).$$

Lemma 17 (Corollary 5.4 [43]). *Let n be a positive integer, q be a prime, and $m \geq 2n \log(q)$. Then, for all but a $\mathsf{negl}(n)$ fraction of matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, and for any $\sigma \geq \omega(\sqrt{\log(m)})$, the distribution of $\mathbf{A}\mathbf{e} \bmod q$, where $\mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z}_q^m, \sigma}$ is statistically close to uniform over $\mathbb{Z}_q^n$.*

Lemma 18 (Discrete Gaussian Preimages, variant of Lemma 3.6 [62]). *Let $n, m, \overline{m}, q, t$ be lattice parameters such that $m, \overline{m} > n$. For any $\ell, k = \mathsf{poly}(n, \log(q))$, consider matrices $\mathbf{A}_1, \ldots, \mathbf{A}_\ell \in \mathbb{Z}_q^{n \times m}$, $\tilde{\mathbf{A}}_1, \ldots, \tilde{\mathbf{A}}_\ell \in \mathbb{Z}_q^{n \times \overline{m}}$, $\mathbf{B}_1, \ldots, \mathbf{B}_\ell \in \mathbb{Z}_q^{n \times k}$, and a target vector $\mathbf{t} \in \mathbb{Z}_q^{n\ell}$. Here, both $\mathbf{A}_i$ and $\tilde{\mathbf{A}}_i$ are full rank, and $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}}_i) \geq \sigma_0$ for all $i \in [\ell]$. Define the matrix:*

$$\mathbf{A} \coloneqq \begin{bmatrix} \mathbf{A}_1 & & & \mathbf{B}_1 & \tilde{\mathbf{A}}_1 & & \\ & \mathbf{A}_2 & & \mathbf{B}_2 & & \tilde{\mathbf{A}}_2 & \\ & & \ldots & \ldots & & & \ldots \\ & & & \mathbf{A}_\ell & \mathbf{B}_\ell & & & \tilde{\mathbf{A}}_\ell \end{bmatrix} \quad \text{and} \quad \mathbf{t} \coloneqq \begin{bmatrix} \mathbf{t}_1 \\ \mathbf{t}_2 \\ \ldots \\ \mathbf{t}_\ell \end{bmatrix}$$

Then, for all width parameters $\sigma_1 \geq (q/\sigma_0)\log(2\ell m)$, the statistical distance between the following distributions is negligible in m:

$$\{\mathbf{v} : \mathbf{v} \leftarrow \mathbf{A}_{\sigma_1}^{-1}(\mathbf{t})\} \text{ and } \left\{ \mathrm{col}(\mathbf{v}_1^1, \ldots, \mathbf{v}_\ell^1, \hat{\mathbf{c}}, \mathbf{v}_1^2, \ldots, \mathbf{v}_\ell^2) : \substack{\hat{\mathbf{c}} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma_1}^k \\ \mathrm{col}(\mathbf{v}_i^1, \mathbf{v}_i^2) \leftarrow [\mathbf{A}_i | \tilde{\mathbf{A}}_i]_{\sigma_1}^{-1}(\mathbf{t}_i - \mathbf{B}_i \hat{\mathbf{c}})} \right\}$$

When $\lambda_1^\infty(\mathbf{A}_i), \lambda_1^\infty(\tilde{\mathbf{A}}_i) \geq \sigma_0$, and $\mathbf{A}_i$ and $\tilde{\mathbf{A}}_i$ are full rank for each $i \in [\ell]$, it holds that $\lambda_1^\infty\left(\mathrm{diag}([\mathbf{A}_1|\tilde{\mathbf{A}}_1], \ldots, [\mathbf{A}_\ell|\tilde{\mathbf{A}}_\ell])\right) \geq \sigma_0$, and $\mathrm{diag}([\mathbf{A}_1|\tilde{\mathbf{A}}_1], \ldots, [\mathbf{A}_\ell|\tilde{\mathbf{A}}_\ell])$ is full rank by [64, Lemma 2.3]. Then, Lemma 18 follows from [64, Lemma 2.10].

Lemma 19 (Lemma 5.1 [43]). *Let $m \geq 2n\log(q)$. Then, for all but at most a q^{-n} fraction of the matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, the subset-sums of the columns of $\mathbf{A}$ generate $\mathbb{Z}_q^n$;* i.e., *for every vector $\mathbf{u} \in \mathbb{Z}_q^n$, there exists an $\mathbf{e} \in \{0,1\}^m$ such that $\mathbf{A}\mathbf{e} = \mathbf{u} \bmod q$.*

Lemma 20 (Adapted from Lemma 5.3 [43]). *Let n be a positive integer, q be a prime, and $m \geq 2n\log(q)$. Then, for all but at most a q^{-n} fraction of the matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, it holds that $\lambda_1^\infty(\mathbf{A}) \geq \frac{q}{4}$.*

A.2.2 The Gadget Matrix and Trapdoors.

For $n, q \in \mathbb{N}$, the gadget matrix $\mathbf{G}_n$ is defined as follows [50]:

$$\mathbf{G}_n \coloneqq \mathbf{I}_n \otimes \mathbf{g}^\top \in \mathbb{Z}_q^{n \times \tilde{m}},$$

where $\mathbf{g}^\top \coloneqq [1, 2, \ldots, 2^{\lfloor \log(q) \rfloor}]$ and $\tilde{m} \coloneqq n(\lfloor \log(q) \rfloor + 1)$.

Lemma 21 (Adapted from Theorem 4.1 [50]). *For any integers $q \geq 2$ and $n \geq 1$, the lattice $\Lambda^\perp(\mathbf{G})$, $\mathbf{G} \in \mathbb{Z}_q^{n \times \tilde{m}}$, has a known basis $\mathbf{S} \in \mathbb{Z}^{\tilde{m} \times \tilde{m}}$ with $\|\mathbf{S}\|_{\mathsf{GS}} \leq \sqrt{5}$ and $\|\mathbf{S}\| \leq \max\{\sqrt{5}, \sqrt{\lceil \log(q) \rceil}\}$, and preimage sampling from $\mathbf{G}_\sigma^{-1}$ with Gaussian parameter $\sigma \geq \|\mathbf{S}\|_{\mathsf{GS}} \cdot \omega(\sqrt{\log(n)})$ can be performed in time $O(n \cdot \log^c(n))$ for some constant c.*

Lemma 22 (From [43]). *Let n be a positive integer and q be a prime. Then, for any $\sigma \geq \omega(\sqrt{\log(\tilde{m})})$, the distribution of $\mathbf{u} \leftarrow \mathbf{G}_\sigma^{-1}(\mathbf{v})$, where $\mathbf{v} \leftarrow \mathbb{Z}_q^n$, is statistically close to $\mathcal{D}_{\mathbb{Z}_q,\sigma}^{\tilde{m}}$.*

Lemma 22 follows from the proof of [43, Corollary 5.4], along with [43, Lemma 3.1] and [50, Theorem 4.1]. Here, [50, Theorem 4.1] states that $\Lambda^\perp(\mathbf{G})$ has a known basis $\mathbf{S} \in \mathbb{Z}_q^{\tilde{m} \times \tilde{m}}$ such that $\|\mathbf{S}\|_{\mathsf{GS}} \leq \sqrt{5}$. On the other hand, by [43, Lemma 3.1], the smoothing parameter η_ϵ for $\Lambda^\perp(\mathbf{G})$ is bounded by $\sqrt{5} \cdot \omega(\sqrt{\log(\tilde{m})})$. This implies that for $\sigma \geq \omega(\sqrt{\log(\tilde{m})})$, it holds that $\sigma \geq \eta_\epsilon(\Lambda^\perp(\mathbf{G}))$. Finally, as the columns of $\mathbf{G}$ generate $\mathbb{Z}_q^n$, by the proof of [43, Corollary 5.4], the distribution of $\mathbf{u} \leftarrow \mathbf{G}_\sigma^{-1}(\mathbf{v})$, where $\mathbf{v} \leftarrow \mathbb{Z}_q^n$, is statistically close to $\mathcal{D}_{\mathbb{Z}_q,\sigma}^{\tilde{m}}$.

Lemma 23 (Adapted from [50], Theorem 3.9 [62]). *Let $n, m, \tilde{m}, q$ be lattice parameters with $\tilde{m}$ as defined above. Then, there exist efficient algorithms* $(\mathsf{TrapGen}, \mathsf{SamplePre})$ *such that;*

- $\mathsf{TrapGen}(1^n, q, m) \to (\mathbf{A}, \mathbf{T_A})$: *Given a lattice dimension n, modulus q and number of samples m,* $\mathsf{TrapGen}$ *outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a gadget trapdoor $\mathbf{T_A} \in \mathbb{Z}_q^{m \times \tilde{m}}$.*
- $\mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma) \to \mathbf{v}$: *Given a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a trapdoor $\mathbf{T_A} \in \mathbb{Z}_q^{m \times \tilde{m}}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$ and a Gaussian width parameter σ,* $\mathsf{SamplePre}$ *outputs a vector $\mathbf{v} \in \mathbb{Z}_q^m$.*

For all $m \geq O(n \log(q))$, the algorithms satisfy the following:

- ***Trapdoor distribution:*** *Given $(\mathbf{A}, \mathbf{T_A}) \leftarrow \mathsf{TrapGen}(1^n, q, m)$ and $\mathbf{A}' \leftarrow \mathbb{Z}_q^{n \times m}$, it holds that $\Delta(\mathbf{A}, \mathbf{A}') \leq 2^{-n}$. Moreover, $\mathbf{AT_A} = \mathbf{G}$ and $\|\mathbf{T_A}\|_\infty = 1$.*
- ***Preimage sampling:*** *For all matrices $\mathbf{T_A} \in \mathbb{Z}_q^{m \times \tilde{m}}$, parameters $\sigma > 0$, and target vectors $\mathbf{u} \in \mathbb{Z}_q^n$ in the column span of $\mathbf{A}$, $\mathbf{v} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma)$ satisfies $\mathbf{Av} = \mathbf{u}$.*
- ***Preimage distribution:*** *Suppose $\mathbf{T_A}$ is a gadget trapdoor for $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ (*i.e.*, $\mathbf{AT_A} = \mathbf{G}$). Then, for all $\sigma \geq \sqrt{m\tilde{m}} \|\mathbf{T_A}\|_\infty \cdot \omega(\sqrt{\log(n)})$ and target vectors $\mathbf{u} \in \mathbb{Z}_q^n$, the statistical distance between the following distributions is at most 2^{-n}: $\{\mathbf{v} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma)\}$ and $\{\mathbf{v} \leftarrow \mathbf{A}_\sigma^{-1}(\mathbf{u})\}$.*

In this work, we define trapdoors such that $\mathbf{AT_A} = \mathbf{G}$, as opposed to the version in [50], where $\mathbf{A} \begin{bmatrix} \mathbf{T_A} \\ \mathbf{I} \end{bmatrix} = \mathbf{G}$.

A.2.3 Sampling Algorithm. $\mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T_A}, \mathbf{u}, \sigma)$, defined in [5, §4.1], takes a short basis $\mathbf{T_A} \in \mathbb{Z}_q^{m \times \tilde{m}}$ for $\Lambda^\perp(\mathbf{A})$, a matrix $\mathbf{M}_1 \in \mathbb{Z}_q^{n \times m_1}$ and a Gaussian width parameter σ as input, and outputs a short vector $\mathbf{v} \in \Lambda^{\mathbf{u}}(\mathbf{F})$ from a distribution statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$, where $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$. For this purpose, the algorithm uses Peikert's basis extension method [52] to construct a basis $\mathbf{T_F}$ of the lattice $\Lambda^\perp(\mathbf{F})$ such that $\|\mathbf{T_F}\|_{\mathsf{GS}} = \|\mathbf{T_A}\|_{\mathsf{GS}}$. It then calls the algorithm $\mathsf{SamplePre}(\mathbf{F}, \mathbf{T_F}, \mathbf{u}, \sigma)$ to obtain a vector statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$.

Lemma 24 (Lemma 17 [5]). *Given $q > 2$, $m > n$, $\sigma > \|\mathbf{T_A}\|_{\mathsf{GS}} \cdot \omega(\log(m + m_1))$, the output of $\mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T_A}, \mathbf{u}, \sigma)$ is distributed statistically close to $\mathbf{F}_\sigma^{-1}(\mathbf{u})$, where $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$.*

Although $\mathsf{SampleLeft}$ takes as input a basis $\mathbf{T_A}$ for $\Lambda^\perp(\mathbf{A})$, it is possible to invoke the algorithm with a gadget trapdoor $\hat{\mathbf{T}}_\mathbf{A}$ such that $\mathbf{A}\hat{\mathbf{T}}_\mathbf{A} = \mathbf{G}$ and obtain the corresponding basis using [28, Lemma 7.4]. Then, Lemma 24 holds when the algorithm is invoked with the gadget trapdoor as well, except that now, by the proof of [28, Lemma 7.8, eq. 7.1], the lower bound on σ becomes $2m^{3/2}\|\hat{\mathbf{T}}_\mathbf{A}\|_\infty \cdot \omega(\log(m + m_1))$.

A.2.4 Leftover Hash Lemma. The following lemma follows from a generalization of the leftover hash lemma [37].

Lemma 25 (Lemma 13 [5], from [37]). *Suppose that $m \geq (n+1)\log(q) + \omega(\log(n))$, and $q > 2$ is prime. Let $\mathbf{R}$ be a uniformly-random matrix in $\{1, -1\}^{m \times k}$, where $k = k(n)$ is a polynomial in n. Let $\mathbf{A}$ and $\mathbf{B}$ be uniformly-random matrices in $\mathbb{Z}_q^{n \times m}$ and $\mathbb{Z}_q^{n \times k}$, respectively. Then, for all vectors $\mathbf{w} \in \mathbb{Z}_q^m$, the distribution $(\mathbf{A}, \mathbf{AR}, \mathbf{R}^\top \mathbf{w})$ is statistically close to the distribution $(\mathbf{A}, \mathbf{B}, \mathbf{R}^\top \mathbf{w})$.*

Lemma 26 (Lemma 12 [5]). *Let $\mathbf{e} \in \mathbb{Z}^m$ be some vector, and let $\mathbf{y} \in [0, q-1]^m$ be an integer vector such that $\mathbf{y} \leftarrow \Psi_\alpha^m$. Then the quantity $|\mathbf{e}^\top \mathbf{y}|$, treated as an integer, satisfies the following except with probability $\mathsf{negl}(m)$:*

$$|\mathbf{e}^\top \mathbf{y}| \leq \|\mathbf{e}\| q\alpha \cdot \omega(\log(m)) + \|\mathbf{e}\| \frac{\sqrt{m}}{2}$$

Definition 22 (Inhomogeneous Short Integer Solutions). *Let $B > 0$. The $\mathsf{ISIS}_{q,n,m,B}$ problem instance asks to find a vector $\mathbf{z} \in \mathbb{Z}^m$ such that $\mathbf{C}\mathbf{z} = \mathbf{y} \bmod q$ and $\|\mathbf{z}\|_2 \leq B$ given some $(\mathbf{C}, \mathbf{y}) \leftarrow \mathbb{Z}_q^{n \times m} \times \mathbb{Z}_q^n$. For a PPT adversary $\mathcal{B}$, we define $\mathsf{Adv}_{\mathcal{B}}^{\mathsf{ISIS}}(1^\lambda)$ as the probability that $\mathcal{B}$ solves this problem. We say that $\mathsf{ISIS}_{q,n,m,B}$ is hard if $\mathsf{Adv}_{\mathcal{B}}^{\mathsf{ISIS}}(1^\lambda) = \mathsf{negl}(\lambda)$ for every PPT adversary $\mathcal{B}$.*

A.3 Rényi Divergence and Exposed-Syndrome Smudging

Definition 23 (Rényi divergence, Def. 2.26 [6]). *Let P and Q be discrete probability distributions with $\mathrm{Supp}(P) \subseteq \mathrm{Supp}(Q)$. Then for $a \in (1, \infty)$, the Rényi divergence of order a is defined as*

$$R_a(P\|Q) \coloneqq \left(\sum_{x \in \mathrm{Supp}(P)} \frac{P(x)^a}{Q(x)^{a-1}} \right)^{1/(a-1)}$$

Moreover,

$$R_1(P\|Q) \coloneqq \exp\left(\sum_{x \in \mathrm{Supp}(P)} P(x) \log\left(\frac{P(x)}{Q(x)} \right) \right) \qquad R_\infty(P\|Q) \coloneqq \max_{x \in \mathrm{Supp}(P)} \frac{P(x)}{Q(x)}$$

Lemma 27. *Let P and Q denote discrete probability distributions with $\mathrm{Supp}(P) \subseteq \mathrm{Supp}(Q)$. Then,*

1. *For every $a \in (1, \infty)$, $\Delta(P, Q) \leq \sqrt{\ln R_a(P\|Q)/2}$, where $\Delta(\cdot, \cdot)$ denotes statistical distance (Pinsker's inequality).*
2. *For every event $E \subseteq \mathrm{Supp}(Q)$ and $a \in (1, \infty)$, $Q(E) \geq P(E)^{a/(a-1)}/R_a(P\|Q)$ (Probability preservation).*
3. *Let P and Q denote distributions of a transcript $(X_1, \ldots, X_w)$. Suppose for every $j \in [w]$ and history hist_{j-1} in the common support of the first $j-1$ variables, $R_a(P[X_j \mid \mathsf{hist}_{j-1}]\|Q[X_j \mid \mathsf{hist}_{j-1}]) \leq \rho_j$, where $P[X_j \mid \mathsf{hist}_{j-1}]$ denotes the conditional distribution of X_j given $(X_1, \ldots, X_{j-1}) = \mathsf{hist}_{j-1}$. Then $R_a(P\|Q) \leq \prod_{j=1}^{Q} \rho_j$ (Multiplicativity property).*
4. *For any $a \in (1, \infty)$ and function $f(\cdot)$, $R_a(f(P)\|f(Q)) \leq R_a(P\|Q)$, where $f(S)$ denotes the distribution of $f(x)$, $x \leftarrow S$ (Data processing inequality).*

Lemma 27 is adapted from [6, Lemma 2.27].

Lemma 28 (Lemma 2.28 [6], [8]). *For any n dimensional lattice Λ, given centers $\boldsymbol{\mu}, \boldsymbol{\nu} \in \mathbb{R}^m$ and an order $a \in (1, \infty)$, if $\boldsymbol{\mu}, \boldsymbol{\nu} \in \Lambda$, let $\varepsilon = 0$; otherwise fix some $\varepsilon \in (0, 1)$ and suppose $s > \eta_\varepsilon(\Lambda^\perp(\mathbf{A}))$. Then*

$$R_a(\mathcal{D}_{L,s,\boldsymbol{\mu}}\|\mathcal{D}_{L,s,\boldsymbol{\nu}}) \in \left[\left(\frac{1-\varepsilon}{1+\varepsilon} \right)^{2/(a-1)}, \left(\frac{1+\varepsilon}{1-\varepsilon} \right)^{2/(a-1)} \right] \exp\left(a\pi \frac{\|\boldsymbol{\mu} - \boldsymbol{\nu}\|_2^2}{s^2} \right).$$

A.4 BEAT-MEV: Batch Threshold Encryption by Bormet, Faust, Othman, Qu

Bormet, Faust, Othman and Qu introduced the first efficient batch threshold encryption scheme without epochs [26]. At the core of BEAT-MEV is a *key-homomorphic puncturable PRF*, adapted from the PRF by Dujmovic et al. [40]. Each encryptor evaluates the PRF at a unique identity i using an ephemeral PRF key k_i, and masks its message by the PRF output: $\gamma_i = \mathsf{m}_i + \mathsf{PRF}(k_i, i)$. The ciphertext has three elements: the masked message γ_i, an ElGamal encryption of the key k_i (in the exponent), and a punctured key k_i^* that is punctured at the identity i. The pre-decryption key sbk, created for a batch of ℓ identities $\{1, \ldots, \ell\}$, is computed using the homomorphic property of ElGamal encryption and satisfies $\mathsf{PRF}(\mathsf{sbk}, i) = \sum_{j\in[\ell]} \mathsf{PRF}(k_j, i)$. Finally, decryption recovers m_i by calculating

$$\gamma_i - \mathsf{PRF}(\mathsf{sbk}, i) + \sum_{j\in[\ell], j\neq i} \mathsf{PuncturedPRF}(k_j^*, i),$$

since $\mathsf{PRF}(\mathsf{sbk}, i) = \sum_{j\in[\ell]} \mathsf{PRF}(k_j, i)$, and $\mathsf{PuncturedPRF}(k_j^*, i) = \mathsf{PRF}(k_j, i)$ for $j \neq i$.

The pre-decryption key is a single BLS12-381 $\mathbb{G}_2$ group element, and each ciphertext contains 3 group elements from $\mathbb{G}_1$ (*i.e.*, the punctured key and the ElGamal ciphertext) and one group element from $\mathbb{G}_T$ (*i.e.*, γ). However, the public keys for the key-homomorphic puncturable PRF have size quadratic in the domain of the PRF (this can be reduced to linear [40,25]). Although the domain can be restricted to ℓ integers, as the success of decryption requires the PRF keys to be punctured at unique integers, this would require advance coordination among the encryptors. To remove coordination, the authors propose splitting a batch into multiple sub-batches, each containing ciphertexts with distinct identities, and publishing pre-decryption keys for each sub-batch (*cf.*, [26, §5.3] for numerical analysis). Since the decryption algorithm requires $O(\ell^2)$ pairings in total, this technique also reduces the computational burden of decryption, however at the expense of larger pre-decryption keys and ciphertexts. Recall that using sub-batches also introduces a censorship vulnerability in the system (see Section 1).

Lattice-based BEAT-MEV. We briefly describe the version of BEAT-MEV that uses the puncturable key-homomorphic PRF construction of Boneh, Lewi, Montgomery, and Raghunathan (BLMR) [21, §5] and estimate its parameter sizes.

The public parameters of the BLMR PRF consist only of two random matrices $\mathbf{A}_0, \mathbf{A}_1 \in \mathbb{Z}_q^{m\times m}$, and its key is a vector $\mathsf{sk} \in \mathbb{Z}_q^m$. The PRF is evaluated at a point $x = x_1 \ldots, x_\ell \in \{0,1\}^k$ as follows:

$$\mathsf{PRF}(\mathsf{sk}, x) = \left\lfloor \prod_{i=1}^{k} \mathbf{A}_{x_i} \cdot \mathsf{sk} \right\rceil_p,$$

This ensures that $\mathsf{PRF}(\mathsf{sk}_1 + \mathsf{sk}_2, x) = \mathsf{PRF}(\mathsf{sk}_1, x) + \mathsf{PRF}(\mathsf{sk}_2, x) + e$, where the error term is $e \in \{0,1,2\}^m$. Here, the security parameter is n, and $q = O(\sqrt{n}2^{n^\epsilon})$, where $\epsilon \in [0,1]$ links the LWE problem to the problem of approximating the worst-case GapSVP to $\tilde{O}(n/2^{n^\epsilon})$ factors. Moreover, $m = \lceil n \log q \rceil$, $k = n^\epsilon / \log(n)$ and $p = 2^{n^\epsilon - \omega(\log(n))}$, which implies $m = O(kn\log(n))$ and $\log(p) = O(k\log(n))$.

One can view the evaluation of the BLMR PRF as calculating a single branch in a tree of evaluations, whose leaf at an index x is the PRF evaluated at index x. Let $u_{\perp, x_1, \ldots, x_{k'}}$, $k' \leq k$, denote the inner node at depth k' of the tree that is reached from the root by moving to the

left child at depth i, if $x_i = 0$, and to the right child, if $x_i = 1$, proceeding through the depths $i = 1, \ldots, k$. Then,

$$u_{\perp, x_1, \ldots, x_{k'}} \coloneqq \left\lfloor \prod_{i=1}^{k'} \mathbf{A}_{x_i} \cdot \mathsf{sk} \right\rfloor_p ,$$

Now, the BLMR PRF can be punctured at a point x by publishing as the punctured key, all the inner nodes that correspond to the siblings of the nodes on the path from the leaf x to the root. In other words, the punctured key consists of $u_{\perp,\overline{x_1}}, u_{\perp,x_1,\overline{x_2}}, \ldots, u_{\perp,x_1,\ldots,x_{k-1}\overline{x_k}}$. This enables calculating the PRF at any value $x' \neq x$ that first differs from x at some index j (*i.e.*, $x'_i = x_i$ for $i < j$, and $x'_j \neq x_j$) as follows:

$$\mathsf{PRF}(\mathsf{sk}, x) = \left\lfloor u_{\perp, x_1, \ldots, x_{j-1}, \overline{x_j}} \prod_{i=j+1}^{k} \mathbf{A}_{x_i} \right\rfloor_{p'} ,$$

where $p' = O(m^k p)$ is made larger to account for the increase in the noise when multiplying $\prod_{i=j+1}^{\ell} \mathbf{A}_{x_i}$ with the error term e associated with the inner node. This implies $\log(p') = k \log(m) + \log(p) = O(k(\log(k) + \log(n)))$.

Since BEAT-MEV requires the encryptors to publish their encrypted PRF keys through a homomorphic encryption scheme, we must also pick an appropriate homomorphic scheme for encrypting the BLMR PRF keys. We choose Regev encryption to preserve the structure of the PRF keys [54]. This also enables thresholdizing the new BEAT-MEV scheme by using a threshold version of Regev encryption [11].

Finally, we express the parameter sizes of the batch encryption scheme in terms of our notation for the security parameter, namely λ. As the matrices $\mathbf{A}_0$ and $\mathbf{A}_1$ are random, they can be generated from a small seed of size $O_\lambda(1)$ using a pseudo-random generator. Since Regev encryption has public keys of size $O_\lambda(1)$, the public key of the batch encryption scheme becomes $O_\lambda(1)$. However, the ciphertext now contains k vectors in $\mathbb{Z}_{p'}^m$:

$$\begin{aligned} |\mathsf{ct}| = km \log(p') &= O(k^3 n \log(n)(\log(k) + \log(n))) \\ &= O(k^3 \lambda(\log^2(\lambda) + \log(k)\log(\lambda))) = O_\lambda(k^3). \end{aligned}$$

Finally, the pre-decryption key is simply the sum of the k PRF keys:

$$|\mathsf{sbk}| = m \log q = O(k^2 n \log^2(n)) = O(k^2 \lambda \log^2(\lambda)) = O_\lambda(k^2).$$

B Shifted Multi-Preimage Trapdoor Sampler with *d*-bit Labels

In this section, we describe the properties and construction by Waters, Wee, and Wu [62, Construction 4.6] modified to work with d-bit labels.

B.1 Definition

Definition 24. *Let ℓ be the batch size and d, n, t, q, σ be functions of λ and ℓ. An (n, t, q, σ)-shifted multi-preimage trapdoor sampler consists of the following efficient algorithms:*

- $\mathsf{Gen}(1^\lambda, 1^\ell, 1^d) \to \mathsf{crs}$*: The randomized generation algorithm outputs a CRS.*

– $\mathsf{Expand}(1^\lambda, 1^\ell, 1^d, \mathsf{crs}, (u_i)_{i\in[\ell]}) \to (\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td})$*: The deterministic expand algorithm takes up to ℓ distinct d-bit vectors $(u_i)_{i\in[\ell]}$, and outputs matrices $\mathbf{A}_1, \ldots, \mathbf{A}_\ell \in \mathbb{Z}_q^{n\times t}$ and a matrix-trapdoor tuple* td.
– $\mathsf{SampleMultPre}(\mathsf{td}, \mathbf{t}_1, \ldots, \mathbf{t}_\ell) \to (\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c})$*: Given a matrix-trapdoor tuple* td *and target vectors $\mathbf{t}_1, \ldots, \mathbf{t}_\ell$, the randomized sampling algorithm outputs a shift $\mathbf{c} \in \mathbb{Z}_q^n$ and the preimages $\boldsymbol{\pi}_i \in \mathbb{Z}_q^t$.*

B.2 Properties

A shifted multi-preimage trapdoor sampler with d-bit labels must satisfy the following properties.

– **Correctness:** For all $\lambda, \ell, d \in \mathbb{N}$, all crs in the support of $\mathsf{Gen}(1^\lambda, 1^\ell, 1^d)$, all distinct ℓ d-bit vectors $(u_i)_{i\in[\ell]}$, and all $\mathbf{t}_1, \ldots, \mathbf{t}_\ell \in \mathbb{Z}_q^n$, given $(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td}) \coloneqq \mathsf{Expand}(1^\lambda, 1^\ell, 1^d, \mathsf{crs}, (u_i)_{i\in[\ell]})$, it holds that

$$\Pr\left[\mathbf{A}_i\boldsymbol{\pi}_i = \mathbf{t}_i + \mathbf{c}\ \forall i \in [\ell] \colon (\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c}) \leftarrow \mathsf{SampleMultPre}(\mathsf{td}, \mathbf{t}_1, \ldots, \mathbf{t}_\ell)\right] = 1.$$

– **Preimage distribution:** For all polynomials $\ell = \ell(\lambda)$ and $d = d(\lambda)$, for all given ℓ distinct d-bit vectors $(u_i)_{i\in[\ell]}$, except with negligible probability over $\mathsf{crs} \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell, 1^d)$, $(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td}) \coloneqq \mathsf{Expand}(1^\lambda, 1^\ell, 1^d, \mathsf{crs})$, the statistical distance between the following distributions is $\mathsf{negl}(\lambda)$:

$$\{(\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c}) \leftarrow \mathsf{SampleMultPre}(\mathsf{td}, \mathbf{t}_1, \ldots, \mathbf{t}_\ell)\} \text{ and}$$
$$\left\{(\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \mathbf{c}) \middle| \begin{array}{c} \mathbf{c}\leftarrow\mathbb{Z}_q^n, \\ \boldsymbol{\pi}_i\leftarrow(\mathbf{A}_i^{-1})_\sigma(\mathbf{t}_i+\mathbf{c})\ \forall i\in[\ell] \end{array}\right\}$$

– **Somewhere programmable:** There exists an efficient algorithm $\mathsf{GenProg}$ such that for all polynomials $\ell = \ell(\lambda)$, and ℓ distinct d-bit vectors $(u_i)_{i\in[\ell]}$, the following holds:
  • $\mathsf{GenProg}(1^\lambda, 1^\ell, 1^d, u_i, \mathbf{A}_i)$ takes a d-bit vector u_i and matrix $\mathbf{A}_i \in \mathbb{Z}_q^{n\times t}$, and outputs a common reference string $\tilde{\mathsf{crs}}$.
  • For all $i \in [\ell]$ and matrices $\forall \mathbf{A}_i \in \mathbb{Z}_q^{n\times t}$,

$$\Pr\left[\mathbf{A}_i = \tilde{\mathbf{A}}_i \middle| \begin{array}{c} \tilde{\mathsf{crs}}:=\mathsf{GenProg}(1^\lambda,1^\ell,1^d,u_i,\mathbf{A}_i), \\ (\tilde{\mathbf{A}}_1,\ldots,\tilde{\mathbf{A}}_\ell,\mathsf{td}):=\mathsf{Expand}(1^\lambda,1^\ell,1^d,\tilde{\mathsf{crs}},(u_i)_{i\in[\ell]}) \end{array}\right] = 1.$$

  • The statistical distance between the following distributions is $\mathsf{negl}(\lambda)$:

$$\{\mathsf{crs} \leftarrow \mathsf{Gen}(1^\lambda, 1^\ell, 1^d)\} \text{ and } \left\{\tilde{\mathsf{crs}} \middle| \begin{array}{c} \mathbf{A}_i\leftarrow\mathbb{Z}_q^{n\times t}, \\ \tilde{\mathsf{crs}}:=\mathsf{GenProg}(1^\lambda,1^\ell,1^d,u_i,\mathbf{A}_i) \end{array}\right\}$$

– **Local expansion:** There exists an efficient deterministic algorithm $\mathsf{ExpandLocal}$ such that:
  • $\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, u_i)$ takes a d-bit vector u_i and outputs $\mathbf{A}_i \in \mathbb{Z}_q^{n\times t}$.
  • For all $\lambda, \ell \in \mathbb{N}$, crs, and ℓ distinct d-bit vectors $(u_i)_{i\in[\ell]}$, given

$$(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td}) \coloneqq \mathsf{Expand}(1^\lambda, 1^\ell, 1^d, \mathsf{crs}, (u_i)_{i\in[\ell]}),$$

  it holds that for all $i \in [\ell]$,

$$\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, u_i) = \mathbf{A}_i$$

  Moreover, $\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, u_i)$ only reads $\mathsf{poly}(\lambda, d)$ bits of crs and runs in time $\mathsf{poly}(\lambda, d)$).

B.3 Construction

We next describe the shifted multi-preimage trapdoor sampler construction by Waters, Wee, and Wu [62, Construction 4.6] modified to work with d-bit labels, where $d \geq \lceil \log(\ell) \rceil$. Let $n = n(\lambda, \ell, d)$, $q = q(\lambda, \ell, d)$, $\sigma = \sigma(\lambda, \ell, d)$, $m = 3n\lceil \log(q) \rceil$ and $t = \tilde{m}(d+1)$. The vectors u_i are drawn from the set $\{0,1\}^d$.

- $\mathsf{Gen}(1^\lambda, 1^\ell, 1^d)$: This is a randomized algorithm that samples $[\mathbf{A} \mid \mathbf{B}] \leftarrow \mathbb{Z}_q^{n \times t}$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times \tilde{m}}$ and $\mathbf{B} \in \mathbb{Z}_q^{n \times \tilde{m}d}$, and outputs $\mathsf{crs} \coloneqq [\mathbf{A} \mid \mathbf{B}]$.
- $\mathsf{Expand}(1^\lambda, 1^\ell, 1^d, \mathsf{crs}, (u_i)_{i \in [\ell]})$: Given $\mathsf{crs} \coloneqq [\mathbf{A} \mid \mathbf{B}]$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times \tilde{m}}$ and $\mathbf{B} \in \mathbb{Z}_q^{n \times \tilde{m}d}$, the deterministic expand algorithm does the following:
  - For each $i \in [\ell]$, let $\mathbf{B}_i \coloneqq \mathbf{B} - u_i^\top \otimes \mathbf{G}$, and
  $$\mathbf{A}_i \coloneqq [\mathbf{A} \mid \mathbf{B} - u_i^\top \otimes \mathbf{G}] = [\mathbf{A} \mid \mathbf{B}_i] \in \mathbb{Z}_q^{n \times t}.$$
  - Let $\boldsymbol{\Pi} \in \{0,1\}^{(\ell t + \tilde{m}) \times (\ell t + \tilde{m})}$ be the permutation matrix such that:
  $$\mathbf{D}_\ell \coloneqq \begin{bmatrix} \mathbf{A}\ \mathbf{B}_1 & & & \mathbf{G} \\ & \mathbf{A}\ \mathbf{B}_2 & & \mathbf{G} \\ & & \cdots & \mathbf{G} \\ & & & \mathbf{A}\ \mathbf{B}_\ell\ \mathbf{G} \end{bmatrix} = \begin{bmatrix} \mathbf{A} & & \mathbf{B}_1 & & \mathbf{G} \\ & \cdots & & \cdots & \mathbf{G} \\ & & \cdots & & \mathbf{G} \\ & & \mathbf{A} & & \mathbf{B}_\ell\ \mathbf{G} \end{bmatrix} \boldsymbol{\Pi} \in \mathbb{Z}_q^{\ell n \times (\ell t + \tilde{m})}$$
  - It computes $\mathbf{T}' \leftarrow \mathsf{StructTrapGen}(\mathbf{B}, (u_i)_{i \in [\ell]}) \in \mathbb{Z}_q^{(\ell \tilde{m} d + \tilde{m}) \times \ell \tilde{m}}$, where $\mathbf{T}'$ is a trapdoor for the following matrix such that $\|\mathbf{T}'\|_\infty = 1$:
  $$\begin{bmatrix} \mathbf{B} - u_1 \otimes \mathbf{G} & & & \mathbf{G} \\ & \mathbf{B} - u_2 \otimes \mathbf{G} & & \mathbf{G} \\ & & \cdots & \mathbf{G} \\ & & & \mathbf{B} - u_\ell \otimes \mathbf{G}\ \mathbf{G} \end{bmatrix}$$
  (The algorithm $\mathsf{StructTrapGen}$ is described in Section B.4.) Finally, it defines the trapdoor $\mathbf{T} \in \mathbb{Z}_q^{(\ell t + \tilde{m}) \times \ell \tilde{m}}$ as:
  $$\mathbf{T} \coloneqq \boldsymbol{\Pi}^{-1} \begin{bmatrix} 0_{\ell \tilde{m} \times \ell \tilde{m}} \\ \mathbf{T}' \end{bmatrix}.$$
  It sets $\mathsf{td} \coloneqq (\mathbf{D}_\ell, \mathbf{T})$ and outputs $(\mathbf{A}_1, \ldots, \mathbf{A}_\ell, \mathsf{td})$.
- $\mathsf{SampleMultPre}(\mathsf{td}, \mathbf{t}_1, \ldots, \mathbf{t}_\ell)$: This is a randomized algorithm that given the trapdoor $\mathsf{td} \coloneqq (\mathbf{D}_\ell, \mathbf{T})$ and the target vectors $\mathbf{t}_1, \ldots, \mathbf{t}_\ell \in \mathbb{Z}_q^n$, defines the vector $\mathbf{t} \coloneqq \mathrm{col}(\mathbf{t}_1, \ldots, \mathbf{t}_\ell)$ and outputs $(\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, -\mathbf{G}\hat{\mathbf{c}})$, obtained using $\mathsf{SamplePre}$: $\mathrm{col}(\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_\ell, \hat{\mathbf{c}}) \leftarrow \mathsf{SamplePre}(\mathbf{D}_\ell, \mathbf{T}, \mathbf{t}, \sigma)$.
- $\mathsf{GenProg}(1^\lambda, 1^\ell, 1^d, u_i, \mathbf{A}_i)$: This is a deterministic algorithm that given the vector $u_i \in \{0,1\}^d$ and the matrix $\mathbf{A}_i \coloneqq [\mathbf{A} \mid \mathbf{B}]$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{B} \in \mathbb{Z}_q^{n \times md}$, outputs $\mathsf{crs} \coloneqq [\mathbf{A} \mid \mathbf{B} + u_i^\top \otimes \mathbf{G}]$.
- $\mathsf{ExpandLocal}(1^\lambda, \mathsf{crs}, u_i)$: This is a deterministic algorithm that given the $\mathsf{crs} \coloneqq [\mathbf{A} \mid \mathbf{B}]$ and the vector $u_i \in \{0,1\}^d$, outputs $\mathbf{A}_i \coloneqq [\mathbf{A} \mid \mathbf{B} - u_i^\top \otimes \mathbf{G}] \in \mathbb{Z}_q^{n \times t}$.

B.4 Sparse Indicator Function Evaluation

Recall that by [62, Theorem 3.10], the running time of Expand is bounded by $2^d \cdot \mathsf{poly}(n, m, d, \log(q))$. Inspecting the description of the Expand algorithm, we see that this loose bound is due to the $\mathsf{StructTrapGen}$ algorithm in [62, Lemma 4.1], which in turn inherits the bound from the procedures EvalF and EvalFX described below.

Theorem 10 (Theorem 3.10 [62]). *Let n, q be lattice parameters. Take any $m \geq n\lceil \log(q) \rceil$, and let $d = d(\lambda)$ be an input length. Then, there exist algorithms* $(\mathsf{EvalF}, \mathsf{EvalFX})$ *with the following syntax:*

- $\mathsf{EvalF}(\mathbf{A}, \delta_u) \to \mathbf{A}_u$*: on input a matrix* $\mathbf{A} \in \mathbb{Z}_q^{n \times dm}$ *and the indicator function* $\delta_u : \{0,1\}^d \to \{0,1\}$, $\delta_u(x) = 1 \iff x = u$, *where* $u \in \{0,1\}^d$*, the input-independent evaluation algorithm outputs a matrix* $\mathbf{A}_u \in \mathbb{Z}_q^{n \times m}$.
- $\mathsf{EvalFX}(\mathbf{A}, \delta_u, x) \to \mathbf{H}_{\mathbf{A},u,x}$*: on input a matrix* $\mathbf{A} \in \mathbb{Z}_q^{n \times dm}$*, an indicator function* δ_u*, where* $u \in \{0,1\}^d$*, and an input* $x \in \{0,1\}^d$*, the input-dependent evaluation algorithm outputs a matrix* $\mathbf{H}_{A,u,x} \in \mathbb{Z}_q^{dm \times m}$.

Moreover, for all $\lambda \in \mathbb{N}$, matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times dm}$, indicator functions δ_u, and inputs $x \in \{0,1\}^d$, the matrices $\mathbf{A}_u \leftarrow \mathsf{EvalF}(\mathbf{A}, \delta_u)$ and $\mathbf{H}_{A,u,x} \leftarrow \mathsf{EvalFX}(\mathbf{A}, \delta_u, x)$ satisfy $\mathbf{H}_{A,u,x} \in \{-1,0,1\}^{dm \times m}$ and

$$(\mathbf{A} - x^\top \otimes \mathbf{G}) \cdot \mathbf{H}_{A,u,x} = \mathbf{A}_u - \delta_u(x) \cdot \mathbf{G}$$

Note that [62, Theorem 3.10] is adapted from [38, Theorem 4.5], and the function being evaluated is always an indicator function, whereas the exponential running time of [38, Theorem 4.5] is due to its support for a larger class of functions. Therefore, we adapt the generic algorithms EvalF and EvalFX to indicator functions and show that EvalF and EvalFX can be invoked in polynomial time in λ.

EvalF *for indicator functions.* Given $\mathbf{B} = [\mathbf{B}_1 \mid \cdots \mid \mathbf{B}_d]$ and δ_u, where $u \in \{0,1\}^d$, set $\mathbf{F}^{(0)} \coloneqq \mathbf{G}$. For $r = 1, \ldots, d$, compute $\mathbf{R}^{(r-1)} \coloneqq \mathbf{G}^{-1}(\mathbf{F}^{(r-1)})$, where

$$\mathbf{F}^{(r)} = \begin{cases} \mathbf{B}_r \mathbf{R}^{(r-1)} & \text{if } u_r = 1 \\ \mathbf{F}^{(r-1)} - \mathbf{B}_r \mathbf{R}^{(r-1)} & \text{if } u_r = 0. \end{cases}$$

The output is $\mathbf{B}_u \coloneqq \mathbf{F}^{(d)} \in \mathbb{Z}_q^{n \times m}$.

EvalFX *for indicator functions.* Given $\mathbf{B}$, δ_u, and $x \in \{0,1\}^d$, first compute the matrices $\mathbf{R}^{(0)}, \ldots, \mathbf{R}^{(d-1)}$ as above. For $r \in [d]$, define

$$\mu_r(u, x) \coloneqq \prod_{s=r+1}^{d} \mathbf{1}[x_s = u_s],$$

where the empty product is 1, and define

$$\eta_r(u) \coloneqq \begin{cases} 1 & \text{if } u_r = 1, \\ -1 & \text{if } u_r = 0. \end{cases}$$

The r-th $m \times m$ block of the output matrix is

$$\mathbf{H}^{(r)}_{\mathbf{B},u,x} \coloneqq \mu_r(u, x) \eta_r(u) \mathbf{R}^{(r-1)}.$$

Finally, output

$$\mathbf{H}_{\mathbf{B},u,x} \coloneqq \mathrm{col}\left(\mathbf{H}^{(1)}_{\mathbf{B},u,x}, \ldots, \mathbf{H}^{(d)}_{\mathbf{B},u,x}\right) \in \{-1,0,1\}^{dm \times m}.$$

Lemma 29 (Sparse point-function evaluation). *For all* $\mathbf{B} \in \mathbb{Z}_q^{n\times dm}$*, all* $u, x \in \{0,1\}^d$*, let* $\mathbf{B}_u \leftarrow \mathsf{EvalF}(\mathbf{B}, \delta_u)$ *and* $\mathbf{H}_{\mathbf{B},u,x} \leftarrow \mathsf{EvalFX}(\mathbf{B}, \delta_u, x)$*. Then,* $\mathbf{H}_{\mathbf{B},u,x} \in \{-1,0,1\}^{dm\times m}$*, and*

$$(\mathbf{B} - x^\top \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{B},u,x} = \mathbf{B}_u - \delta_u(x)\mathbf{G}.$$

Moreover, EvalF *and* EvalFX *run in time* $\mathsf{poly}(d, n, m, \log(q))$*.*

Proof. The bound on $\mathbf{H}_{\mathbf{B},u,x}$ follows from the fact that each $\mathbf{R}^{(r)}$ is a binary matrix and each scalar $\mu_r(u,x)\eta_r(u)$ lies in $\{-1,0,1\}$.

For $r \in \{0,\dots,d\}$, write $\mathbf{B}_{\le r} \coloneqq [\mathbf{B}_1 \mid \cdots \mid \mathbf{B}_r]$, and let $u_{\le r}$ and $x_{\le r}$ denote the first r bits of u and x. We prove by induction on r that there exists a matrix $\mathbf{H}_{\le r} \in \{-1,0,1\}^{rm\times m}$ satisfying

$$(\mathbf{B}_{\le r} - x_{\le r}^\top \otimes \mathbf{G})\mathbf{H}_{\le r} = \mathbf{F}^{(r)} - \delta_{u_{\le r}}(x_{\le r})\mathbf{G}$$

For $r = 0$, the left-hand side is 0, while $\mathbf{F}^{(0)} = \mathbf{G}$, and the empty indicator function evaluates to 1, so the base case holds.

Now, for the inductive step, suppose the induction hypothesis holds for $r-1$, and set $\mathbf{R}^{(r-1)} = \mathbf{G}^{-1}(\mathbf{F}^{(r-1)})$, so that $\mathbf{G}\mathbf{R}^{(r-1)} = \mathbf{F}^{(r-1)}$. If $u_r = 1$, define

$$\mathbf{H}_{\le r} \coloneqq \begin{bmatrix} x_r\mathbf{H}_{\le r-1} \\ \mathbf{R}^{(r-1)} \end{bmatrix}.$$

Then

$$\begin{aligned}
&(\mathbf{B}_{\le r} - x_{\le r}^\top \otimes \mathbf{G})\mathbf{H}_{\le r} \\
&\quad = x_r\big(\mathbf{F}^{(r-1)} - \delta_{u_{\le r-1}}(x_{\le r-1})\mathbf{G}\big) + (\mathbf{B}_r - x_r\mathbf{G})\mathbf{R}^{(r-1)} \\
&\quad = \mathbf{B}_r\mathbf{R}^{(r-1)} - x_r\delta_{u_{\le r-1}}(x_{\le r-1})\mathbf{G} \\
&\quad = \mathbf{F}^{(r)} - \delta_{u_{\le r}}(x_{\le r})\mathbf{G},
\end{aligned}$$

where the last equality uses $u_r = 1$.

If $u_r = 0$, define

$$\mathbf{H}_{\le r} \coloneqq \begin{bmatrix} (1-x_r)\mathbf{H}_{\le r-1} \\ -\mathbf{R}^{(r-1)} \end{bmatrix}.$$

Then

$$\begin{aligned}
&(\mathbf{B}_{\le r} - x_{\le r}^\top \otimes \mathbf{G})\mathbf{H}_{\le r} \\
&\quad = (1-x_r)\big(\mathbf{F}^{(r-1)} - \delta_{u_{\le r-1}}(x_{\le r-1})\mathbf{G}\big) - (\mathbf{B}_r - x_r\mathbf{G})\mathbf{R}^{(r-1)} \\
&\quad = \mathbf{F}^{(r-1)} - \mathbf{B}_r\mathbf{R}^{(r-1)} - (1-x_r)\delta_{u_{\le r-1}}(x_{\le r-1})\mathbf{G} \\
&\quad = \mathbf{F}^{(r)} - \delta_{u_{\le r}}(x_{\le r})\mathbf{G},
\end{aligned}$$

where the last equality uses $u_r = 0$. This proves the identity for $r = d$. The closed-form block definition of $\mathbf{H}_{\mathbf{B},u,x}$ above is exactly the matrix obtained by unrolling this recursion.

For the running time, EvalF performs d gadget inversions and d products of an $n \times m$ matrix with an $m \times m$ matrix, plus additions over $\mathbb{Z}_q$. Thus, it runs in $d \cdot \mathsf{poly}(n, m, \log(q))$ time. Once the matrices $\mathbf{R}^{(0)}, \dots, \mathbf{R}^{(d-1)}$ are available, EvalFX writes d blocks of size $m \times m$, and therefore runs in $O(dm^2) + d \cdot \mathsf{poly}(n, m, \log(q))$ time. Both bounds are polynomial in $d, n, m, \log(q)$. □

Lemma 30 (Lemma 4.1 [62] adapted with EvalF and EvalFX above). *Let $n = n(\lambda)$, $m = m(\lambda)$, and $q = q(\lambda)$ be lattice parameters. Suppose $m \geq n\lceil \log(q) \rceil$. Then, for all $\ell \in \mathbb{N}$ and for $t = \tilde{m}(d+1)$, there exists an explicit polynomial-time algorithm* StructTrapGen *that takes as input $(\mathbf{B}, u_1, \ldots, u_\ell)$, where $\mathbf{B} \in \mathbb{Z}_q^{n \times t}$, $u_1, \ldots, u_\ell \in \{0,1\}^d$ are distinct vectors, and outputs a gadget trapdoor $\mathbf{T}' \in \mathbb{Z}_q^{(\ell\tilde{m}d+\tilde{m}) \times \ell\tilde{m}}$ with $\|\mathbf{T}'\|_\infty \leq 1$ for the matrix*

$$\mathbf{D}'_\ell = \begin{bmatrix} \mathbf{B} - u_1^\top \otimes \mathbf{G} & 0 & \cdots & 0 & \mathbf{G} \\ 0 & \mathbf{B} - u_2^\top \otimes \mathbf{G} & \cdots & 0 & \mathbf{G} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & \mathbf{B} - u_\ell^\top \otimes \mathbf{G} & \mathbf{G} \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times (\ell\tilde{m}d+\tilde{m})}.$$

Namely, the output $\mathbf{T}'$ satisfies $\mathbf{D}'_\ell \mathbf{T}' = \mathbf{I}_\ell \otimes \mathbf{G}$.

Proof. The proof of Lemma 30 follows from that of [62, Lemma 4.1]. By Lemma 29, for each $i, j \in [\ell]$

$$(\mathbf{B} - u_i^\top \otimes \mathbf{G})\mathbf{H}_{\mathbf{B},u_j,u_i} = \mathbf{B}_{u_j} - \delta_{u_j}(u_i)\mathbf{G}.$$

Since the u_i's are distinct, $\delta_{u_j}(u_i) = 1$ if and only if $i = j$. Therefore, as in the proof of [62, Lemma 4.1], the trapdoor matrix defined as

$$\mathbf{T}' = \begin{bmatrix} -\mathbf{H}_{\mathbf{B},u_1,u_1} & \cdots & -\mathbf{H}_{\mathbf{B},u_\ell,u_1} \\ \vdots & \ddots & \vdots \\ -\mathbf{H}_{\mathbf{B},u_1,u_\ell} & \cdots & -\mathbf{H}_{\mathbf{B},u_\ell,u_\ell} \\ \mathbf{G}^{-1}(\mathbf{B}_{u_1}) & \cdots & \mathbf{G}^{-1}(\mathbf{B}_{u_\ell}) \end{bmatrix}$$

satisfies $\mathbf{D}'_\ell \mathbf{T}' = \mathbf{I}_\ell \otimes \mathbf{G}$. Moreover, every entry of $\mathbf{T}'$ lies in $\{-1, 0, 1\}$, and hence $\|\mathbf{T}'\|_\infty \leq 1$.

For the runtime, pre-computing $\mathbf{B}_{u_i}$ for all $i \in [\ell]$ and finding their $\mathbb{G}^{-1}(\cdot)$ decompositions takes time $\ell d \cdot \mathsf{poly}(n, m, \log(q))$. Then, constructing $\mathbf{H}_{\mathbf{B},u_j,u_i}$ for all the pairs $i, j \in [\ell]$ costs

$$O\big(\ell d \cdot \mathsf{poly}(n, m, \log(q)) + \ell^2 d m^2\big),$$

giving a polynomial running time in $\ell, d, n, m, \log(q)$ for StructTrapGen. □

The results above imply that $\mathsf{Expand}(1^\lambda, 1^\ell, 1^d, \mathsf{crs}_{\mathsf{samp}}, (u_i)_{i\in[\ell]})$ runs in time polynomial in $\ell, d, n, m, \log(q)$.

C Explainable SampleLeft and the Proof of Theorem 1

We first present the definition of the explainable SamplePre sampler from [28, Definition 4.1]:

Definition 25 (Explainable SamplePre Sampler, Definition 4.1 of [28]). *Let n, m, q be lattice parameters. A $(\rho, \sigma_{\mathsf{loss}})$-explainable* SamplePre *sampler Π_{SP} with randomness length $\rho(\lambda, n, m, q)$ and Gaussian width loss $\sigma_{\mathsf{loss}}(\lambda, n, m, q)$ is a pair of efficient algorithms* (SamplePre, ExplainSP) *with the following syntax:*

- $\mathsf{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma; \mathsf{rb}) \to \mathbf{v}$*: Given a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a gadget trapdoor $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times \tilde{m}}$, a target vector $\mathbf{u} \in \mathbb{Z}_q^n$, a Gaussian width parameter σ, and randomness $\mathsf{rb} \in \{0,1\}^{\rho(\lambda,n,m,q)}$, the sampling algorithm deterministically outputs a vector $\mathbf{v} \in \mathbb{Z}_q^m$.*

– $\mathsf{ExplainSP}(1^\kappa, \mathbf{A}, \mathbf{T_A}, \mathbf{u}, \mathbf{v}, \sigma) \to \mathsf{rb}$*: Given a precision parameter* κ*, a matrix* $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$*, a gadget trapdoor* $\mathbf{T_A} \in \mathbb{Z}_q^{m\times\tilde{m}}$*, a target vector* $\mathbf{u} \in \mathbb{Z}_q^n$*, a preimage* $\mathbf{v} \in \mathbb{Z}_q^m$*, and a Gaussian width parameter* σ*, the explain algorithm outputs a bit string* $\mathsf{rb} \in \{0,1\}^{\rho(\lambda,n,m,q)}$*.*

We require the following two properties to hold except with negligible probability for all $\lambda \in \mathbb{N}$*, matrices* $\mathbf{A} \in \mathbb{Z}_q^{n\times m}, \mathbf{T_A} \in \mathbb{Z}_q^{m\times\tilde{m}}$ *such that* $\mathbf{AT_A} = \mathbf{G}$*, target vectors* $\mathbf{u} \in \mathbb{Z}_q^n$*, where* $\|\mathbf{u}\|_\infty \le 2^\lambda$*, and Gaussian width parameters* σ*, where* $\|\mathbf{T_A}\|_\infty \cdot \sigma_{\mathsf{loss}}(\lambda,n,m,q) \le \sigma \le 2^\lambda$*:*

– ***Correctness:*** *The following distributions are statistically close:*

$$\{\mathbf{v} \coloneqq \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma; \mathsf{rb});\ \mathsf{rb} \leftarrow \{0,1\}^\rho\} \quad \textit{and} \quad \left\{\mathbf{v} \leftarrow \mathbf{F}_\sigma^{-1}(\mathbf{v})\right\},$$

Moreover, for all $\mathbf{v}$ *in the support of* $\mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma; \mathsf{rb})$*, we have* $\mathbf{Av} = \mathbf{u}$*.*

– ***Explainability:*** *For all precision parameters* $\kappa \in \mathbb{N}$*, the statistical distance between the following distributions is bounded by* $1/\kappa + \mathsf{negl}(\lambda)$*:*

- $\mathsf{DSamplePre}$*: Sample* $\mathsf{rb} \leftarrow \{0,1\}^\rho$*, and set* $\mathbf{v} \coloneqq \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma; \mathsf{rb})$*. Output* $(\mathbf{v}, \mathsf{rb})$*.*
- $\mathsf{DExplain}$*: Sample* $\mathsf{rb}' \leftarrow \{0,1\}^\rho$*, and set* $\mathbf{v} \coloneqq \mathsf{SamplePre}(\mathbf{A}, \mathbf{T_A}, \mathbf{u}, \sigma; \mathsf{rb}')$*. Find* $\mathsf{rb} \coloneqq \mathsf{ExplainSP}(1^\kappa, \mathbf{A}, \mathbf{T_A}, \mathbf{u}, \mathbf{v}, \sigma)$*, and output* $(\mathbf{v}, \mathsf{rb})$*.*

Theorem 11 (Theorem 4.2 of [28]). *There exists a* $(\rho, \sigma_{\mathsf{loss}})$*-explainable* $\mathsf{SamplePre}$ *sampler for*

$$\rho = \mathsf{poly}(\lambda, n, m, \log q)$$

and

$$\sigma_{\mathsf{loss}}(\lambda, n, m, q) = 18m^{3/2} \log(m\lambda) \log(\log(q)).$$

Let $\Pi_{\mathsf{SP}} = (\mathsf{SamplePre}, \mathsf{ExplainSP})$ denote the $(\rho, \sigma_{\mathsf{loss}})$-explainable $\mathsf{SamplePre}$ sampler in [28, Construction 7.6], which satisfies Theorem 11.

Our explainable $\mathsf{SampleLeft}$ construction also relies on the following lemma:

Lemma 31 (Lemma 7.4 of [28], From [50]). *Let* n, q *be lattice parameters and* $\tilde{m} = n\lceil\log(q)\rceil$*. Take any* $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$ *and* $\mathbf{T_A} \in \mathbb{Z}_q^{m\times\tilde{m}}$ *such that* $\mathbf{AT_A} = \mathbf{G}$*. Let*

$$\mathbf{T'_A} = \left[\mathbf{I}_m - \mathbf{T_A}\mathbf{G}^{-1}\mathbf{A} \mid \mathbf{T_A}\mathbf{S}\right] \in \mathbb{Z}_q^{m\times(m+\tilde{m})},$$

where $\mathbf{S} \in \mathbb{Z}_q^{\tilde{m}\times\tilde{m}}$ *is an Ajtai trapdoor for* $\mathbf{G}$ *(full-rank basis for* $\Lambda^\perp(\mathbf{G})$*). Then* $\mathbf{AT'_A} = \mathbf{0} \mod q$ *and* $\mathbf{T'_A}$ *is full rank over* $\mathbb{R}$*.*

C.1 The Explainable SampleLeft Construction

$\mathsf{SampleLeft}(\mathbf{A}, \mathbf{M}_1, \mathbf{T_A}, \mathbf{u}, \sigma; \mathsf{rb})$**:** Let $\mathbf{T'_A} \in \mathbb{Z}_q^{\tilde{m}\times\tilde{m}}$ denote the first m linearly-independent (in $\mathbb{R}$) columns of $\left[\mathbf{I}_m - \mathbf{T_A}\mathbf{G}^{-1}\mathbf{A} \mid \mathbf{T_A}\mathbf{S}\right]$, where $\mathbf{S} \in \mathbb{Z}_q^{m\times m}$ is the Ajtai trapdoor for $\mathbf{G}$ (full-rank basis for $\Lambda^\perp(\mathbf{G})$) from Lemma 21. Note that $\mathbf{T'_A}$ is an Ajtai trapdoor for $\mathbf{A}$. The sampling algorithm first finds this trapdoor $\mathbf{T'_A}$. It then uses the basis extension algorithm $\mathsf{ExtBasis}$ [52] to construct an Ajtai trapdoor $\mathbf{T'_F} \in \mathbb{Z}^{(m+m_1)\times(m+m_1)}$ for the vector $\mathbf{F} = [\mathbf{A} \mid \mathbf{M}_1]$ such that $\|\mathbf{T'_F}\|_{\mathsf{GS}} = \|\mathbf{T'_A}\|_{\mathsf{GS}}$ ([52, Lemma 3.2]). Crucially, it does this in a *deterministic* manner by calling $\mathbf{T'_F} = \mathsf{DetExtBasis}(\mathbf{T'_A}, [\mathbf{A} \mid \mathbf{M}_1])$ described below:

- For $i = 1, \ldots, m$, it sets $\mathbf{s}'_i := \mathbf{S}[i] \| \mathbf{0} \in \mathbb{Z}^{m+m_1}$, where $\mathbf{S}[i] \in \mathbb{Z}^m$ denotes the vector at the i-th column of the matrix $\mathbf{S}$.
- For $i = 1, \ldots, m_1$, it uses Gaussian elimination to compute a vector $\mathbf{t}_i \in \mathbb{Z}^m$ such that $\mathbf{A}\mathbf{t}_i = -\mathbf{M}_1[i] \in \mathbb{Z}_q^n$, where $\mathbf{M}_1[i]$ denotes the vector at the i-th column of the matrix $\mathbf{M}_1$. It then sets $\mathbf{s}'_{m+i} := \mathbf{t}_i \| \mathbf{e}_i \in \mathbb{Z}^{m+m_1}$, where $\mathbf{e}_i \in \mathbb{Z}^{m_1}$ is the i-th standard basis vector.

Finally, the sampling algorithm returns the output of $\mathsf{SamplePre}(\mathbf{F}, \mathbf{T}'_{\mathbf{F}}, \mathbf{u}, \sigma; \mathsf{rb})$ from Π_{SP}, where $\mathbf{T}_{\mathsf{Ajtai}}$ in [28, Construction 7.6] is replaced by $\mathbf{T}'_{\mathbf{F}}$.

$\mathsf{ExplainSL}(1^\kappa, \mathbf{A}, \mathbf{M}_1, \mathbf{T}_{\mathbf{A}}, \mathbf{u}, \mathbf{v}, \sigma)$**:** The algorithm first obtains

$$\mathbf{T}_{\mathbf{F}} = \mathsf{DetExtBasis}(\mathbf{T}_{\mathbf{A}}, [\mathbf{A} \mid \mathbf{M}_1])$$

deterministically as described above. It then returns the output of

$$\mathsf{ExplainSP}(1^\kappa, \mathbf{F}, \mathbf{T}_{\mathbf{F}}, \mathbf{u}, \mathbf{v}, \sigma)$$

from Π_{SP}, where $\mathbf{T}_{\mathsf{Ajtai}}$ in [28, Construction 7.6] is replaced by $\mathbf{T}'_{\mathbf{F}}$.

C.2 Proof of Theorem 1

Finally, we prove that Π_{SL} described above is an explainable SampleLeft sampler.

Proof of Theorem 1. By Lemma 31, $\mathbf{T}'_{\mathbf{A}}$ is a full-rank Ajtai trapdoor for $\mathbf{A}$, and by [52, Lemma 3.2], $\mathbf{T}'_{\mathbf{F}}$ is an Ajtai trapdoor for $\mathbf{F}$ such that $\|\mathbf{T}'_{\mathbf{F}}\|_{\mathsf{GS}} = \|\mathbf{T}'_{\mathbf{A}}\|_{\mathsf{GS}}$. Moreover, it is full-rank.

Proof of correctness of Π_{SP} requires $\|\mathbf{T}_{\mathbf{A}}\|_\infty \cdot \sigma_{\mathsf{loss}}$ to be lower-bounded by the ℓ_2-norm of the Gram-Schmidt orthogonalization of $\mathbf{T}'_{\mathbf{F}}$ (*i.e.*, $\|\mathbf{T}'_{\mathbf{F}}\|_{\mathsf{GS}}$) as follows:

$$\|\mathbf{T}_{\mathbf{A}}\|_\infty \cdot \sigma_{\mathsf{loss}} \geq \|\mathbf{T}'_{\mathbf{F}}\|_{\mathsf{GS}} \cdot (\log((m+m_1)\lambda) + 2\log(m+m_1) + \log(\log(q)) + 5).$$

Since we know that $\|\mathbf{T}'_{\mathbf{F}}\|_{\mathsf{GS}} = \|\mathbf{T}'_{\mathbf{A}}\|_{\mathsf{GS}}$, we can replace $\|\mathbf{T}'_{\mathbf{F}}\|_{\mathsf{GS}}$ by $\|\mathbf{T}'_{\mathbf{A}}\|_{\mathsf{GS}}$ in the inequality above:

$$\|\mathbf{T}_{\mathbf{A}}\|_\infty \cdot \sigma_{\mathsf{loss}} \geq \|\mathbf{T}'_{\mathbf{A}}\|_{\mathsf{GS}} \cdot (\log((m+m_1)\lambda) + 2\log(m+m_1) + \log(\log(q)) + 5)$$

Now, by the proof of Theorem 11, this is satisfied by

$$\sigma_{\mathsf{loss}}(\lambda, n, m, m_1, q) = 18(m+m_1)^{3/2} \log((m+m_1)\lambda) \log(\log(q))$$

Therefore, by Theorem 11, given this $\sigma_{\mathsf{loss}}(\lambda, n, m, m_1, q)$ and $\rho = \mathsf{poly}(\lambda, n, m + m_1 \log(q)) = \mathsf{poly}(\lambda, n, m, m_1 \log(q))$, Π_{SP} satisfies correctness and explainability per Definition 25:

1. The statistical distance between the distributions $\mathbf{v} := \mathsf{SamplePre}(\mathbf{F}, \mathbf{T}_{\mathbf{F}}, \mathbf{u}, \sigma; \mathsf{rb})$, $\mathsf{rb} \leftarrow \{0,1\}^\rho$ and $\mathbf{v} \leftarrow \mathbf{F}_\sigma^{-1}(\mathbf{u})$ is negligible in λ, and for all $\mathbf{v}$ in the support of $\mathsf{SamplePre}(\mathbf{F}, \mathbf{T}_{\mathbf{F}}, \mathbf{u}, \sigma; \mathsf{rb})$, it holds that $\mathbf{F}\mathbf{v} = \mathbf{u}$.
2. For all $\kappa \in \mathbb{N}$, the statistical distance between the following distributions is bounded by $1/\kappa + \mathsf{negl}(\lambda)$:
   - DSamplePre: Sample $\mathsf{rb} \leftarrow \{0,1\}^\rho$, and set $\mathbf{v} := \mathsf{SamplePre}(1^\lambda, \mathbf{F}, \mathbf{T}_{\mathbf{F}}, \mathbf{u}, \sigma; \mathsf{rb})$. Output $(\mathbf{v}, \mathsf{rb})$.
   - DExplainPre: Sample $r' \leftarrow \{0,1\}^\rho$ and set $\mathbf{v} := \mathsf{SamplePre}(1^\lambda, \mathbf{F}, \mathbf{T}_{\mathbf{F}}, \mathbf{u}, \sigma; \mathsf{rb}')$. Compute $\mathsf{rb} := \mathsf{ExplainSP}(1^\lambda, 1^\kappa, \mathbf{F}, \mathbf{T}_{\mathbf{F}}, \mathbf{u}, \mathbf{v}, \sigma)$, and output $(\mathbf{v}, \mathsf{rb})$.

Finally, since the algorithm DetExtBasis is deterministic, correctness and explainability of Π_{SL} per Definition 12 directly follow from those of Π_{SP}. □